LateralusOS Architecture Reference

API, Subsystems, and Driver Reference

Lateralus Language

bad-antics · April 2026 · Lateralus Language Research

ABSTRACT Lateralus Language -- lateralus.dev

0. Introduction to LateralusOS Reference

Writing an operating system in a high-level language was considered impractical for decades because high-level languages required garbage collectors, runtime overhead, and standard libraries that themselves depended on the OS. Lateralus's freestanding mode changes this calculus: the language's core -- arithmetic, pattern matching, closures, traits, and ownership -- is available with zero runtime dependencies. The standard library is optional and modular. A kernel written in Lateralus gets memory safety, trait-based polymorphism, and expressive pattern matching without paying any runtime tax.

The x86-64 boot sequence begins long before the kernel runs. Firmware (UEFI or legacy BIOS) initializes hardware, selects a boot device, loads the bootloader, and jumps to it in a specific CPU mode. For UEFI-based systems, the bootloader runs in 64-bit long mode with a flat memory map provided by the firmware. The kernel must still configure its own page tables, set up the interrupt descriptor table, initialize the APIC interrupt controller, and map its own code and data before it is ready to run Lateralus-level code. LateralusOS uses a minimal assembly stub for this pre-kernel setup, transitioning to Lateralus as early as possible.

The physical memory allocator is the foundation of all kernel memory management. It must track which physical pages are free and which are in use, supporting allocation requests of various sizes. LateralusOS uses a buddy allocator: physical memory is organized in power-of-two blocks, and splitting and merging of blocks is O(log n) in the memory size. The allocator is initialized by parsing the memory map provided by the bootloader, marking firmware-reserved and kernel-occupied regions as allocated. After initialization, all kernel heap allocations and virtual memory mappings draw from the buddy allocator's pool.

Implementation Details

Virtual memory allows the kernel to present each process with the illusion of a private, contiguous address space independent of the physical layout of RAM. The MMU translates virtual addresses to physical addresses using a hierarchy of page tables. On x86-64, the hierarchy is four levels deep: PML4, PDPT, PD, PT. Each level is a 4KB page containing 512 8-byte entries. A virtual address is split into five fields: the PML4 index, the PDPT index, the PD index, the PT index, and the page offset. Only present pages have valid translations; an access to a non-present page triggers a page fault, which the kernel handles by mapping the page or terminating the process.

The kernel scheduler determines which thread runs on which CPU at each moment. LateralusOS implements a Completely Fair Scheduler (CFS) variant inspired by Linux's CFS. Each thread has a virtual runtime -- the total CPU time it has consumed, weighted by its priority. The scheduler always runs the thread with the lowest virtual runtime, which ensures fairness over time. Priority is implemented as a weight: a high-priority thread accumulates virtual runtime more slowly, so it gets proportionally more CPU time. SMP support requires a per-CPU run queue to avoid lock contention, with periodic load balancing to migrate threads between CPUs.

```
// Kernel entry point (after assembly stub sets up stack)
@no_mangle
pub fn kernel_main(boot_info: &BootInfo) {
    serial_init()
    println("[BOOT] LateralusOS kernel_main entered")
    memory::init(boot_info.memory_map)
    interrupts::init_idt()
```

```
    apic::init()
    scheduler::init()
    vfs::init()
    drivers::init_all()
    println("[BOOT] All subsystems initialized")
    scheduler::start()  // does not return
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

1. Why Use a High-Level Language for OS Development

The x86-64 boot sequence begins long before the kernel runs. Firmware (UEFI or legacy BIOS) initializes hardware, selects a boot device, loads the bootloader, and jumps to it in a specific CPU mode. For UEFI-based systems, the bootloader runs in 64-bit long mode with a flat memory map provided by the firmware. The kernel must still configure its own page tables, set up the interrupt descriptor table, initialize the APIC interrupt controller, and map its own code and data before it is ready to run Lateralus-level code. LateralusOS uses a minimal assembly stub for this pre-kernel setup, transitioning to Lateralus as early as possible.

The physical memory allocator is the foundation of all kernel memory management. It must track which physical pages are free and which are in use, supporting allocation requests of various sizes. LateralusOS uses a buddy allocator: physical memory is organized in power-of-two blocks, and splitting and merging of blocks is O(log n) in the memory size. The allocator is initialized by parsing the memory map provided by the bootloader, marking firmware-reserved and kernel-occupied regions as allocated. After initialization, all kernel heap allocations and virtual memory mappings draw from the buddy allocator's pool.

Virtual memory allows the kernel to present each process with the illusion of a private, contiguous address space independent of the physical layout of RAM. The MMU translates virtual addresses to physical addresses using a hierarchy of page tables. On x86-64, the hierarchy is four levels deep: PML4, PDPT, PD, PT. Each level is a 4KB page containing 512 8-byte entries. A virtual address is split into five fields: the PML4 index, the PDPT index, the PD index, the PT index, and the page offset. Only present pages have valid translations; an access to a non-present page triggers a page fault, which the kernel handles by mapping the page or terminating the process.

Implementation Details

The kernel scheduler determines which thread runs on which CPU at each moment. LateralusOS implements a Completely Fair Scheduler (CFS) variant inspired by Linux's CFS. Each thread has a virtual runtime -- the total CPU time it has consumed, weighted by its priority. The scheduler always runs the thread with the lowest virtual runtime, which ensures fairness over time. Priority is implemented as a weight: a high-priority thread accumulates virtual runtime more slowly, so it gets proportionally more

CPU time. SMP support requires a per-CPU run queue to avoid lock contention, with periodic load balancing to migrate threads between CPUs.

Interrupt handling is the mechanism by which hardware events reach the kernel. When a keyboard is pressed, a timer fires, or a disk read completes, the hardware asserts an interrupt request line. The APIC (Advanced Programmable Interrupt Controller) arbitrates between multiple interrupt sources and delivers them to a CPU, which saves its current state and jumps to the appropriate interrupt handler via the IDT (Interrupt Descriptor Table). Interrupt handlers must be fast, non-blocking, and careful about which kernel locks they acquire. LateralusOS's type system enforces these constraints: functions marked as interrupt handlers are restricted to a subset of the kernel API.

```
// Page table entry construction
const PAGE_PRESENT:  u64 = 1 << 0
const PAGE_WRITABLE: u64 = 1 << 1
const PAGE_USER:     u64 = 1 << 2
const PAGE_NOEXEC:   u64 = 1 << 63

fn make_pte(phys: u64, flags: u64) -> u64 {
    // Align to 4KB boundary, set flags
    return (phys & 0xFFFF_FFFF_F000) | flags
}

fn map_page(virt: u64, phys: u64, writable: bool) {
    let flags = PAGE_PRESENT | (if writable { PAGE_WRITABLE } else { 0 })
    PT_ROOT.insert(virt, make_pte(phys, flags))
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

2. Lateralus Freestanding Mode

The physical memory allocator is the foundation of all kernel memory management. It must track which physical pages are free and which are in use, supporting allocation requests of various sizes. LateralusOS uses a buddy allocator: physical memory is organized in power-of-two blocks, and splitting and merging of blocks is O(log n) in the memory size. The allocator is initialized by parsing the memory map provided by the bootloader, marking firmware-reserved and kernel-occupied regions as allocated. After initialization, all kernel heap allocations and virtual memory mappings draw from the buddy allocator's pool.

Virtual memory allows the kernel to present each process with the illusion of a private, contiguous address space independent of the physical layout of RAM. The MMU translates virtual addresses to physical addresses using a hierarchy of page tables. On x86-64, the hierarchy is four levels deep: PML4, PDPT, PD, PT. Each level is a 4KB page containing 512 8-byte entries. A virtual address is split into five fields: the PML4 index, the PDPT index, the PD index, the PT index, and the page offset. Only

present pages have valid translations; an access to a non-present page triggers a page fault, which the kernel handles by mapping the page or terminating the process.

The kernel scheduler determines which thread runs on which CPU at each moment. LateralusOS implements a Completely Fair Scheduler (CFS) variant inspired by Linux's CFS. Each thread has a virtual runtime -- the total CPU time it has consumed, weighted by its priority. The scheduler always runs the thread with the lowest virtual runtime, which ensures fairness over time. Priority is implemented as a weight: a high-priority thread accumulates virtual runtime more slowly, so it gets proportionally more CPU time. SMP support requires a per-CPU run queue to avoid lock contention, with periodic load balancing to migrate threads between CPUs.

Implementation Details

Interrupt handling is the mechanism by which hardware events reach the kernel. When a keyboard is pressed, a timer fires, or a disk read completes, the hardware asserts an interrupt request line. The APIC (Advanced Programmable Interrupt Controller) arbitrates between multiple interrupt sources and delivers them to a CPU, which saves its current state and jumps to the appropriate interrupt handler via the IDT (Interrupt Descriptor Table). Interrupt handlers must be fast, non-blocking, and careful about which kernel locks they acquire. LateralusOS's type system enforces these constraints: functions marked as interrupt handlers are restricted to a subset of the kernel API.

The Virtual File System (VFS) layer provides a uniform interface to all file systems. Applications use the same system calls (open, read, write, close, stat) regardless of whether the underlying storage is a local disk, a network share, a FUSE user-space file system, or a synthetic file system like /proc. The VFS layer defines abstract types: Inode (a file or directory), Dentry (a name-to-inode mapping), and File (an open file descriptor). Concrete file system drivers implement these abstractions. LateralusOS supports FAT32 for boot media compatibility and ext2 for general storage, with the VFS layer making both appear identical to user programs.

```
// CFS scheduler -- pick thread with min vruntime
fn schedule(cpu: &mut CpuState) {
    let next = cpu.run_queue
        |> min_by(fn(t) { t.vruntime })  // O(log n) with red-black tree
    match next {
        Some(thread) => {
            cpu.run_queue.remove(thread.id)
            cpu.current = thread
            context_switch(cpu.current, thread)
        },
        None => { cpu.current = IDLE_THREAD },

    }
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

3. Boot Process: UEFI vs. Legacy BIOS

Virtual memory allows the kernel to present each process with the illusion of a private, contiguous address space independent of the physical layout of RAM. The MMU translates virtual addresses to physical addresses using a hierarchy of page tables. On x86-64, the hierarchy is four levels deep: PML4, PDPT, PD, PT. Each level is a 4KB page containing 512 8-byte entries. A virtual address is split into five fields: the PML4 index, the PDPT index, the PD index, the PT index, and the page offset. Only present pages have valid translations; an access to a non-present page triggers a page fault, which the kernel handles by mapping the page or terminating the process.

The kernel scheduler determines which thread runs on which CPU at each moment. LateralusOS implements a Completely Fair Scheduler (CFS) variant inspired by Linux's CFS. Each thread has a virtual runtime -- the total CPU time it has consumed, weighted by its priority. The scheduler always runs the thread with the lowest virtual runtime, which ensures fairness over time. Priority is implemented as a weight: a high-priority thread accumulates virtual runtime more slowly, so it gets proportionally more CPU time. SMP support requires a per-CPU run queue to avoid lock contention, with periodic load balancing to migrate threads between CPUs.

Interrupt handling is the mechanism by which hardware events reach the kernel. When a keyboard is pressed, a timer fires, or a disk read completes, the hardware asserts an interrupt request line. The APIC (Advanced Programmable Interrupt Controller) arbitrates between multiple interrupt sources and delivers them to a CPU, which saves its current state and jumps to the appropriate interrupt handler via the IDT (Interrupt Descriptor Table). Interrupt handlers must be fast, non-blocking, and careful about which kernel locks they acquire. LateralusOS's type system enforces these constraints: functions marked as interrupt handlers are restricted to a subset of the kernel API.

Implementation Details

The Virtual File System (VFS) layer provides a uniform interface to all file systems. Applications use the same system calls (open, read, write, close, stat) regardless of whether the underlying storage is a local disk, a network share, a FUSE user-space file system, or a synthetic file system like /proc. The VFS layer defines abstract types: Inode (a file or directory), Dentry (a name-to-inode mapping), and File (an open file descriptor). Concrete file system drivers implement these abstractions. LateralusOS supports FAT32 for boot media compatibility and ext2 for general storage, with the VFS layer making both appear identical to user programs.

Device drivers are the most error-prone component of any operating system. They run in privileged kernel mode but interact with hardware that may misbehave, return unexpected data, or fail silently. LateralusOS's driver framework uses Lateralus traits to define safe interfaces: a block device driver implements the BlockDevice trait (read_sector, write_sector, sector_count), a character device implements CharDevice (read_byte, write_byte), and a network interface implements NetDevice (send_frame, recv_frame, get_mac). The trait system ensures that all required methods are implemented and that callers cannot accidentally call driver-specific methods through the generic interface.

```
// VFS: trait for file system drivers
trait FileSystem {
    fn read(&self, inode: &Inode, buf: &mut [u8], offset: u64) -> Result[usize, KernelError]
    fn write(&mut self, inode: &Inode, buf: &[u8], offset: u64) -> Result[usize, KernelError]
    fn lookup(&self, dir: &Inode, name: str) -> Result[Inode, KernelError]
    fn readdir(&self, dir: &Inode) -> Result[list[DirEntry], KernelError]
    fn create(&mut self, dir: &Inode, name: str, kind: InodeKind) -> Result[Inode, KernelError]

    fn unlink(&mut self, dir: &Inode, name: str) -> Result[(), KernelError]
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

4. Multiboot2 Header and GRUB Integration

The kernel scheduler determines which thread runs on which CPU at each moment. LateralusOS implements a Completely Fair Scheduler (CFS) variant inspired by Linux's CFS. Each thread has a virtual runtime -- the total CPU time it has consumed, weighted by its priority. The scheduler always runs the thread with the lowest virtual runtime, which ensures fairness over time. Priority is implemented as a weight: a high-priority thread accumulates virtual runtime more slowly, so it gets proportionally more CPU time. SMP support requires a per-CPU run queue to avoid lock contention, with periodic load balancing to migrate threads between CPUs.

Interrupt handling is the mechanism by which hardware events reach the kernel. When a keyboard is pressed, a timer fires, or a disk read completes, the hardware asserts an interrupt request line. The APIC (Advanced Programmable Interrupt Controller) arbitrates between multiple interrupt sources and delivers them to a CPU, which saves its current state and jumps to the appropriate interrupt handler via the IDT (Interrupt Descriptor Table). Interrupt handlers must be fast, non-blocking, and careful about which kernel locks they acquire. LateralusOS's type system enforces these constraints: functions marked as interrupt handlers are restricted to a subset of the kernel API.

The Virtual File System (VFS) layer provides a uniform interface to all file systems. Applications use the same system calls (open, read, write, close, stat) regardless of whether the underlying storage is a local disk, a network share, a FUSE user-space file system, or a synthetic file system like /proc. The VFS layer defines abstract types: Inode (a file or directory), Dentry (a name-to-inode mapping), and File (an open file descriptor). Concrete file system drivers implement these abstractions. LateralusOS supports FAT32 for boot media compatibility and ext2 for general storage, with the VFS layer making both appear identical to user programs.

Implementation Details

Device drivers are the most error-prone component of any operating system. They run in privileged kernel mode but interact with hardware that may misbehave, return unexpected data, or fail silently. LateralusOS's driver framework uses Lateralus traits to define safe interfaces: a block device driver implements the BlockDevice trait (read_sector, write_sector, sector_count), a character device implements CharDevice (read_byte, write_byte), and a network interface implements NetDevice (send_frame, recv_frame, get_mac). The trait system ensures that all required methods are implemented and that callers cannot accidentally call driver-specific methods through the generic interface.

The RISC-V instruction set architecture offers a clean, modular design that makes OS implementation particularly straightforward. The ISA is divided into standard extensions: I (integer), M (multiply/divide), A (atomic), F (single-precision float), D (double-precision float), C (compressed 16-bit instructions). The

privileged architecture defines three privilege levels: Machine (M-mode, highest), Supervisor (S-mode, for OS kernels), and User (U-mode, for applications). The RISC-V Supervisor Binary Interface (SBI) provides a standard firmware abstraction layer, allowing OS kernels to interact with platform hardware through well-defined firmware calls rather than direct hardware access.

```
// RISC-V SBI call wrapper
fn sbi_call(eid: u32, fid: u32, a0: u64, a1: u64, a2: u64) -> SbiReturn {
    // Place args in registers a0-a6, ecall into M-mode firmware
    let (error, value): (i64, i64)
    asm {
        "li a7, {eid}"
        "li a6, {fid}"

        "mv a0, {a0}"
        "mv a1, {a1}"
        "mv a2, {a2}"
        "ecall"
        "mv {error}, a0"
        "mv {value}, a1"
    }
    return SbiReturn { error, value }
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

5. Early Assembly: Stack Setup and Long Mode

Interrupt handling is the mechanism by which hardware events reach the kernel. When a keyboard is pressed, a timer fires, or a disk read completes, the hardware asserts an interrupt request line. The APIC (Advanced Programmable Interrupt Controller) arbitrates between multiple interrupt sources and delivers them to a CPU, which saves its current state and jumps to the appropriate interrupt handler via the IDT (Interrupt Descriptor Table). Interrupt handlers must be fast, non-blocking, and careful about which kernel locks they acquire. LateralusOS's type system enforces these constraints: functions marked as interrupt handlers are restricted to a subset of the kernel API.

The Virtual File System (VFS) layer provides a uniform interface to all file systems. Applications use the same system calls (open, read, write, close, stat) regardless of whether the underlying storage is a local disk, a network share, a FUSE user-space file system, or a synthetic file system like /proc. The VFS layer defines abstract types: Inode (a file or directory), Dentry (a name-to-inode mapping), and File (an open file descriptor). Concrete file system drivers implement these abstractions. LateralusOS supports FAT32 for boot media compatibility and ext2 for general storage, with the VFS layer making both appear identical to user programs.

Device drivers are the most error-prone component of any operating system. They run in privileged kernel mode but interact with hardware that may misbehave, return unexpected data, or fail silently. LateralusOS's driver framework uses Lateralus traits to define safe interfaces: a block device driver implements the BlockDevice trait (read_sector, write_sector, sector_count), a character device implements CharDevice (read_byte, write_byte), and a network interface implements NetDevice (send_frame, recv_frame, get_mac). The trait system ensures that all required methods are implemented and that callers cannot accidentally call driver-specific methods through the generic interface.

Implementation Details

The RISC-V instruction set architecture offers a clean, modular design that makes OS implementation particularly straightforward. The ISA is divided into standard extensions: I (integer), M (multiply/divide), A (atomic), F (single-precision float), D (double-precision float), C (compressed 16-bit instructions). The privileged architecture defines three privilege levels: Machine (M-mode, highest), Supervisor (S-mode, for OS kernels), and User (U-mode, for applications). The RISC-V Supervisor Binary Interface (SBI) provides a standard firmware abstraction layer, allowing OS kernels to interact with platform hardware through well-defined firmware calls rather than direct hardware access.

OS performance optimization requires careful attention to hardware characteristics. Modern CPUs execute instructions out-of-order and speculatively, making the cost of a branch misprediction 15-20 cycles. Memory accesses are dominated by cache behavior: an L1 hit takes ~4 cycles, an L2 hit ~12, an L3 hit ~40, and a DRAM access ~200. The kernel's hot paths -- interrupt dispatch, system call entry, scheduler context switch -- are heavily optimized to minimize cache misses and branch mispredictions. LateralusOS's scheduler data structures are designed to fit in cache: the per-CPU run queue and per-thread control block are sized to avoid cache line splits, and frequently accessed fields are

grouped together.

```
// Kernel entry point (after assembly stub sets up stack)
@no_mangle
pub fn kernel_main(boot_info: &BootInfo) {
    serial_init()
    println("[BOOT] LateralusOS kernel_main entered")
    memory::init(boot_info.memory_map)
    interrupts::init_idt()
    apic::init()
    scheduler::init()
    vfs::init()
    drivers::init_all()
    println("[BOOT] All subsystems initialized")
    scheduler::start()  // does not return
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

6. Transitioning from Assembly to Lateralus

The Virtual File System (VFS) layer provides a uniform interface to all file systems. Applications use the same system calls (open, read, write, close, stat) regardless of whether the underlying storage is a local disk, a network share, a FUSE user-space file system, or a synthetic file system like /proc. The VFS layer defines abstract types: Inode (a file or directory), Dentry (a name-to-inode mapping), and File (an open file descriptor). Concrete file system drivers implement these abstractions. LateralusOS supports FAT32 for boot media compatibility and ext2 for general storage, with the VFS layer making both appear identical to user programs.

Device drivers are the most error-prone component of any operating system. They run in privileged kernel mode but interact with hardware that may misbehave, return unexpected data, or fail silently. LateralusOS's driver framework uses Lateralus traits to define safe interfaces: a block device driver implements the BlockDevice trait (read_sector, write_sector, sector_count), a character device implements CharDevice (read_byte, write_byte), and a network interface implements NetDevice (send_frame, recv_frame, get_mac). The trait system ensures that all required methods are implemented and that callers cannot accidentally call driver-specific methods through the generic interface.

The RISC-V instruction set architecture offers a clean, modular design that makes OS implementation particularly straightforward. The ISA is divided into standard extensions: I (integer), M (multiply/divide), A (atomic), F (single-precision float), D (double-precision float), C (compressed 16-bit instructions). The privileged architecture defines three privilege levels: Machine (M-mode, highest), Supervisor (S-mode, for OS kernels), and User (U-mode, for applications). The RISC-V Supervisor Binary Interface (SBI) provides a standard firmware abstraction layer, allowing OS kernels to interact with platform hardware through well-defined firmware calls rather than direct hardware access.

Implementation Details

OS performance optimization requires careful attention to hardware characteristics. Modern CPUs execute instructions out-of-order and speculatively, making the cost of a branch misprediction 15-20 cycles. Memory accesses are dominated by cache behavior: an L1 hit takes ~4 cycles, an L2 hit ~12, an L3 hit ~40, and a DRAM access ~200. The kernel's hot paths -- interrupt dispatch, system call entry, scheduler context switch -- are heavily optimized to minimize cache misses and branch mispredictions. LateralusOS's scheduler data structures are designed to fit in cache: the per-CPU run queue and per-thread control block are sized to avoid cache line splits, and frequently accessed fields are grouped together.

Writing an operating system in a high-level language was considered impractical for decades because high-level languages required garbage collectors, runtime overhead, and standard libraries that themselves depended on the OS. Lateralus's freestanding mode changes this calculus: the language's core -- arithmetic, pattern matching, closures, traits, and ownership -- is available with zero runtime dependencies. The standard library is optional and modular. A kernel written in Lateralus gets memory safety, trait-based polymorphism, and expressive pattern matching without paying any runtime tax.

```
// Page table entry construction
const PAGE_PRESENT:  u64 = 1 << 0
const PAGE_WRITABLE: u64 = 1 << 1
const PAGE_USER:     u64 = 1 << 2
const PAGE_NOEXEC:   u64 = 1 << 63

fn make_pte(phys: u64, flags: u64) -> u64 {
    // Align to 4KB boundary, set flags
```

```
    return (phys & 0xFFFF_FFFF_F000) | flags
}

fn map_page(virt: u64, phys: u64, writable: bool) {
    let flags = PAGE_PRESENT | (if writable { PAGE_WRITABLE } else { 0 })
    PT_ROOT.insert(virt, make_pte(phys, flags))
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

7. Physical Memory Map (E820)

Device drivers are the most error-prone component of any operating system. They run in privileged kernel mode but interact with hardware that may misbehave, return unexpected data, or fail silently. LateralusOS's driver framework uses Lateralus traits to define safe interfaces: a block device driver implements the BlockDevice trait (read_sector, write_sector, sector_count), a character device implements CharDevice (read_byte, write_byte), and a network interface implements NetDevice (send_frame, recv_frame, get_mac). The trait system ensures that all required methods are implemented and that callers cannot accidentally call driver-specific methods through the generic interface.

The RISC-V instruction set architecture offers a clean, modular design that makes OS implementation particularly straightforward. The ISA is divided into standard extensions: I (integer), M (multiply/divide), A (atomic), F (single-precision float), D (double-precision float), C (compressed 16-bit instructions). The privileged architecture defines three privilege levels: Machine (M-mode, highest), Supervisor (S-mode, for OS kernels), and User (U-mode, for applications). The RISC-V Supervisor Binary Interface (SBI) provides a standard firmware abstraction layer, allowing OS kernels to interact with platform hardware through well-defined firmware calls rather than direct hardware access.

OS performance optimization requires careful attention to hardware characteristics. Modern CPUs execute instructions out-of-order and speculatively, making the cost of a branch misprediction 15-20 cycles. Memory accesses are dominated by cache behavior: an L1 hit takes ~4 cycles, an L2 hit ~12, an L3 hit ~40, and a DRAM access ~200. The kernel's hot paths -- interrupt dispatch, system call entry, scheduler context switch -- are heavily optimized to minimize cache misses and branch mispredictions. LateralusOS's scheduler data structures are designed to fit in cache: the per-CPU run queue and per-thread control block are sized to avoid cache line splits, and frequently accessed fields are grouped together.

Implementation Details

Writing an operating system in a high-level language was considered impractical for decades because high-level

languages required garbage collectors, runtime overhead, and standard libraries that themselves depended on the OS. Lateralus's freestanding mode changes this calculus: the language's core -- arithmetic, pattern matching, closures, traits, and ownership -- is available with zero runtime dependencies. The standard library is optional and modular. A kernel written in Lateralus gets memory safety, trait-based polymorphism, and expressive pattern matching without paying any runtime tax.

The x86-64 boot sequence begins long before the kernel runs. Firmware (UEFI or legacy BIOS) initializes hardware, selects a boot device, loads the bootloader, and jumps to it in a specific CPU mode. For UEFI-based systems, the bootloader runs in 64-bit long mode with a flat memory map provided by the firmware. The kernel must still configure its own page tables, set up the interrupt descriptor table, initialize the APIC interrupt controller, and map its own code and data before it is ready to run Lateralus-level code. LateralusOS uses a minimal assembly stub for this pre-kernel setup, transitioning to Lateralus as early as possible.

```
// CFS scheduler -- pick thread with min vruntime
fn schedule(cpu: &mut CpuState) {
    let next = cpu.run_queue
        |> min_by(fn(t) { t.vruntime })  // O(log n) with red-black tree
    match next {
        Some(thread) => {
            cpu.run_queue.remove(thread.id)
            cpu.current = thread
            context_switch(cpu.current, thread)
        },
        None => { cpu.current = IDLE_THREAD },
    }
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

8. Buddy Allocator Design

The RISC-V instruction set architecture offers a clean, modular design that makes OS implementation particularly straightforward. The ISA is divided into standard extensions: I (integer), M (multiply/divide), A (atomic), F (single-precision float), D (double-precision float), C (compressed 16-bit instructions). The privileged architecture defines three privilege levels: Machine (M-mode, highest), Supervisor (S-mode, for OS kernels), and User (U-mode, for applications). The RISC-V Supervisor Binary Interface (SBI) provides a standard firmware abstraction layer, allowing OS kernels to interact with platform hardware through well-defined firmware calls rather than direct hardware access.

OS performance optimization requires careful attention to hardware characteristics. Modern CPUs execute instructions out-of-order and speculatively, making the cost of a branch misprediction 15-20 cycles. Memory accesses are dominated by cache behavior: an L1 hit takes ~4 cycles, an L2 hit ~12, an

L3 hit ~40, and a DRAM access ~200. The kernel's hot paths -- interrupt dispatch, system call entry, scheduler context switch -- are heavily optimized to minimize cache misses and branch mispredictions. LateralusOS's scheduler data structures are designed to fit in cache: the per-CPU run queue and per-thread control block are sized to avoid cache line splits, and frequently accessed fields are grouped together.

Writing an operating system in a high-level language was considered impractical for decades because high-level languages required garbage collectors, runtime overhead, and standard libraries that themselves depended on the OS. Lateralus's freestanding mode changes this calculus: the language's core -- arithmetic, pattern matching, closures, traits, and ownership -- is available with zero runtime dependencies. The standard library is optional and modular. A kernel written in Lateralus gets memory safety, trait-based polymorphism, and expressive pattern matching without paying any

runtime tax.

Implementation Details

The x86-64 boot sequence begins long before the kernel runs. Firmware (UEFI or legacy BIOS) initializes hardware, selects a boot device, loads the bootloader, and jumps to it in a specific CPU mode. For UEFI-based systems, the bootloader runs in 64-bit long mode with a flat memory map provided by the firmware. The kernel must still configure its own page tables, set up the interrupt descriptor table, initialize the APIC interrupt controller, and map its own code and data before it is ready to run Lateralus-level code. LateralusOS uses a minimal assembly stub for this pre-kernel setup, transitioning to Lateralus as early as possible.

The physical memory allocator is the foundation of all kernel memory management. It must track which physical pages are free and which are in use, supporting allocation requests of various sizes. LateralusOS uses a buddy allocator: physical memory is organized in power-of-two blocks, and splitting and merging of blocks is O(log n) in the memory size. The allocator is initialized by parsing the memory map provided by the bootloader, marking firmware-reserved and kernel-occupied regions as allocated. After initialization, all kernel heap allocations and virtual memory mappings draw from the buddy allocator's pool.

```
// VFS: trait for file system drivers
trait FileSystem {
    fn read(&self, inode: &Inode, buf: &mut [u8], offset: u64) -> Result[usize, KernelError]
    fn write(&mut self, inode: &Inode, buf: &[u8], offset: u64) -> Result[usize, KernelError]
    fn lookup(&self, dir: &Inode, name: str) -> Result[Inode, KernelError]
    fn readdir(&self, dir: &Inode) -> Result[list[DirEntry], KernelError]
    fn create(&mut self, dir: &Inode, name: str, kind: InodeKind) -> Result[Inode, KernelError]
    fn unlink(&mut self, dir: &Inode, name: str) -> Result[(), KernelError]
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

9. Slab Allocator for Kernel Objects

OS performance optimization requires careful attention to hardware characteristics. Modern CPUs execute instructions out-of-order and speculatively, making the cost of a branch misprediction 15-20 cycles. Memory accesses are dominated by cache behavior: an L1 hit takes ~4 cycles, an L2 hit ~12, an L3 hit ~40, and a DRAM access ~200. The kernel's hot paths -- interrupt dispatch, system call entry, scheduler context switch -- are heavily optimized to minimize cache misses and branch mispredictions. LateralusOS's scheduler data structures are designed to fit in cache: the per-CPU run queue and per-thread control block are sized to avoid cache line splits, and frequently accessed fields are grouped together.

Writing an operating system in a high-level language was considered impractical for decades because high-level languages required garbage collectors, runtime overhead, and standard libraries that themselves depended on the OS. Lateralus's freestanding mode changes this calculus: the language's core -- arithmetic, pattern matching, closures, traits, and ownership -- is available with zero runtime dependencies. The standard library is optional and modular. A kernel written in Lateralus gets memory safety, trait-based polymorphism, and expressive pattern matching without paying any runtime tax.

The x86-64 boot sequence begins long before the kernel runs. Firmware (UEFI or legacy BIOS) initializes hardware, selects a boot device, loads the bootloader, and jumps to it in a specific CPU mode. For UEFI-based systems, the bootloader runs in 64-bit long mode with a flat memory map provided by the firmware. The kernel must still configure its own page tables, set up the interrupt descriptor table, initialize the APIC interrupt controller, and map its own code and

data before it is ready to run Lateralus-level code. LateralusOS uses a minimal assembly stub for this pre-kernel setup, transitioning to Lateralus as early as possible.

Implementation Details

The physical memory allocator is the foundation of all kernel memory management. It must track which physical pages are free and which are in use, supporting allocation requests of various sizes. LateralusOS uses a buddy allocator: physical memory is organized in power-of-two blocks, and splitting and merging of blocks is O(log n) in the memory size. The allocator is initialized by parsing the memory map provided by the bootloader, marking firmware-reserved and kernel-occupied regions as allocated. After initialization, all kernel heap allocations and virtual memory mappings draw from the buddy allocator's pool.

Virtual memory allows the kernel to present each process with the illusion of a private, contiguous address space independent of the physical layout of RAM. The MMU translates virtual addresses to physical addresses using a hierarchy of page tables. On x86-64, the hierarchy is four levels deep: PML4, PDPT, PD, PT. Each level is a 4KB page containing 512 8-byte entries. A virtual address is split into five fields: the PML4 index, the PDPT index, the PD index, the PT index, and the page offset. Only present pages have valid translations; an access to a non-present page triggers a page fault, which the kernel handles by mapping the page or terminating the process.

```
// RISC-V SBI call wrapper
fn sbi_call(eid: u32, fid: u32, a0: u64, a1: u64, a2: u64) -> SbiReturn {
    // Place args in registers a0-a6, ecall into M-mode firmware
    let (error, value): (i64, i64)
    asm {
        "li a7, {eid}"
        "li a6, {fid}"
        "mv a0, {a0}"
        "mv a1, {a1}"
        "mv a2, {a2}"
```

```
        "ecall"
        "mv {error}, a0"
        "mv {value}, a1"
    }
    return SbiReturn { error, value }
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

10. Page Tables: PML4, PDPT, PD, PT

Writing an operating system in a high-level language was considered impractical for decades because high-level languages required garbage collectors, runtime overhead, and standard libraries that themselves depended on the OS. Lateralus's freestanding mode changes this calculus: the language's core -- arithmetic, pattern matching, closures, traits, and ownership -- is available with zero runtime dependencies. The standard library is optional and modular. A kernel written in Lateralus gets memory safety, trait-based polymorphism, and expressive pattern matching without paying any runtime tax.

The x86-64 boot sequence begins long before the kernel runs. Firmware (UEFI or legacy BIOS) initializes hardware, selects a boot device, loads the bootloader, and jumps to it in a specific CPU mode. For UEFI-based systems, the bootloader runs in 64-bit long mode with a flat memory map provided by the firmware. The kernel must still configure its own page tables, set up the interrupt descriptor table, initialize the APIC interrupt controller, and map its own code and data before it is ready to run Lateralus-level code. LateralusOS uses a minimal assembly stub for this pre-kernel setup,

transitioning to Lateralus as early as possible.

The physical memory allocator is the foundation of all kernel memory management. It must track which physical pages are free and which are in use, supporting allocation requests of various sizes. LateralusOS uses a buddy allocator: physical memory is organized in power-of-two blocks, and splitting and merging of blocks is O(log n) in the memory size. The allocator is initialized by parsing the memory map provided by the bootloader, marking firmware-reserved and kernel-occupied regions as allocated. After initialization, all kernel heap allocations and virtual memory mappings draw from the buddy allocator's pool.

Implementation Details

Virtual memory allows the kernel to present each process with the illusion of a private, contiguous address space independent of the physical layout of RAM. The MMU translates virtual addresses to physical addresses using a hierarchy of page tables. On x86-64, the hierarchy is four levels deep: PML4, PDPT, PD, PT. Each level is a 4KB page containing 512 8-byte entries. A virtual address is split into five fields: the PML4 index, the PDPT index, the PD index, the PT index, and the page offset. Only present pages have valid translations; an access to a non-present page triggers a page fault, which the

kernel handles by mapping the page or terminating the process.

The kernel scheduler determines which thread runs on which CPU at each moment. LateralusOS implements a Completely Fair Scheduler (CFS) variant inspired by Linux's CFS. Each thread has a virtual runtime -- the total CPU time it has consumed, weighted by its priority. The scheduler always runs the thread with the lowest virtual runtime, which ensures fairness over time. Priority is implemented as a weight: a high-priority thread accumulates virtual runtime more slowly, so it gets proportionally more CPU time. SMP support requires a per-CPU run queue to avoid lock contention, with periodic load balancing to migrate threads between CPUs.

```
// Kernel entry point (after assembly stub sets up stack)
@no_mangle
pub fn kernel_main(boot_info: &BootInfo) {
    serial_init()
    println("[BOOT] LateralusOS kernel_main entered")
    memory::init(boot_info.memory_map)
    interrupts::init_idt()
    apic::init()
    scheduler::init()
    vfs::init()
    drivers::init_all()
    println("[BOOT] All subsystems initialized")
    scheduler::start()  // does not return
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

11. Virtual Memory Layout (x86-64)

The x86-64 boot sequence begins long before the kernel runs. Firmware (UEFI or legacy BIOS) initializes hardware, selects a boot device, loads the bootloader, and jumps to it in a specific CPU mode. For UEFI-based systems, the bootloader runs in 64-bit long mode with a flat memory map provided by the firmware. The kernel must still configure its own page tables, set up the interrupt descriptor table, initialize the APIC interrupt controller, and map its own code and data before it is ready to run Lateralus-level code. LateralusOS uses a minimal assembly stub for this pre-kernel setup, transitioning to Lateralus as early as possible.

The physical memory allocator is the foundation of all kernel memory management. It must track which physical pages

are free and which are in use, supporting allocation requests of various sizes. LateralusOS uses a buddy allocator: physical memory is organized in power-of-two blocks, and splitting and merging of blocks is O(log n) in the memory size. The allocator is initialized by parsing the memory map provided by the bootloader, marking firmware-reserved and kernel-occupied regions as allocated. After

initialization, all kernel heap allocations and virtual memory mappings draw from the buddy allocator's pool.

Virtual memory allows the kernel to present each process with the illusion of a private, contiguous address space independent of the physical layout of RAM. The MMU translates virtual addresses to physical addresses using a hierarchy of page tables. On x86-64, the hierarchy is four levels deep: PML4, PDPT, PD, PT. Each level is a 4KB page containing 512 8-byte entries. A virtual address is split into five fields: the PML4 index, the PDPT index, the PD index, the PT index, and the page offset. Only present pages have valid translations; an access to a non-present page triggers a page fault, which the kernel handles by mapping the page or terminating the process.

Implementation Details

The kernel scheduler determines which thread runs on which CPU at each moment. LateralusOS implements a Completely Fair Scheduler (CFS) variant inspired by Linux's CFS. Each thread has a virtual runtime -- the total CPU time it has consumed, weighted by its priority. The scheduler always runs the thread with the lowest virtual runtime, which ensures fairness over time. Priority is implemented as a weight: a high-priority thread accumulates virtual runtime more slowly, so it gets proportionally more CPU time. SMP support requires a per-CPU run queue to avoid lock contention, with periodic load balancing to migrate threads between CPUs.

Interrupt handling is the mechanism by which hardware events reach the kernel. When a keyboard is pressed, a timer fires, or a disk read completes, the hardware asserts an interrupt request line. The APIC (Advanced Programmable Interrupt Controller) arbitrates between multiple interrupt sources and delivers them to a CPU, which saves its current state and jumps to the appropriate interrupt handler via the IDT (Interrupt Descriptor Table). Interrupt handlers must be fast, non-blocking, and careful about which kernel locks they acquire. LateralusOS's type system enforces these constraints: functions marked as interrupt handlers are restricted to a subset of the kernel API.

```
// Page table entry construction
const PAGE_PRESENT:  u64 = 1 << 0
const PAGE_WRITABLE: u64 = 1 << 1
const PAGE_USER:     u64 = 1 << 2
const PAGE_NOEXEC:   u64 = 1 << 63

fn make_pte(phys: u64, flags: u64) -> u64 {
    // Align to 4KB boundary, set flags
    return (phys & 0xFFFF_FFFF_F000) | flags
}

fn map_page(virt: u64, phys: u64, writable: bool) {
    let flags = PAGE_PRESENT | (if writable { PAGE_WRITABLE } else { 0 })
    PT_ROOT.insert(virt, make_pte(phys, flags))
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

12. Demand Paging and Page Fault Handler

The physical memory allocator is the foundation of all kernel memory management. It must track which physical pages are free and which are in use, supporting allocation requests of various sizes. LateralusOS uses a buddy allocator: physical memory is organized in power-of-two blocks, and splitting and merging of blocks is O(log n) in the memory size.

The allocator is initialized by parsing the memory map provided by the bootloader, marking firmware-reserved and kernel-occupied regions as allocated. After initialization, all kernel heap allocations and virtual memory mappings draw from the buddy allocator's pool.

Virtual memory allows the kernel to present each process with the illusion of a private, contiguous address space independent of the physical layout of RAM. The MMU translates virtual addresses to physical addresses using a hierarchy of page tables. On x86-64, the hierarchy is four levels deep: PML4, PDPT, PD, PT. Each level is a 4KB page containing 512 8-byte entries. A virtual address is split into five fields: the PML4 index, the PDPT index, the PD index, the PT index, and the page offset. Only present pages have valid translations; an access to a non-present page triggers a page fault, which the kernel handles by mapping the page or terminating the process.

The kernel scheduler determines which thread runs on which CPU at each moment. LateralusOS implements a Completely Fair Scheduler (CFS) variant inspired by Linux's CFS. Each thread has a virtual runtime -- the total CPU time it has consumed, weighted by its priority. The scheduler always runs the thread with the lowest virtual runtime, which ensures fairness over time. Priority is implemented as a weight: a high-priority thread accumulates virtual runtime more slowly, so it gets proportionally more CPU time. SMP support requires a per-CPU run queue to avoid lock contention, with periodic load balancing to migrate threads between CPUs.

Implementation Details

Interrupt handling is the mechanism by which hardware events reach the kernel. When a keyboard is pressed, a timer fires, or a disk read completes, the hardware asserts an interrupt request line. The APIC (Advanced Programmable Interrupt Controller) arbitrates between multiple interrupt sources and delivers them to a CPU, which saves its current state and jumps to the appropriate interrupt handler via the IDT (Interrupt Descriptor Table). Interrupt handlers must be fast, non-blocking, and careful about which kernel locks they acquire. LateralusOS's type system enforces these constraints: functions marked as interrupt handlers are restricted to a subset of the kernel API.

The Virtual File System (VFS) layer provides a uniform interface to all file systems. Applications use the same system calls (open, read, write, close, stat) regardless of whether the underlying storage is a local disk, a network share, a FUSE user-space file system, or a synthetic file system like /proc. The VFS layer defines abstract types: Inode (a file or directory), Dentry (a name-to-inode mapping), and File (an open file descriptor). Concrete file system drivers implement these abstractions. LateralusOS supports FAT32 for boot media compatibility and ext2 for general storage, with the VFS layer making both appear identical to user programs.

```
// CFS scheduler -- pick thread with min vruntime
fn schedule(cpu: &mut CpuState) {
    let next = cpu.run_queue
        |> min_by(fn(t) { t.vruntime })  // O(log n) with red-black tree
    match next {
        Some(thread) => {
            cpu.run_queue.remove(thread.id)
            cpu.current = thread
```

```
            context_switch(cpu.current, thread)
        },
        None => { cpu.current = IDLE_THREAD },
    }
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

13. Copy-on-Write Semantics

Virtual memory allows the kernel to present each process with the illusion of a private, contiguous address space independent of the physical layout of RAM. The MMU translates virtual addresses to physical addresses using a hierarchy of page tables. On x86-64, the hierarchy is four levels deep: PML4, PDPT, PD, PT. Each level is a 4KB page containing 512 8-byte entries. A virtual address is split into five fields: the PML4 index, the PDPT index, the PD index, the PT index, and the page offset. Only present pages have valid translations; an access to a non-present page triggers a page fault, which the kernel handles by mapping the page or terminating the process.

The kernel scheduler determines which thread runs on which CPU at each moment. LateralusOS implements a Completely Fair Scheduler (CFS) variant inspired by Linux's CFS. Each thread has a virtual runtime -- the total CPU time it has consumed, weighted by its priority. The scheduler always runs the thread with the lowest virtual runtime, which ensures fairness over time. Priority is implemented as a weight: a high-priority thread accumulates virtual runtime more slowly, so it gets proportionally more CPU time. SMP support requires a per-CPU run queue to avoid lock contention, with periodic load balancing to migrate threads between CPUs.

Interrupt handling is the mechanism by which hardware events reach the kernel. When a keyboard is pressed, a timer fires, or a disk read completes, the hardware asserts an interrupt request line. The APIC (Advanced Programmable Interrupt Controller) arbitrates between multiple interrupt sources and delivers them to a CPU, which saves its current state and jumps to the appropriate interrupt handler via the IDT (Interrupt Descriptor Table). Interrupt handlers must be fast, non-blocking, and careful about which kernel locks they acquire. LateralusOS's type system enforces these constraints: functions marked as interrupt handlers are restricted to a subset of the kernel API.

Implementation Details

The Virtual File System (VFS) layer provides a uniform interface to all file systems. Applications use the same system calls (open, read, write, close, stat) regardless of whether the underlying storage is a local disk, a network share, a FUSE user-space file system, or a synthetic file system like /proc. The VFS layer defines abstract types: Inode (a file or directory), Dentry (a name-to-inode mapping), and File (an open file descriptor). Concrete file system drivers implement these abstractions. LateralusOS supports FAT32 for boot media compatibility and ext2 for general storage, with the VFS layer making both appear identical to user programs.

Device drivers are the most error-prone component of any operating system. They run in privileged kernel mode but interact with hardware that may misbehave, return unexpected data, or fail silently. LateralusOS's driver framework uses Lateralus traits to define safe interfaces: a block device driver implements the BlockDevice trait (read_sector, write_sector, sector_count), a character device implements CharDevice (read_byte, write_byte), and a network interface implements NetDevice (send_frame, recv_frame, get_mac). The trait system ensures that all required methods are implemented and that callers cannot accidentally call driver-specific methods through the generic interface.

```
// VFS: trait for file system drivers
trait FileSystem {
    fn read(&self, inode: &Inode, buf: &mut [u8], offset: u64) -> Result[usize, KernelError]
    fn write(&mut self, inode: &Inode, buf: &[u8], offset: u64) -> Result[usize, KernelError]
    fn lookup(&self, dir: &Inode, name: str) -> Result[Inode, KernelError]
    fn readdir(&self, dir: &Inode) -> Result[list[DirEntry], KernelError]
    fn create(&mut self, dir: &Inode, name: str, kind: InodeKind) -> Result[Inode, KernelError]
    fn unlink(&mut self, dir: &Inode, name: str) -> Result[(), KernelError]
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

14. Process Control Block Design

The kernel scheduler determines which thread runs on which CPU at each moment. LateralusOS implements a Completely Fair Scheduler (CFS) variant inspired by Linux's CFS. Each thread has a virtual runtime -- the total CPU time it has consumed, weighted by its priority. The scheduler always runs the thread with the lowest virtual runtime, which ensures fairness over time. Priority is implemented as a weight: a high-priority thread accumulates virtual runtime more slowly, so it gets proportionally more CPU time. SMP support requires a per-CPU run queue to avoid lock contention, with periodic load balancing to migrate threads between CPUs.

Interrupt handling is the mechanism by which hardware events reach the kernel. When a keyboard is pressed, a timer fires, or a disk read completes, the hardware asserts an interrupt request line. The APIC (Advanced Programmable Interrupt Controller) arbitrates between multiple interrupt sources and delivers them to a CPU, which saves its current state and jumps to the appropriate interrupt handler via the IDT (Interrupt Descriptor Table). Interrupt handlers must be fast, non-blocking, and careful about which kernel locks they acquire. LateralusOS's type system enforces these constraints: functions marked as interrupt handlers are restricted to a subset of the kernel API.

The Virtual File System (VFS) layer provides a uniform interface to all file systems. Applications use the same system calls (open, read, write, close, stat) regardless of whether the underlying storage is a local disk, a network share, a FUSE user-space file system, or a synthetic file system like /proc. The VFS layer defines abstract types: Inode (a file or directory), Dentry (a name-to-inode mapping), and File (an

open file descriptor). Concrete file system drivers implement these abstractions. LateralusOS supports FAT32 for boot media compatibility and ext2 for general storage, with the VFS layer making both appear identical to user programs.

Implementation Details

Device drivers are the most error-prone component of any operating system. They run in privileged kernel mode but interact with hardware that may misbehave, return unexpected data, or fail silently. LateralusOS's driver framework uses Lateralus traits to define safe interfaces: a block device driver implements the BlockDevice trait (read_sector, write_sector, sector_count), a character device implements CharDevice (read_byte, write_byte), and a network interface implements NetDevice (send_frame, recv_frame, get_mac). The trait system ensures that all required methods are implemented and that callers cannot accidentally call driver-specific methods through the generic interface.

The RISC-V instruction set architecture offers a clean, modular design that makes OS implementation particularly straightforward. The ISA is divided into standard extensions: I (integer), M (multiply/divide), A (atomic), F (single-precision float), D (double-precision float), C (compressed 16-bit instructions). The privileged architecture defines three privilege levels: Machine (M-mode, highest), Supervisor (S-mode, for OS kernels), and User (U-mode, for applications). The RISC-V Supervisor Binary Interface (SBI) provides a standard firmware abstraction layer, allowing OS kernels to interact with platform hardware through well-defined firmware calls rather than direct hardware access.

```
// RISC-V SBI call wrapper
fn sbi_call(eid: u32, fid: u32, a0: u64, a1: u64, a2: u64) -> SbiReturn {
    // Place args in registers a0-a6, ecall into M-mode firmware
    let (error, value): (i64, i64)
    asm {
        "li a7, {eid}"
        "li a6, {fid}"
        "mv a0, {a0}"
        "mv a1, {a1}"
        "mv a2, {a2}"
        "ecall"
        "mv {error}, a0"
        "mv {value}, a1"
    }
    return SbiReturn { error, value }
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

15. Thread Abstraction

Interrupt handling is the mechanism by which hardware events reach the kernel. When a keyboard is pressed, a timer fires, or a disk read completes, the hardware asserts an interrupt request line. The APIC (Advanced Programmable Interrupt Controller) arbitrates between multiple interrupt sources and delivers them to a CPU, which saves its current state and jumps to the appropriate interrupt handler via the IDT (Interrupt Descriptor Table). Interrupt handlers must be fast, non-blocking, and careful about which kernel locks they acquire. LateralusOS's type system enforces these constraints: functions marked as interrupt handlers are restricted to a subset of the kernel API.

The Virtual File System (VFS) layer provides a uniform interface to all file systems. Applications use the same system calls (open, read, write, close, stat) regardless of whether the underlying storage is a local disk, a network share, a FUSE user-space file system, or a synthetic file system like /proc. The VFS layer defines abstract types: Inode (a file or directory), Dentry (a name-to-inode mapping), and File (an open file descriptor). Concrete file system drivers implement these abstractions. LateralusOS supports FAT32 for boot media compatibility and ext2 for general storage, with the VFS layer making both appear identical to user programs.

Device drivers are the most error-prone component of any operating system. They run in privileged kernel mode but interact with hardware that may misbehave, return unexpected data, or fail silently. LateralusOS's driver framework uses Lateralus traits to define safe interfaces: a block device driver implements the BlockDevice trait (read_sector, write_sector, sector_count), a character device implements CharDevice (read_byte, write_byte), and a network interface implements NetDevice (send_frame, recv_frame, get_mac). The trait system ensures that all required methods are implemented and that callers cannot accidentally call driver-specific methods through the generic interface.

Implementation Details

The RISC-V instruction set architecture offers a clean, modular design that makes OS implementation particularly straightforward. The ISA is divided into standard extensions: I (integer), M (multiply/divide), A (atomic), F (single-precision float), D (double-precision float), C (compressed 16-bit instructions). The privileged architecture defines three privilege levels: Machine (M-mode, highest), Supervisor (S-mode, for OS kernels), and User (U-mode, for applications). The RISC-V Supervisor Binary Interface (SBI) provides a standard firmware abstraction layer, allowing OS kernels to interact with platform hardware through well-defined firmware calls rather than direct hardware access.

OS performance optimization requires careful attention to hardware characteristics. Modern CPUs execute instructions out-of-order and speculatively, making the cost of a branch misprediction 15-20 cycles. Memory accesses are dominated by cache behavior: an L1 hit takes ~4 cycles, an L2 hit ~12, an L3 hit ~40, and a DRAM access ~200. The kernel's hot paths -- interrupt dispatch, system call entry, scheduler context switch -- are heavily optimized to minimize cache misses and branch mispredictions. LateralusOS's scheduler data structures are designed to fit in cache: the per-CPU run queue and per-thread control block are sized to avoid cache line splits, and frequently accessed fields are grouped together.

```
// Kernel entry point (after assembly stub sets up stack)
@no_mangle
pub fn kernel_main(boot_info: &BootInfo) {
    serial_init()
    println("[BOOT] LateralusOS kernel_main entered")
    memory::init(boot_info.memory_map)
    interrupts::init_idt()
    apic::init()
    scheduler::init()
```

```
    vfs::init()
    drivers::init_all()
    println("[BOOT] All subsystems initialized")
    scheduler::start()  // does not return
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

16. Context Switch Implementation

The Virtual File System (VFS) layer provides a uniform interface to all file systems. Applications use the same system calls (open, read, write, close, stat) regardless of whether the underlying storage is a local disk, a network share, a FUSE user-space file system, or a synthetic file system like /proc. The VFS layer defines abstract types: Inode (a file or directory), Dentry (a name-to-inode mapping), and File (an open file descriptor). Concrete file system drivers implement these abstractions. LateralusOS supports FAT32 for boot media compatibility and ext2 for general storage, with the VFS layer making both appear identical to user programs.

Device drivers are the most error-prone component of any operating system. They run in privileged kernel mode but interact with hardware that may misbehave, return unexpected data, or fail silently. LateralusOS's driver framework uses Lateralus traits to define safe interfaces: a block device driver implements the BlockDevice trait (read_sector, write_sector, sector_count), a character device implements CharDevice (read_byte, write_byte), and a network interface implements NetDevice (send_frame, recv_frame, get_mac). The trait system ensures that all required methods are implemented and that callers cannot accidentally call driver-specific methods through the generic interface.

The RISC-V instruction set architecture offers a clean, modular design that makes OS implementation particularly straightforward. The ISA is divided into standard extensions: I (integer), M (multiply/divide), A (atomic), F (single-precision float), D (double-precision float), C (compressed 16-bit instructions). The privileged architecture defines three privilege levels: Machine (M-mode, highest), Supervisor (S-mode, for OS kernels), and User (U-mode, for applications). The RISC-V Supervisor Binary Interface (SBI) provides a standard firmware abstraction layer, allowing OS kernels to interact with platform hardware through well-defined firmware calls rather than direct hardware access.

Implementation Details

OS performance optimization requires careful attention to hardware characteristics. Modern CPUs execute instructions out-of-order and speculatively, making the cost of a branch misprediction 15-20 cycles. Memory accesses are dominated by cache behavior: an L1 hit takes ~4 cycles, an L2 hit ~12, an L3 hit ~40, and a DRAM access ~200. The kernel's hot paths -- interrupt dispatch, system call entry, scheduler context switch -- are heavily optimized to minimize cache misses and branch mispredictions. LateralusOS's scheduler data structures are designed to fit in cache: the per-CPU run queue and

per-thread control block are sized to avoid cache line splits, and frequently accessed fields are grouped together.

Writing an operating system in a high-level language was considered impractical for decades because high-level languages required garbage collectors, runtime overhead, and standard libraries that themselves depended on the OS. Lateralus's freestanding mode changes this calculus: the language's core -- arithmetic, pattern matching, closures, traits, and ownership -- is available with zero runtime dependencies. The standard library is optional and modular. A kernel written in Lateralus gets memory safety, trait-based polymorphism, and expressive pattern matching without paying any runtime tax.

```
// Page table entry construction
const PAGE_PRESENT:  u64 = 1 << 0
const PAGE_WRITABLE: u64 = 1 << 1
const PAGE_USER:     u64 = 1 << 2
const PAGE_NOEXEC:   u64 = 1 << 63

fn make_pte(phys: u64, flags: u64) -> u64 {
    // Align to 4KB boundary, set flags
    return (phys & 0xFFFF_FFFF_F000) | flags

}

fn map_page(virt: u64, phys: u64, writable: bool) {
    let flags = PAGE_PRESENT | (if writable { PAGE_WRITABLE } else { 0 })
    PT_ROOT.insert(virt, make_pte(phys, flags))
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

17. Completely Fair Scheduler (CFS)

Device drivers are the most error-prone component of any operating system. They run in privileged kernel mode but interact with hardware that may misbehave, return unexpected data, or fail silently. LateralusOS's driver framework uses Lateralus traits to define safe interfaces: a block device driver implements the BlockDevice trait (read_sector, write_sector, sector_count), a character device implements CharDevice (read_byte, write_byte), and a network interface implements NetDevice (send_frame, recv_frame, get_mac). The trait system ensures that all required methods are implemented and that callers cannot accidentally call driver-specific methods through the generic interface.

The RISC-V instruction set architecture offers a clean, modular design that makes OS implementation particularly straightforward. The ISA is divided into standard extensions: I (integer), M (multiply/divide), A (atomic), F (single-precision float), D (double-precision float), C (compressed 16-bit instructions). The privileged architecture defines three privilege levels: Machine (M-mode, highest), Supervisor (S-mode, for OS kernels), and User (U-mode, for applications). The RISC-V Supervisor Binary Interface (SBI)

provides a standard firmware abstraction layer, allowing OS kernels to interact with platform hardware through well-defined firmware calls rather than direct hardware access.

OS performance optimization requires careful attention to hardware characteristics. Modern CPUs execute instructions out-of-order and speculatively, making the cost of a branch misprediction 15-20 cycles. Memory accesses are dominated by cache behavior: an L1 hit takes ~4 cycles, an L2 hit ~12, an L3 hit ~40, and a DRAM access ~200. The kernel's hot paths -- interrupt dispatch, system call entry, scheduler context switch -- are heavily optimized to minimize cache misses and branch mispredictions. LateralusOS's scheduler data structures are designed to fit in cache: the per-CPU run queue and per-thread control block are sized to avoid cache line splits, and frequently accessed fields are grouped together.

Implementation Details

Writing an operating system in a high-level language was considered impractical for decades because high-level languages required garbage collectors, runtime overhead, and standard libraries that themselves depended on the OS. Lateralus's freestanding mode changes this calculus: the language's core -- arithmetic, pattern matching, closures, traits, and ownership -- is available with zero runtime dependencies. The standard library is optional and modular. A kernel written in Lateralus gets memory safety, trait-based polymorphism, and expressive pattern matching without paying any runtime tax.

The x86-64 boot sequence begins long before the kernel runs. Firmware (UEFI or legacy BIOS) initializes hardware, selects a boot device, loads the bootloader, and jumps to it in a specific CPU mode. For UEFI-based systems, the bootloader runs in 64-bit long mode with a flat memory map provided by the firmware. The kernel must still configure its own page tables, set up the interrupt descriptor table, initialize the APIC interrupt controller, and map its own code and data before it is ready to run Lateralus-level code. LateralusOS uses a minimal assembly stub for this pre-kernel setup, transitioning to Lateralus as early as possible.

```
// CFS scheduler -- pick thread with min vruntime
fn schedule(cpu: &mut CpuState) {

    let next = cpu.run_queue
        |> min_by(fn(t) { t.vruntime })  // O(log n) with red-black tree
    match next {
        Some(thread) => {
            cpu.run_queue.remove(thread.id)
            cpu.current = thread
            context_switch(cpu.current, thread)
        },
        None => { cpu.current = IDLE_THREAD },
    }
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

18. Real-Time Scheduling

The RISC-V instruction set architecture offers a clean, modular design that makes OS implementation particularly straightforward. The ISA is divided into standard extensions: I (integer), M (multiply/divide), A (atomic), F (single-precision float), D (double-precision float), C (compressed 16-bit instructions). The privileged architecture defines three privilege levels: Machine (M-mode, highest), Supervisor (S-mode, for OS kernels), and User (U-mode, for applications). The RISC-V Supervisor Binary Interface (SBI) provides a standard firmware abstraction layer, allowing OS kernels to interact with platform hardware through well-defined firmware calls rather than direct hardware access.

OS performance optimization requires careful attention to hardware characteristics. Modern CPUs execute instructions out-of-order and speculatively, making the cost of a branch misprediction 15-20 cycles. Memory accesses are dominated by cache behavior: an L1 hit takes ~4 cycles, an L2 hit ~12, an L3 hit ~40, and a DRAM access ~200. The kernel's hot paths -- interrupt dispatch, system call entry, scheduler context switch -- are heavily optimized to minimize cache misses and branch mispredictions. LateralusOS's scheduler data structures are designed to fit in cache: the per-CPU run queue and per-thread control block are sized to avoid cache line splits, and frequently accessed fields are grouped together.

Writing an operating system in a high-level language was considered impractical for decades because high-level languages required garbage collectors, runtime overhead, and standard libraries that themselves depended on the OS. Lateralus's freestanding mode changes this calculus: the language's core -- arithmetic, pattern matching, closures, traits, and ownership -- is available with zero runtime dependencies. The standard library is optional and modular. A kernel written in Lateralus gets memory safety, trait-based polymorphism, and expressive pattern matching without paying any runtime tax.

Implementation Details

The x86-64 boot sequence begins long before the kernel runs. Firmware (UEFI or legacy BIOS) initializes hardware, selects a boot device, loads the bootloader, and jumps to it in a specific CPU mode. For UEFI-based systems, the bootloader runs in 64-bit long mode with a flat memory map provided by the firmware. The kernel must still configure its own page tables, set up the interrupt descriptor table, initialize the APIC interrupt controller, and map its own code and data before it is ready to run Lateralus-level code. LateralusOS uses a minimal assembly stub for this pre-kernel setup, transitioning to Lateralus as early as possible.

The physical memory allocator is the foundation of all kernel memory management. It must track which physical pages are free and which are in use, supporting allocation requests of various sizes. LateralusOS uses a buddy allocator: physical memory is organized in power-of-two blocks, and splitting and merging of blocks is O(log n) in the memory size. The allocator is initialized by parsing the memory map provided by the bootloader, marking firmware-reserved and

kernel-occupied regions as allocated. After initialization, all kernel heap allocations and virtual memory mappings draw from the buddy allocator's pool.

```
// VFS: trait for file system drivers
trait FileSystem {
    fn read(&self, inode: &Inode, buf: &mut [u8], offset: u64) -> Result[usize, KernelError]
    fn write(&mut self, inode: &Inode, buf: &[u8], offset: u64) -> Result[usize, KernelError]
    fn lookup(&self, dir: &Inode, name: str) -> Result[Inode, KernelError]
    fn readdir(&self, dir: &Inode) -> Result[list[DirEntry], KernelError]
    fn create(&mut self, dir: &Inode, name: str, kind: InodeKind) -> Result[Inode, KernelError]
    fn unlink(&mut self, dir: &Inode, name: str) -> Result[(), KernelError]
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

19. SMP: Per-CPU Run Queues

OS performance optimization requires careful attention to hardware characteristics. Modern CPUs execute instructions out-of-order and speculatively, making the cost of a branch misprediction 15-20 cycles. Memory accesses are dominated by cache behavior: an L1 hit takes ~4 cycles, an L2 hit ~12, an L3 hit ~40, and a DRAM access ~200. The kernel's hot paths -- interrupt dispatch, system call entry, scheduler context switch -- are heavily optimized to minimize cache misses and branch mispredictions. LateralusOS's scheduler data structures are designed to fit in cache: the per-CPU run queue and per-thread control block are sized to avoid cache line splits, and frequently accessed fields are grouped together.

Writing an operating system in a high-level language was considered impractical for decades because high-level languages required garbage collectors, runtime overhead, and standard libraries that themselves depended on the OS. Lateralus's freestanding mode changes this calculus: the language's core -- arithmetic, pattern matching, closures, traits, and ownership -- is available with zero runtime dependencies. The standard library is optional and modular. A kernel written in Lateralus gets memory safety, trait-based polymorphism, and expressive pattern matching without paying any runtime tax.

The x86-64 boot sequence begins long before the kernel runs. Firmware (UEFI or legacy BIOS) initializes hardware, selects a boot device, loads the bootloader, and jumps to it in a specific CPU mode. For UEFI-based systems, the bootloader runs in 64-bit long mode with a flat memory map provided by the firmware. The kernel must still configure its own page tables, set up the interrupt descriptor table, initialize the APIC interrupt controller, and map its own code and data before it is ready to run Lateralus-level code. LateralusOS uses a minimal assembly stub for this pre-kernel setup, transitioning to Lateralus as early as possible.

Implementation Details

The physical memory allocator is the foundation of all kernel memory management. It must track which physical pages are free and which are in use, supporting allocation requests of various sizes. LateralusOS uses a buddy allocator: physical memory is organized in power-of-two blocks, and splitting and merging of blocks is O(log n) in the memory size. The allocator is initialized by parsing the memory map provided by the bootloader, marking firmware-reserved and kernel-occupied regions as allocated. After initialization, all kernel heap allocations and virtual memory mappings draw from the buddy allocator's pool.

Virtual memory allows the kernel to present each process with the illusion of a private, contiguous address space independent of the physical layout of RAM. The MMU translates virtual addresses to physical addresses using a hierarchy of page tables. On x86-64, the hierarchy is four levels deep: PML4, PDPT, PD, PT. Each level is a 4KB page

containing 512 8-byte entries. A virtual address is split into five fields: the PML4 index, the PDPT index, the PD index, the PT index, and the page offset. Only present pages have valid translations; an access to a non-present page triggers a page fault, which the kernel handles by mapping the page or terminating the process.

```
// RISC-V SBI call wrapper
fn sbi_call(eid: u32, fid: u32, a0: u64, a1: u64, a2: u64) -> SbiReturn {
    // Place args in registers a0-a6, ecall into M-mode firmware
    let (error, value): (i64, i64)
    asm {
        "li a7, {eid}"
        "li a6, {fid}"
        "mv a0, {a0}"
        "mv a1, {a1}"
        "mv a2, {a2}"
        "ecall"
        "mv {error}, a0"
        "mv {value}, a1"
    }
    return SbiReturn { error, value }
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

20. SMP: Load Balancing

Writing an operating system in a high-level language was considered impractical for decades because high-level languages required garbage collectors, runtime overhead, and standard libraries that themselves depended on the OS. Lateralus's freestanding mode changes this calculus: the language's core -- arithmetic, pattern matching, closures, traits, and ownership -- is available with zero runtime dependencies. The standard library is optional and modular. A kernel written in Lateralus gets memory safety, trait-based polymorphism, and expressive pattern matching without paying any runtime tax.

The x86-64 boot sequence begins long before the kernel runs. Firmware (UEFI or legacy BIOS) initializes hardware, selects a boot device, loads the bootloader, and jumps to it in a specific CPU mode. For UEFI-based systems, the bootloader runs in 64-bit long mode with a flat memory map provided by the firmware. The kernel must still configure its own page tables, set up the interrupt descriptor table, initialize the APIC interrupt controller, and map its own code and data before it is ready to run Lateralus-level code. LateralusOS uses a minimal assembly stub for this pre-kernel setup, transitioning to Lateralus as early as possible.

The physical memory allocator is the foundation of all kernel memory management. It must track which physical pages are free and which are in use, supporting allocation requests of various sizes. LateralusOS uses a buddy allocator: physical memory is organized in power-of-two blocks, and splitting

and merging of blocks is O(log n) in the memory size. The allocator is initialized by parsing the memory map provided by the bootloader, marking firmware-reserved and kernel-occupied regions as allocated. After initialization, all kernel heap allocations and virtual memory mappings draw from the buddy allocator's pool.

Implementation Details

Virtual memory allows the kernel to present each process with the illusion of a private, contiguous address space independent of the physical layout of RAM. The MMU translates virtual addresses to physical addresses using a hierarchy of page tables. On x86-64, the hierarchy is four levels deep: PML4, PDPT, PD, PT. Each level is a 4KB page containing 512 8-byte entries. A virtual address is split into five fields: the PML4 index, the PDPT index, the PD index,

the PT index, and the page offset. Only present pages have valid translations; an access to a non-present page triggers a page fault, which the kernel handles by mapping the page or terminating the process.

The kernel scheduler determines which thread runs on which CPU at each moment. LateralusOS implements a Completely Fair Scheduler (CFS) variant inspired by Linux's CFS. Each thread has a virtual runtime -- the total CPU time it has consumed, weighted by its priority. The scheduler always runs the thread with the lowest virtual runtime, which ensures fairness over time. Priority is implemented as a weight: a high-priority thread accumulates virtual runtime more slowly, so it gets proportionally more CPU time. SMP support requires a per-CPU run queue to avoid lock contention, with periodic load balancing to migrate threads between CPUs.

```
// Kernel entry point (after assembly stub sets up stack)
@no_mangle
pub fn kernel_main(boot_info: &BootInfo) {
    serial_init()
    println("[BOOT] LateralusOS kernel_main entered")
    memory::init(boot_info.memory_map)
    interrupts::init_idt()
    apic::init()
    scheduler::init()
    vfs::init()
    drivers::init_all()
    println("[BOOT] All subsystems initialized")
    scheduler::start()  // does not return
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

21. SMP: IPI Delivery

The x86-64 boot sequence begins long before the kernel runs. Firmware (UEFI or legacy BIOS) initializes hardware, selects a boot device, loads the bootloader, and jumps to it in a specific CPU mode. For UEFI-based systems, the bootloader runs in 64-bit long mode with a flat memory map provided by the firmware. The kernel must still configure its own page tables, set up the interrupt descriptor table, initialize the APIC interrupt controller, and map its own code and data before it is ready to run Lateralus-level code. LateralusOS uses a minimal assembly stub for this pre-kernel setup, transitioning to Lateralus as early as possible.

The physical memory allocator is the foundation of all kernel memory management. It must track which physical pages are free and which are in use, supporting allocation requests of various sizes. LateralusOS uses a buddy allocator: physical memory is organized in power-of-two blocks, and splitting and merging of blocks is O(log n) in the memory size. The allocator is initialized by parsing the memory map provided by the bootloader, marking firmware-reserved and kernel-occupied regions as allocated. After initialization, all kernel heap allocations and virtual memory mappings draw from the buddy allocator's pool.

Virtual memory allows the kernel to present each process with the illusion of a private, contiguous address space independent of the physical layout of RAM. The MMU translates virtual addresses to physical addresses using a hierarchy of page tables. On x86-64, the hierarchy is four levels deep: PML4, PDPT, PD, PT. Each level is a 4KB page containing 512 8-byte entries. A virtual address is split into five fields: the PML4 index, the PDPT index, the PD index, the PT index, and the page offset. Only present pages have valid translations; an access to a non-present page triggers a page fault, which the kernel handles by mapping the page or terminating the process.

Implementation Details

The kernel scheduler determines which thread runs on which CPU at each moment. LateralusOS implements a Completely Fair Scheduler (CFS) variant inspired by Linux's CFS. Each thread has a virtual runtime -- the total CPU time it has consumed, weighted by its priority. The scheduler always runs the thread with the lowest virtual runtime, which ensures fairness over time. Priority is implemented as a weight: a high-priority thread accumulates virtual runtime more slowly, so it gets proportionally more CPU time. SMP support requires a per-CPU run queue to avoid lock contention, with periodic load balancing to migrate threads between CPUs.

Interrupt handling is the mechanism by which hardware events reach the kernel. When a keyboard is pressed, a timer fires, or a disk read completes, the hardware asserts an interrupt request line. The APIC (Advanced Programmable Interrupt Controller) arbitrates between multiple interrupt sources and delivers them to a CPU, which saves its current state and jumps to the appropriate interrupt handler via the IDT (Interrupt Descriptor Table). Interrupt handlers must be fast, non-blocking, and careful about which kernel locks they acquire. LateralusOS's type system enforces these constraints: functions marked as interrupt handlers are restricted to a subset of the kernel API.

```
// Page table entry construction
const PAGE_PRESENT:  u64 = 1 << 0
const PAGE_WRITABLE: u64 = 1 << 1
const PAGE_USER:     u64 = 1 << 2
const PAGE_NOEXEC:   u64 = 1 << 63

fn make_pte(phys: u64, flags: u64) -> u64 {
    // Align to 4KB boundary, set flags
    return (phys & 0xFFFF_FFFF_F000) | flags
}

fn map_page(virt: u64, phys: u64, writable: bool) {
```

```
    let flags = PAGE_PRESENT | (if writable { PAGE_WRITABLE } else { 0 })
    PT_ROOT.insert(virt, make_pte(phys, flags))
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

22. Spinlocks and Mutexes

The physical memory allocator is the foundation of all kernel memory management. It must track which physical pages are free and which are in use, supporting allocation requests of various sizes. LateralusOS uses a buddy allocator: physical memory is organized in power-of-two blocks, and splitting and merging of blocks is O(log n) in the memory size. The allocator is initialized by parsing the memory map provided by the bootloader, marking firmware-reserved and kernel-occupied regions as allocated. After initialization, all kernel heap allocations and virtual memory mappings draw from the buddy allocator's pool.

Virtual memory allows the kernel to present each process with the illusion of a private, contiguous address space independent of the physical layout of RAM. The MMU translates virtual addresses to physical addresses using a hierarchy of page tables. On x86-64, the hierarchy is four levels deep: PML4, PDPT, PD, PT. Each level is a 4KB page containing 512 8-byte entries. A virtual address is split into five fields: the PML4 index, the PDPT index, the PD index, the PT index, and the page offset. Only present pages have valid translations; an access to a non-present page triggers a page fault, which the kernel handles by mapping the page or terminating the process.

The kernel scheduler determines which thread runs on which CPU at each moment. LateralusOS implements a Completely Fair Scheduler (CFS) variant inspired by Linux's CFS. Each thread has a virtual runtime -- the total CPU time it has consumed, weighted by its priority. The scheduler always runs the thread with the lowest virtual runtime,

which ensures fairness over time. Priority is implemented as a weight: a high-priority thread accumulates virtual runtime more slowly, so it gets proportionally more CPU time. SMP support requires a per-CPU run queue to avoid lock contention, with periodic load balancing to migrate threads between CPUs.

Implementation Details

Interrupt handling is the mechanism by which hardware events reach the kernel. When a keyboard is pressed, a timer fires, or a disk read completes, the hardware asserts an interrupt request line. The APIC (Advanced Programmable Interrupt Controller) arbitrates between multiple interrupt sources and delivers them to a CPU, which saves its current state and jumps to the appropriate interrupt handler via the IDT (Interrupt Descriptor Table). Interrupt handlers must be fast, non-blocking, and careful about which kernel locks they acquire. LateralusOS's type system enforces these constraints: functions marked as interrupt handlers are restricted to a subset of the kernel API.

The Virtual File System (VFS) layer provides a uniform interface to all file systems. Applications use the same system calls (open, read, write, close, stat) regardless of whether the underlying storage is a local disk, a network share, a FUSE user-space file system, or a synthetic file system like /proc. The VFS layer defines abstract types: Inode (a file or directory), Dentry (a name-to-inode mapping), and File (an open file descriptor). Concrete file system drivers implement these abstractions. LateralusOS supports FAT32 for boot media compatibility and ext2 for general storage, with the VFS layer making both appear identical to user programs.

```
// CFS scheduler -- pick thread with min vruntime
fn schedule(cpu: &mut CpuState) {
    let next = cpu.run_queue
        |> min_by(fn(t) { t.vruntime })  // O(log n) with red-black tree
    match next {
        Some(thread) => {
            cpu.run_queue.remove(thread.id)
            cpu.current = thread
            context_switch(cpu.current, thread)
        },
        None => { cpu.current = IDLE_THREAD },
    }
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

23. Wait Queues and Condition Variables

Virtual memory allows the kernel to present each process with the illusion of a private, contiguous address space independent of the physical layout of RAM. The MMU translates virtual addresses to physical addresses using a hierarchy of page tables. On x86-64, the hierarchy is four levels deep: PML4, PDPT, PD, PT. Each level is a 4KB page containing 512 8-byte entries. A virtual address is split into five fields: the PML4 index, the PDPT index, the PD index, the PT index, and the page offset. Only present pages have valid translations; an access to a non-present page triggers a page fault, which the kernel handles by mapping the page or terminating the process.

The kernel scheduler determines which thread runs on which CPU at each moment. LateralusOS implements a Completely Fair Scheduler (CFS) variant inspired by Linux's CFS. Each thread has a virtual runtime -- the total CPU time it has consumed, weighted by its priority. The scheduler always runs the thread with the lowest virtual runtime, which ensures fairness over time. Priority is implemented as a weight: a high-priority thread accumulates virtual runtime more slowly, so it gets proportionally more CPU time. SMP support requires a per-CPU run queue to avoid lock contention, with periodic load balancing to migrate threads between CPUs.

Interrupt handling is the mechanism by which hardware events reach the kernel. When a keyboard is pressed, a timer fires, or a disk read completes, the hardware asserts an interrupt request line. The APIC (Advanced Programmable Interrupt Controller) arbitrates between multiple interrupt sources and delivers them to a CPU, which saves its current state and jumps to the appropriate interrupt handler via the IDT (Interrupt Descriptor Table). Interrupt handlers must be fast, non-blocking, and careful about which kernel locks they acquire. LateralusOS's type system enforces these constraints: functions marked as interrupt handlers are restricted to a subset of the kernel API.

Implementation Details

The Virtual File System (VFS) layer provides a uniform interface to all file systems. Applications use the same system calls (open, read, write, close, stat) regardless of whether the underlying storage is a local disk, a network share, a FUSE user-space file system, or a synthetic file system like /proc. The VFS layer defines abstract types: Inode (a file or directory), Dentry (a name-to-inode mapping), and File (an open file descriptor). Concrete file system drivers implement these abstractions. LateralusOS supports FAT32 for boot media compatibility and ext2 for general storage, with the VFS layer making both appear identical to user programs.

Device drivers are the most error-prone component of any operating system. They run in privileged kernel mode but interact with hardware that may misbehave, return unexpected data, or fail silently. LateralusOS's driver framework uses Lateralus traits to define safe interfaces: a block device driver implements the BlockDevice trait (read_sector, write_sector, sector_count), a character device implements CharDevice (read_byte, write_byte), and a network interface implements NetDevice (send_frame, recv_frame, get_mac). The trait system ensures that all required methods are implemented and that callers cannot accidentally call driver-specific methods through the generic interface.

```
// VFS: trait for file system drivers
trait FileSystem {
    fn read(&self, inode: &Inode, buf: &mut [u8], offset: u64) -> Result[usize, KernelError]
    fn write(&mut self, inode: &Inode, buf: &[u8], offset: u64) -> Result[usize, KernelError]
    fn lookup(&self, dir: &Inode, name: str) -> Result[Inode, KernelError]
    fn readdir(&self, dir: &Inode) -> Result[list[DirEntry], KernelError]
    fn create(&mut self, dir: &Inode, name: str, kind: InodeKind) -> Result[Inode, KernelError]
    fn unlink(&mut self, dir: &Inode, name: str) -> Result[(), KernelError]
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

24. Read-Write Locks

The kernel scheduler determines which thread runs on which CPU at each moment. LateralusOS implements a Completely Fair Scheduler (CFS) variant inspired by Linux's CFS. Each thread has a virtual runtime -- the total CPU time it has consumed, weighted by its priority. The scheduler always runs the thread with the lowest virtual runtime, which ensures fairness over time. Priority is implemented as a weight: a high-priority thread accumulates virtual runtime more slowly, so it gets proportionally more CPU time. SMP support requires a per-CPU run queue to avoid lock contention, with periodic load balancing to migrate threads between CPUs.

Interrupt handling is the mechanism by which hardware events reach the kernel. When a keyboard is pressed, a timer fires, or a disk read completes, the hardware asserts an interrupt request line. The APIC (Advanced Programmable Interrupt Controller) arbitrates between multiple interrupt sources and delivers them to a CPU, which saves its current state and jumps to the appropriate interrupt handler via the IDT (Interrupt Descriptor Table). Interrupt handlers must be fast, non-blocking, and careful about which kernel locks they acquire. LateralusOS's type system enforces these constraints: functions marked as interrupt handlers are restricted to a subset of the kernel API.

The Virtual File System (VFS) layer provides a uniform interface to all file systems. Applications use the same system calls (open, read, write, close, stat) regardless of whether the underlying storage is a local disk, a network share, a FUSE user-space file system, or a synthetic file system like /proc. The VFS layer defines abstract types: Inode (a file or directory), Dentry (a name-to-inode mapping), and File (an open file descriptor). Concrete file system drivers implement these abstractions. LateralusOS supports FAT32 for boot media compatibility and ext2 for general storage, with the VFS layer making both appear identical to user programs.

Implementation Details

Device drivers are the most error-prone component of any operating system. They run in privileged kernel mode but interact with hardware that may misbehave, return unexpected data, or fail silently. LateralusOS's driver framework uses Lateralus traits to define safe interfaces: a block device driver implements the BlockDevice trait (read_sector, write_sector, sector_count), a character device implements CharDevice (read_byte, write_byte), and a network interface implements NetDevice (send_frame, recv_frame, get_mac). The trait system ensures that all required methods are implemented and that callers cannot accidentally call driver-specific methods through the generic interface.

The RISC-V instruction set architecture offers a clean, modular design that makes OS implementation particularly straightforward. The ISA is divided into standard extensions: I (integer), M (multiply/divide), A (atomic), F (single-precision float), D (double-precision float), C (compressed 16-bit instructions). The privileged architecture defines three privilege levels: Machine (M-mode, highest), Supervisor (S-mode, for OS kernels), and User (U-mode, for applications). The RISC-V Supervisor Binary Interface (SBI) provides a standard firmware abstraction layer, allowing OS kernels to interact with platform hardware through well-defined firmware calls rather than direct hardware access.

```
// RISC-V SBI call wrapper
fn sbi_call(eid: u32, fid: u32, a0: u64, a1: u64, a2: u64) -> SbiReturn {
    // Place args in registers a0-a6, ecall into M-mode firmware
    let (error, value): (i64, i64)
    asm {
        "li a7, {eid}"
        "li a6, {fid}"
        "mv a0, {a0}"
        "mv a1, {a1}"
        "mv a2, {a2}"
```

```
        "ecall"
        "mv {error}, a0"
        "mv {value}, a1"
    }
    return SbiReturn { error, value }
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

25. Interrupt Descriptor Table

Interrupt handling is the mechanism by which hardware events reach the kernel. When a keyboard is pressed, a timer fires, or a disk read completes, the hardware asserts an interrupt request line. The APIC (Advanced Programmable Interrupt Controller) arbitrates between multiple interrupt sources and delivers them to a CPU, which saves its current state and jumps to the appropriate interrupt handler via the IDT (Interrupt Descriptor Table). Interrupt handlers must be fast, non-blocking, and careful about which kernel locks they acquire. LateralusOS's type system enforces these constraints: functions marked as interrupt handlers are restricted to a subset of the kernel API.

The Virtual File System (VFS) layer provides a uniform interface to all file systems. Applications use the same system

calls (open, read, write, close, stat) regardless of whether the underlying storage is a local disk, a network share, a FUSE user-space file system, or a synthetic file system like /proc. The VFS layer defines abstract types: Inode (a file or directory), Dentry (a name-to-inode mapping), and File (an open file descriptor). Concrete file system drivers implement these abstractions. LateralusOS supports FAT32 for boot media compatibility and ext2 for general storage, with the VFS layer making both appear identical to user programs.

Device drivers are the most error-prone component of any operating system. They run in privileged kernel mode but interact with hardware that may misbehave, return unexpected data, or fail silently. LateralusOS's driver framework uses Lateralus traits to define safe interfaces: a block device driver implements the BlockDevice trait (read_sector, write_sector, sector_count), a character device implements CharDevice (read_byte, write_byte), and a network interface implements NetDevice (send_frame, recv_frame, get_mac). The trait system ensures that all required methods are implemented and that callers cannot accidentally call driver-specific methods through the generic interface.

Implementation Details

The RISC-V instruction set architecture offers a clean, modular design that makes OS implementation particularly straightforward. The ISA is divided into standard extensions: I (integer), M (multiply/divide), A (atomic), F (single-precision float), D (double-precision float), C (compressed 16-bit instructions). The

privileged architecture defines three privilege levels: Machine (M-mode, highest), Supervisor (S-mode, for OS kernels), and User (U-mode, for applications). The RISC-V Supervisor Binary Interface (SBI) provides a standard firmware abstraction layer, allowing OS kernels to interact with platform hardware through well-defined firmware calls rather than direct hardware access.

OS performance optimization requires careful attention to hardware characteristics. Modern CPUs execute instructions out-of-order and speculatively, making the cost of a branch misprediction 15-20 cycles. Memory accesses are dominated by cache behavior: an L1 hit takes ~4 cycles, an L2 hit ~12, an L3 hit ~40, and a DRAM access ~200. The kernel's hot paths -- interrupt dispatch, system call entry, scheduler context switch -- are heavily optimized to minimize cache misses and branch mispredictions. LateralusOS's scheduler data structures are designed to fit in cache: the per-CPU run queue and per-thread control block are sized to avoid cache line splits, and frequently accessed fields are grouped together.

```
// Kernel entry point (after assembly stub sets up stack)
@no_mangle
pub fn kernel_main(boot_info: &BootInfo) {
    serial_init()
    println("[BOOT] LateralusOS kernel_main entered")
    memory::init(boot_info.memory_map)
    interrupts::init_idt()
    apic::init()
    scheduler::init()
    vfs::init()
    drivers::init_all()
    println("[BOOT] All subsystems initialized")
    scheduler::start()  // does not return
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

26. APIC Configuration

The Virtual File System (VFS) layer provides a uniform interface to all file systems. Applications use the same system calls (open, read, write, close, stat) regardless of whether the underlying storage is a local disk, a network share, a

FUSE user-space file system, or a synthetic file system like /proc. The VFS layer defines abstract types: Inode (a file or directory), Dentry (a name-to-inode mapping), and File (an open file descriptor). Concrete file system drivers implement these abstractions. LateralusOS supports FAT32 for boot media compatibility and ext2 for general storage, with the VFS layer making both appear identical to user programs.

Device drivers are the most error-prone component of any operating system. They run in privileged kernel mode but interact with hardware that may misbehave, return unexpected data, or fail silently. LateralusOS's driver framework uses Lateralus traits to define safe interfaces: a block device driver implements the BlockDevice trait (read_sector, write_sector, sector_count), a character device implements CharDevice (read_byte, write_byte), and a network interface implements NetDevice (send_frame, recv_frame, get_mac). The trait system ensures that all required methods are implemented and that callers cannot accidentally call driver-specific methods through the generic interface.

The RISC-V instruction set architecture offers a clean, modular design that makes OS implementation particularly straightforward. The ISA is divided into standard extensions: I (integer), M (multiply/divide), A (atomic), F (single-precision float), D (double-precision float), C (compressed 16-bit instructions). The privileged architecture defines three privilege levels: Machine (M-mode, highest), Supervisor (S-mode, for OS kernels), and User (U-mode, for applications). The RISC-V Supervisor Binary Interface (SBI) provides a standard firmware abstraction layer, allowing OS kernels to interact with platform hardware through well-defined firmware calls rather than direct hardware access.

Implementation Details

OS performance optimization requires careful attention to hardware characteristics. Modern CPUs execute instructions out-of-order and speculatively, making the cost of a branch misprediction 15-20 cycles. Memory accesses are dominated by cache behavior: an L1 hit takes ~4 cycles, an L2 hit ~12, an L3 hit ~40, and a DRAM access ~200. The kernel's hot paths -- interrupt dispatch, system call entry, scheduler context switch -- are heavily optimized to minimize cache misses and branch mispredictions. LateralusOS's scheduler data structures are designed to fit in cache: the per-CPU run queue and per-thread control block are sized to avoid cache line splits, and frequently accessed fields are grouped together.

Writing an operating system in a high-level language was considered impractical for decades because high-level languages required garbage collectors, runtime overhead, and standard libraries that themselves depended on the OS. Lateralus's freestanding mode changes this calculus: the language's core -- arithmetic, pattern matching, closures, traits, and ownership -- is available with zero runtime dependencies. The standard library is optional and modular. A kernel written in Lateralus gets memory safety, trait-based polymorphism, and expressive pattern matching without paying any runtime tax.

```
// Page table entry construction
const PAGE_PRESENT:  u64 = 1 << 0
const PAGE_WRITABLE: u64 = 1 << 1
const PAGE_USER:     u64 = 1 << 2
const PAGE_NOEXEC:   u64 = 1 << 63

fn make_pte(phys: u64, flags: u64) -> u64 {
    // Align to 4KB boundary, set flags
    return (phys & 0xFFFF_FFFF_F000) | flags
}

fn map_page(virt: u64, phys: u64, writable: bool) {
    let flags = PAGE_PRESENT | (if writable { PAGE_WRITABLE } else { 0 })
    PT_ROOT.insert(virt, make_pte(phys, flags))
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

27. Timer Interrupt and Timekeeping

Device drivers are the most error-prone component of any operating system. They run in privileged kernel mode but interact with hardware that may misbehave, return unexpected data, or fail silently. LateralusOS's driver framework uses Lateralus traits to define safe interfaces: a block device driver implements the BlockDevice trait (read_sector, write_sector, sector_count), a character device implements CharDevice (read_byte, write_byte), and a network interface implements NetDevice (send_frame, recv_frame, get_mac). The trait system ensures that all required methods are implemented and that callers cannot accidentally call driver-specific methods through the generic interface.

The RISC-V instruction set architecture offers a clean, modular design that makes OS implementation particularly straightforward. The ISA is divided into standard extensions: I (integer), M (multiply/divide), A (atomic), F (single-precision float), D (double-precision float), C (compressed 16-bit instructions). The privileged architecture defines three privilege levels: Machine (M-mode, highest), Supervisor (S-mode, for OS kernels), and User (U-mode, for applications). The RISC-V Supervisor Binary Interface (SBI) provides a standard firmware abstraction layer, allowing OS kernels to interact with platform hardware through well-defined firmware calls rather than direct hardware access.

OS performance optimization requires careful attention to hardware characteristics. Modern CPUs execute instructions out-of-order and speculatively, making the cost of a branch misprediction 15-20 cycles. Memory accesses are dominated by cache behavior: an L1 hit takes ~4 cycles, an L2 hit ~12, an L3 hit ~40, and a DRAM access ~200. The kernel's hot paths -- interrupt dispatch, system call entry, scheduler context switch -- are heavily optimized to minimize cache misses and branch mispredictions. LateralusOS's scheduler data structures are designed to fit in cache: the per-CPU run queue and per-thread control block are sized to avoid cache line splits, and frequently accessed fields are grouped together.

Implementation Details

Writing an operating system in a high-level language was considered impractical for decades because high-level languages required garbage collectors, runtime overhead, and standard libraries that themselves depended on the OS. Lateralus's freestanding mode changes this calculus: the language's core -- arithmetic, pattern matching, closures, traits, and ownership -- is available with zero runtime dependencies. The standard library is optional and modular. A kernel written in Lateralus gets memory safety, trait-based polymorphism, and expressive pattern matching without paying any runtime tax.

The x86-64 boot sequence begins long before the kernel runs. Firmware (UEFI or legacy BIOS) initializes hardware, selects a boot device, loads the bootloader, and jumps to it in a specific CPU mode. For UEFI-based systems, the bootloader runs in 64-bit long mode with a flat memory map provided by the firmware. The kernel must still configure its own page tables, set up the interrupt descriptor table, initialize the APIC interrupt controller, and map its own code and data before it is ready to run Lateralus-level code. LateralusOS uses a minimal assembly stub for this pre-kernel setup, transitioning to Lateralus as early as possible.

```
// CFS scheduler -- pick thread with min vruntime
fn schedule(cpu: &mut CpuState) {
    let next = cpu.run_queue
        |> min_by(fn(t) { t.vruntime })  // O(log n) with red-black tree
    match next {
        Some(thread) => {
            cpu.run_queue.remove(thread.id)
            cpu.current = thread
            context_switch(cpu.current, thread)
        },
        None => { cpu.current = IDLE_THREAD },
    }
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

28. System Call Interface (int 0x80 / syscall)

The RISC-V instruction set architecture offers a clean, modular design that makes OS implementation particularly straightforward. The ISA is divided into standard extensions: I (integer), M (multiply/divide), A (atomic), F (single-precision float), D (double-precision float), C (compressed 16-bit instructions). The privileged architecture defines three privilege levels: Machine (M-mode, highest), Supervisor (S-mode, for OS kernels), and User (U-mode, for applications). The RISC-V Supervisor Binary Interface (SBI) provides a standard firmware abstraction layer, allowing OS kernels to interact with platform hardware through well-defined firmware calls rather than direct hardware access.

OS performance optimization requires careful attention to hardware characteristics. Modern CPUs execute instructions out-of-order and speculatively, making the cost of a branch misprediction 15-20 cycles. Memory accesses are dominated by cache behavior: an L1 hit takes ~4 cycles, an L2 hit ~12, an L3 hit ~40, and a DRAM access ~200. The kernel's hot paths -- interrupt dispatch, system call entry, scheduler context switch -- are heavily optimized to minimize cache misses and branch mispredictions. LateralusOS's scheduler data structures are designed to fit in cache: the per-CPU run queue and per-thread control block are sized to avoid cache line splits, and frequently accessed fields are grouped together.

Writing an operating system in a high-level language was considered impractical for decades because high-level languages required garbage collectors, runtime overhead, and standard libraries that themselves depended on the OS. Lateralus's freestanding mode changes this calculus: the language's core -- arithmetic, pattern matching, closures, traits, and ownership -- is available with zero runtime dependencies. The standard library is optional and modular. A kernel written in Lateralus gets memory safety, trait-based polymorphism, and expressive pattern matching without paying any runtime tax.

Implementation Details

The x86-64 boot sequence begins long before the kernel runs. Firmware (UEFI or legacy BIOS) initializes hardware, selects a boot device, loads the bootloader, and jumps to it in a specific CPU mode. For UEFI-based systems, the bootloader runs in 64-bit long mode with a flat memory map provided by the firmware. The kernel must still configure its own page tables, set up the interrupt descriptor table, initialize the APIC interrupt controller, and map its own code and data before it is ready to run Lateralus-level code. LateralusOS uses a minimal assembly stub for this pre-kernel setup, transitioning to Lateralus as early as possible.

The physical memory allocator is the foundation of all kernel memory management. It must track which physical pages are free and which are in use, supporting allocation requests of various sizes. LateralusOS uses a buddy allocator: physical memory is organized in power-of-two blocks, and splitting and merging of blocks is O(log n) in the memory size. The allocator is initialized by parsing the memory map provided by the bootloader, marking firmware-reserved and kernel-occupied regions as allocated. After initialization, all kernel heap allocations and virtual memory mappings draw from the buddy allocator's pool.

```
// VFS: trait for file system drivers
trait FileSystem {
    fn read(&self, inode: &Inode, buf: &mut [u8], offset: u64) -> Result[usize, KernelError]
    fn write(&mut self, inode: &Inode, buf: &[u8], offset: u64) -> Result[usize, KernelError]
    fn lookup(&self, dir: &Inode, name: str) -> Result[Inode, KernelError]
    fn readdir(&self, dir: &Inode) -> Result[list[DirEntry], KernelError]
    fn create(&mut self, dir: &Inode, name: str, kind: InodeKind) -> Result[Inode, KernelError]
    fn unlink(&mut self, dir: &Inode, name: str) -> Result[(), KernelError]
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

29. System Call: read/write

OS performance optimization requires careful attention to hardware characteristics. Modern CPUs execute instructions out-of-order and speculatively, making the cost of a branch misprediction 15-20 cycles. Memory accesses are dominated by cache behavior: an L1 hit takes ~4 cycles, an L2 hit ~12, an L3 hit ~40, and a DRAM access ~200. The kernel's hot paths -- interrupt dispatch, system call entry, scheduler context switch -- are heavily optimized to minimize cache misses and branch mispredictions. LateralusOS's scheduler data structures are designed to fit in cache: the per-CPU run queue and per-thread control block are sized to avoid cache line splits, and frequently accessed fields are grouped together.

Writing an operating system in a high-level language was considered impractical for decades because high-level languages required garbage collectors, runtime overhead, and standard libraries that themselves depended on the OS. Lateralus's freestanding mode changes this calculus: the language's core -- arithmetic, pattern matching, closures, traits, and ownership -- is available with zero runtime

dependencies. The standard library is optional and modular. A kernel written in Lateralus gets memory safety, trait-based polymorphism, and expressive pattern matching without paying any runtime tax.

The x86-64 boot sequence begins long before the kernel runs. Firmware (UEFI or legacy BIOS) initializes hardware, selects a boot device, loads the bootloader, and jumps to it in a specific CPU mode. For UEFI-based systems, the bootloader runs in 64-bit long mode with a flat memory map provided by the firmware. The kernel must still configure its own page tables, set up the interrupt descriptor table, initialize the APIC interrupt controller, and map its own code and data before it is ready to run Lateralus-level code. LateralusOS uses a minimal assembly stub for this pre-kernel setup, transitioning to Lateralus as early as possible.

Implementation Details

The physical memory allocator is the foundation of all kernel memory management. It must track which physical pages are free and which are in use, supporting allocation requests of various sizes. LateralusOS uses a buddy allocator: physical memory is organized in power-of-two blocks, and splitting and merging of blocks is O(log n) in the memory size. The allocator is initialized by parsing the memory map provided by the bootloader, marking firmware-reserved and kernel-occupied regions as allocated. After initialization, all kernel heap allocations and virtual memory mappings draw from the buddy allocator's pool.

Virtual memory allows the kernel to present each process with the illusion of a private, contiguous address space independent of the physical layout of RAM. The MMU translates virtual addresses to physical addresses using a hierarchy of page tables. On x86-64, the hierarchy is four levels deep: PML4, PDPT, PD, PT. Each level is a 4KB page containing 512 8-byte entries. A virtual address is split into five fields: the PML4 index, the PDPT index, the PD index, the PT index, and the page offset. Only present pages have valid translations; an access to a non-present page triggers a page fault, which the kernel handles by mapping the page or terminating the process.

```
// RISC-V SBI call wrapper
fn sbi_call(eid: u32, fid: u32, a0: u64, a1: u64, a2: u64) -> SbiReturn {
    // Place args in registers a0-a6, ecall into M-mode firmware
    let (error, value): (i64, i64)
    asm {
        "li a7, {eid}"
        "li a6, {fid}"
        "mv a0, {a0}"
        "mv a1, {a1}"
        "mv a2, {a2}"
        "ecall"
        "mv {error}, a0"

        "mv {value}, a1"
    }
    return SbiReturn { error, value }
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

30. System Call: open/close

Writing an operating system in a high-level language was considered impractical for decades because high-level languages required garbage collectors, runtime overhead, and standard libraries that themselves depended on the OS. Lateralus's freestanding mode changes this calculus: the language's core -- arithmetic, pattern matching, closures, traits, and ownership -- is available with zero runtime dependencies. The standard library is optional and modular. A kernel written in Lateralus gets memory safety, trait-based polymorphism, and expressive pattern matching without paying any runtime tax.

The x86-64 boot sequence begins long before the kernel runs. Firmware (UEFI or legacy BIOS) initializes hardware, selects a boot device, loads the bootloader, and jumps to it in a specific CPU mode. For UEFI-based systems, the bootloader runs in 64-bit long mode with a flat memory map provided by the firmware. The kernel must still configure its own page tables, set up the interrupt descriptor table, initialize the APIC interrupt controller, and map its own code and data before it is ready to run Lateralus-level code. LateralusOS uses a minimal assembly stub for this pre-kernel setup, transitioning to Lateralus as early as possible.

The physical memory allocator is the foundation of all kernel memory management. It must track which physical pages are free and which are in use, supporting allocation requests of various sizes. LateralusOS uses a buddy allocator: physical memory is organized in power-of-two blocks, and splitting and merging of blocks is O(log n) in the memory size. The allocator is initialized by parsing the memory map provided by the bootloader, marking firmware-reserved and kernel-occupied regions as allocated. After initialization, all kernel heap allocations and virtual memory mappings draw from the buddy allocator's pool.

Implementation Details

Virtual memory allows the kernel to present each process with the illusion of a private, contiguous address space independent of the physical layout of RAM. The MMU translates virtual addresses to physical addresses using a hierarchy of page tables. On x86-64, the hierarchy is four levels deep: PML4, PDPT, PD, PT. Each level is a 4KB page containing 512 8-byte entries. A virtual address is split into five fields: the PML4 index, the PDPT index, the PD index, the PT index, and the page offset. Only present pages have valid translations; an access to a non-present page triggers a page fault, which the kernel handles by mapping the page or terminating the process.

The kernel scheduler determines which thread runs on which CPU at each moment. LateralusOS implements a Completely Fair Scheduler (CFS) variant inspired by Linux's CFS. Each thread has a virtual runtime -- the total CPU time it has consumed, weighted by its priority. The scheduler always runs the thread with the lowest virtual runtime, which ensures fairness over time. Priority is implemented as a weight: a high-priority thread accumulates virtual runtime more slowly, so it gets proportionally more CPU time. SMP support requires a per-CPU run queue to avoid lock contention, with periodic load balancing to migrate threads between CPUs.

```
// Kernel entry point (after assembly stub sets up stack)
@no_mangle
pub fn kernel_main(boot_info: &BootInfo) {
    serial_init()
    println("[BOOT] LateralusOS kernel_main entered")

    memory::init(boot_info.memory_map)
    interrupts::init_idt()
```

```
    apic::init()
    scheduler::init()
    vfs::init()
    drivers::init_all()
    println("[BOOT] All subsystems initialized")
    scheduler::start()  // does not return
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

31. System Call: mmap/munmap

The x86-64 boot sequence begins long before the kernel runs. Firmware (UEFI or legacy BIOS) initializes hardware, selects a boot device, loads the bootloader, and jumps to it in a specific CPU mode. For UEFI-based systems, the bootloader runs in 64-bit long mode with a flat memory map provided by the firmware. The kernel must still configure its own page tables, set up the interrupt descriptor table, initialize the APIC interrupt controller, and map its own code and data before it is ready to run Lateralus-level code. LateralusOS uses a minimal assembly stub for this pre-kernel setup, transitioning to Lateralus as early as possible.

The physical memory allocator is the foundation of all kernel memory management. It must track which physical pages are free and which are in use, supporting allocation requests of various sizes. LateralusOS uses a buddy allocator: physical memory is organized in power-of-two blocks, and splitting and merging of blocks is O(log n) in the memory size. The allocator is initialized by parsing the memory map provided by the bootloader, marking firmware-reserved and kernel-occupied regions as allocated. After initialization, all kernel heap allocations and virtual memory mappings draw from the buddy allocator's pool.

Virtual memory allows the kernel to present each process with the illusion of a private, contiguous address space independent of the physical layout of RAM. The MMU translates virtual addresses to physical addresses using a hierarchy of page tables. On x86-64, the hierarchy is four levels deep: PML4, PDPT, PD, PT. Each level is a 4KB page containing 512 8-byte entries. A virtual address is split into five fields: the PML4 index, the PDPT index, the PD index, the PT index, and the page offset. Only present pages have valid translations; an access to a non-present page triggers a page fault, which the kernel handles by mapping the page or terminating the process.

Implementation Details

The kernel scheduler determines which thread runs on which CPU at each moment. LateralusOS implements a Completely Fair Scheduler (CFS) variant inspired by Linux's CFS. Each thread has a virtual runtime -- the total CPU time it has consumed, weighted by its priority. The scheduler always runs the thread with the lowest virtual runtime, which ensures fairness over time. Priority is implemented as a weight: a high-priority thread accumulates virtual runtime more slowly, so it gets proportionally more

CPU time. SMP support requires a per-CPU run queue to avoid lock contention, with periodic load balancing to migrate threads between CPUs.

Interrupt handling is the mechanism by which hardware events reach the kernel. When a keyboard is pressed, a timer fires, or a disk read completes, the hardware asserts an interrupt request line. The APIC (Advanced Programmable Interrupt Controller) arbitrates between multiple interrupt sources and delivers them to a CPU, which saves its current state and jumps to the appropriate interrupt handler via the IDT (Interrupt Descriptor Table). Interrupt handlers must be fast, non-blocking, and careful about which kernel locks they acquire. LateralusOS's type system enforces these constraints: functions marked as interrupt handlers are restricted to a subset of the kernel API.

```
// Page table entry construction
const PAGE_PRESENT:  u64 = 1 << 0
const PAGE_WRITABLE: u64 = 1 << 1
const PAGE_USER:     u64 = 1 << 2
const PAGE_NOEXEC:   u64 = 1 << 63

fn make_pte(phys: u64, flags: u64) -> u64 {
    // Align to 4KB boundary, set flags
    return (phys & 0xFFFF_FFFF_F000) | flags
}

fn map_page(virt: u64, phys: u64, writable: bool) {
    let flags = PAGE_PRESENT | (if writable { PAGE_WRITABLE } else { 0 })
    PT_ROOT.insert(virt, make_pte(phys, flags))
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

32. System Call: fork/exec/wait

The physical memory allocator is the foundation of all kernel memory management. It must track which physical pages are free and which are in use, supporting allocation requests of various sizes. LateralusOS uses a buddy allocator: physical memory is organized in power-of-two blocks, and splitting and merging of blocks is O(log n) in the memory size. The allocator is initialized by parsing the memory map provided by the bootloader, marking firmware-reserved and kernel-occupied regions as allocated. After initialization, all kernel heap allocations and virtual memory mappings draw from the buddy allocator's pool.

Virtual memory allows the kernel to present each process with the illusion of a private, contiguous address space independent of the physical layout of RAM. The MMU translates virtual addresses to physical addresses using a hierarchy of page tables. On x86-64, the hierarchy is four levels deep: PML4, PDPT, PD, PT. Each level is a 4KB page containing 512 8-byte entries. A virtual address is split into five fields: the PML4 index, the PDPT index, the PD index, the PT index, and the page offset. Only

present pages have valid translations; an access to a non-present page triggers a page fault, which the kernel handles by mapping the page or terminating the process.

The kernel scheduler determines which thread runs on which CPU at each moment. LateralusOS implements a Completely Fair Scheduler (CFS) variant inspired by Linux's CFS. Each thread has a virtual runtime -- the total CPU time it has consumed, weighted by its priority. The scheduler always runs the thread with the lowest virtual runtime, which ensures fairness over time. Priority is implemented as a weight: a high-priority thread accumulates virtual runtime more slowly, so it gets proportionally more CPU time. SMP support requires a per-CPU run queue to avoid lock contention, with periodic load balancing to migrate threads between CPUs.

Implementation Details

Interrupt handling is the mechanism by which hardware events reach the kernel. When a keyboard is pressed, a timer fires, or a disk read completes, the hardware asserts an interrupt request line. The APIC (Advanced Programmable Interrupt Controller) arbitrates between multiple interrupt sources and delivers them to a CPU, which saves its current state and jumps to the appropriate interrupt handler via the IDT (Interrupt Descriptor Table). Interrupt handlers must be fast, non-blocking, and careful about which kernel locks they acquire. LateralusOS's type system enforces these constraints: functions marked as interrupt handlers are restricted to a subset of the kernel API.

The Virtual File System (VFS) layer provides a uniform interface to all file systems. Applications use the same system calls (open, read, write, close, stat) regardless of whether the underlying storage is a local disk, a network share, a

FUSE user-space file system, or a synthetic file system like /proc. The VFS layer defines abstract types: Inode (a file or directory), Dentry (a name-to-inode mapping), and File (an open file descriptor). Concrete file system drivers implement these abstractions. LateralusOS supports FAT32 for boot media compatibility and ext2 for general storage, with the VFS layer making both appear identical to user programs.

```
// CFS scheduler -- pick thread with min vruntime
fn schedule(cpu: &mut CpuState) {
    let next = cpu.run_queue
        |> min_by(fn(t) { t.vruntime })  // O(log n) with red-black tree
    match next {
        Some(thread) => {
            cpu.run_queue.remove(thread.id)
            cpu.current = thread
            context_switch(cpu.current, thread)
        },
        None => { cpu.current = IDLE_THREAD },
    }
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

33. System Call: socket/connect/send/recv

Virtual memory allows the kernel to present each process with the illusion of a private, contiguous address space independent of the physical layout of RAM. The MMU translates virtual addresses to physical addresses using a hierarchy of page tables. On x86-64, the hierarchy is four levels deep: PML4, PDPT, PD, PT. Each level is a 4KB page containing 512 8-byte entries. A virtual address is split into five fields: the PML4 index, the PDPT index, the PD index, the PT index, and the page offset. Only present pages have valid translations; an access to a non-present page triggers a page fault, which the kernel handles by mapping the page or terminating the process.

The kernel scheduler determines which thread runs on which CPU at each moment. LateralusOS implements a Completely Fair Scheduler (CFS) variant inspired by Linux's CFS. Each thread has a virtual runtime -- the total CPU time it has consumed, weighted by its priority. The scheduler always runs the thread with the lowest virtual runtime, which ensures fairness over time. Priority is implemented as a weight: a high-priority thread accumulates virtual runtime more slowly, so it gets proportionally more CPU time. SMP support requires a per-CPU run queue to avoid lock contention, with periodic load balancing to migrate threads between CPUs.

Interrupt handling is the mechanism by which hardware events reach the kernel. When a keyboard is pressed, a timer fires, or a disk read completes, the hardware asserts an interrupt request line. The APIC (Advanced Programmable Interrupt Controller) arbitrates between multiple interrupt sources and delivers them to a CPU, which saves its current state and jumps to the appropriate interrupt handler via the IDT (Interrupt Descriptor Table). Interrupt handlers must be fast, non-blocking, and careful about which kernel locks they acquire. LateralusOS's type system enforces these constraints: functions marked as interrupt handlers are restricted to a subset of the kernel API.

Implementation Details

The Virtual File System (VFS) layer provides a uniform interface to all file systems. Applications use the same system calls (open, read, write, close, stat) regardless of whether the underlying storage is a local disk, a network share, a FUSE user-space file system, or a synthetic file system like /proc. The VFS layer defines abstract types: Inode (a file or directory), Dentry (a name-to-inode mapping), and File (an open file descriptor). Concrete file system drivers implement these abstractions. LateralusOS supports FAT32 for boot media compatibility and ext2 for general storage, with the VFS

layer making both appear identical to user programs.

Device drivers are the most error-prone component of any operating system. They run in privileged kernel mode but interact with hardware that may misbehave, return unexpected data, or fail silently. LateralusOS's driver framework uses Lateralus traits to define safe interfaces: a block device driver implements the BlockDevice trait (read_sector, write_sector, sector_count), a character device implements CharDevice (read_byte, write_byte), and a network interface implements NetDevice (send_frame, recv_frame, get_mac). The trait system ensures that all required methods are implemented and that callers cannot accidentally call driver-specific methods through the generic interface.

```
// VFS: trait for file system drivers
trait FileSystem {
    fn read(&self, inode: &Inode, buf: &mut [u8], offset: u64) -> Result[usize, KernelError]
    fn write(&mut self, inode: &Inode, buf: &[u8], offset: u64) -> Result[usize, KernelError]
    fn lookup(&self, dir: &Inode, name: str) -> Result[Inode, KernelError]
    fn readdir(&self, dir: &Inode) -> Result[list[DirEntry], KernelError]
    fn create(&mut self, dir: &Inode, name: str, kind: InodeKind) -> Result[Inode, KernelError]
```

```
    fn unlink(&mut self, dir: &Inode, name: str) -> Result[(), KernelError]
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

34. VFS Layer Architecture

The kernel scheduler determines which thread runs on which CPU at each moment. LateralusOS implements a Completely Fair Scheduler (CFS) variant inspired by Linux's CFS. Each thread has a virtual runtime -- the total CPU time it has consumed, weighted by its priority. The scheduler always runs the thread with the lowest virtual runtime, which ensures fairness over time. Priority is implemented as a weight: a high-priority thread accumulates virtual runtime more slowly, so it gets proportionally more CPU time. SMP support requires a per-CPU run queue to avoid lock contention, with periodic load balancing to migrate threads between CPUs.

Interrupt handling is the mechanism by which hardware events reach the kernel. When a keyboard is pressed, a timer fires, or a disk read completes, the hardware asserts an interrupt request line. The APIC (Advanced Programmable Interrupt Controller) arbitrates between multiple interrupt sources and delivers them to a CPU, which saves its current state and jumps to the appropriate interrupt handler via the IDT (Interrupt Descriptor Table). Interrupt handlers must be fast, non-blocking, and careful about which kernel locks they acquire. LateralusOS's type system enforces these constraints: functions marked as interrupt handlers are restricted to a subset of the kernel API.

The Virtual File System (VFS) layer provides a uniform interface to all file systems. Applications use the same system calls (open, read, write, close, stat) regardless of whether the underlying storage is a local disk, a network share, a FUSE user-space file system, or a synthetic file system like /proc. The VFS layer defines abstract types: Inode (a file or directory), Dentry (a name-to-inode mapping), and File (an open file descriptor). Concrete file system drivers implement these abstractions. LateralusOS supports FAT32 for boot media compatibility and ext2 for general storage, with the VFS layer making both appear identical to user programs.

Implementation Details

Device drivers are the most error-prone component of any operating system. They run in privileged kernel mode but interact with hardware that may misbehave, return unexpected data, or fail silently. LateralusOS's driver framework uses Lateralus traits to define safe interfaces: a block device driver implements the BlockDevice trait (read_sector, write_sector, sector_count), a character device implements CharDevice (read_byte, write_byte), and a network interface implements NetDevice (send_frame, recv_frame, get_mac). The trait system ensures that all required methods are

implemented and that callers cannot accidentally call driver-specific methods through the generic interface.

The RISC-V instruction set architecture offers a clean, modular design that makes OS implementation particularly straightforward. The ISA is divided into standard extensions: I (integer), M (multiply/divide), A (atomic), F (single-precision float), D (double-precision float), C (compressed 16-bit instructions). The privileged architecture defines three privilege levels: Machine (M-mode, highest), Supervisor (S-mode, for OS kernels), and User (U-mode, for applications). The RISC-V Supervisor Binary Interface (SBI) provides a standard firmware abstraction layer, allowing OS kernels to interact with platform hardware through well-defined firmware calls rather than direct hardware access.

```
// RISC-V SBI call wrapper
fn sbi_call(eid: u32, fid: u32, a0: u64, a1: u64, a2: u64) -> SbiReturn {
    // Place args in registers a0-a6, ecall into M-mode firmware
    let (error, value): (i64, i64)
    asm {
        "li a7, {eid}"
        "li a6, {fid}"
        "mv a0, {a0}"
        "mv a1, {a1}"
        "mv a2, {a2}"
        "ecall"
        "mv {error}, a0"
        "mv {value}, a1"
    }
    return SbiReturn { error, value }
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

35. Inode and Dentry Cache

Interrupt handling is the mechanism by which hardware events reach the kernel. When a keyboard is pressed, a timer fires, or a disk read completes, the hardware asserts an interrupt request line. The APIC (Advanced Programmable Interrupt Controller) arbitrates between multiple interrupt sources and delivers them to a CPU, which saves its current state and jumps to the appropriate interrupt handler via the IDT (Interrupt Descriptor Table). Interrupt handlers must be fast, non-blocking, and careful about which kernel locks they acquire. LateralusOS's type system enforces these constraints: functions marked as interrupt handlers are restricted to a subset of the kernel API.

The Virtual File System (VFS) layer provides a uniform interface to all file systems. Applications use the same system calls (open, read, write, close, stat) regardless of whether the underlying storage is a local disk, a network share, a FUSE user-space file system, or a synthetic file system like /proc. The VFS layer defines abstract types: Inode (a file or directory), Dentry (a name-to-inode mapping), and File (an open file descriptor). Concrete file system drivers implement these abstractions. LateralusOS supports FAT32 for boot media compatibility and ext2 for general storage, with the VFS layer making both appear

identical to user programs.

Device drivers are the most error-prone component of any operating system. They run in privileged kernel mode but interact with hardware that may misbehave, return unexpected data, or fail silently. LateralusOS's driver framework uses Lateralus traits to define safe interfaces: a block device driver implements the BlockDevice trait (read_sector, write_sector, sector_count), a character device implements CharDevice (read_byte, write_byte), and a network interface implements NetDevice (send_frame, recv_frame, get_mac). The trait system ensures that all required methods are implemented and that callers cannot accidentally call driver-specific methods through the generic interface.

Implementation Details

The RISC-V instruction set architecture offers a clean, modular design that makes OS implementation particularly straightforward. The ISA is divided into standard extensions: I (integer), M (multiply/divide), A (atomic), F (single-precision float), D (double-precision float), C (compressed 16-bit instructions). The privileged architecture defines three privilege levels: Machine (M-mode, highest), Supervisor (S-mode, for OS kernels), and User (U-mode, for applications). The RISC-V Supervisor Binary Interface (SBI) provides a standard firmware abstraction layer, allowing OS kernels to interact with platform hardware through well-defined firmware calls rather than direct hardware access.

OS performance optimization requires careful attention to hardware characteristics. Modern CPUs execute instructions out-of-order and speculatively, making the cost of a branch misprediction 15-20 cycles. Memory accesses are dominated by cache behavior: an L1 hit takes ~4 cycles, an L2 hit ~12, an L3 hit ~40, and a DRAM access ~200. The kernel's hot paths -- interrupt dispatch, system call entry, scheduler context switch -- are heavily optimized to minimize cache misses and branch mispredictions. LateralusOS's scheduler data structures are designed to fit in cache: the per-CPU run queue and per-thread control block are sized to avoid cache line splits, and frequently accessed fields are grouped together.

```
// Kernel entry point (after assembly stub sets up stack)
@no_mangle
pub fn kernel_main(boot_info: &BootInfo) {
    serial_init()
    println("[BOOT] LateralusOS kernel_main entered")
    memory::init(boot_info.memory_map)
    interrupts::init_idt()
    apic::init()
    scheduler::init()
    vfs::init()
    drivers::init_all()
    println("[BOOT] All subsystems initialized")
    scheduler::start()  // does not return
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

36. FAT32 Driver

The Virtual File System (VFS) layer provides a uniform interface to all file systems. Applications use the same system calls (open, read, write, close, stat) regardless of whether the underlying storage is a local disk, a network share, a FUSE user-space file system, or a synthetic file system like /proc. The VFS layer defines abstract types: Inode (a file or directory), Dentry (a name-to-inode mapping), and File (an open file descriptor). Concrete file system drivers implement these abstractions. LateralusOS supports FAT32 for boot media compatibility and ext2 for general storage, with the VFS layer making both appear identical to user programs.

Device drivers are the most error-prone component of any operating system. They run in privileged kernel mode but interact with hardware that may misbehave, return unexpected data, or fail silently. LateralusOS's driver framework uses Lateralus traits to define safe interfaces: a block device driver implements the BlockDevice trait (read_sector, write_sector, sector_count), a character device implements CharDevice (read_byte, write_byte), and a network interface implements NetDevice (send_frame, recv_frame, get_mac). The trait system ensures that all required methods are implemented and that callers cannot accidentally call driver-specific methods through the generic interface.

The RISC-V instruction set architecture offers a clean, modular design that makes OS implementation particularly straightforward. The ISA is divided into standard extensions: I (integer), M (multiply/divide), A (atomic), F (single-precision float), D (double-precision float), C (compressed 16-bit instructions). The privileged architecture defines

three privilege levels: Machine (M-mode, highest), Supervisor (S-mode, for OS kernels), and User (U-mode, for applications). The RISC-V Supervisor Binary Interface (SBI) provides a standard firmware abstraction layer, allowing OS kernels to interact with platform hardware through well-defined firmware calls rather than direct hardware access.

Implementation Details

OS performance optimization requires careful attention to hardware characteristics. Modern CPUs execute instructions out-of-order and speculatively, making the cost of a branch misprediction 15-20 cycles. Memory accesses are dominated by cache behavior: an L1 hit takes ~4 cycles, an L2 hit ~12, an L3 hit ~40, and a DRAM access ~200. The kernel's hot paths -- interrupt dispatch, system call entry, scheduler context switch -- are heavily optimized to minimize cache misses and branch mispredictions. LateralusOS's scheduler data structures are designed to fit in cache: the per-CPU run queue and per-thread control block are sized to avoid cache line splits, and frequently accessed fields are grouped together.

Writing an operating system in a high-level language was considered impractical for decades because high-level languages required garbage collectors, runtime overhead, and standard libraries that themselves depended on the OS. Lateralus's freestanding mode changes this calculus: the language's core -- arithmetic, pattern matching, closures, traits, and ownership -- is available with zero runtime dependencies. The standard library is optional and modular. A kernel written in Lateralus gets memory safety, trait-based polymorphism, and expressive pattern matching without paying any runtime tax.

```
// Page table entry construction
const PAGE_PRESENT:  u64 = 1 << 0
const PAGE_WRITABLE: u64 = 1 << 1
const PAGE_USER:     u64 = 1 << 2
const PAGE_NOEXEC:   u64 = 1 << 63
```

```
fn make_pte(phys: u64, flags: u64) -> u64 {
    // Align to 4KB boundary, set flags
    return (phys & 0xFFFF_FFFF_F000) | flags
}

fn map_page(virt: u64, phys: u64, writable: bool) {
    let flags = PAGE_PRESENT | (if writable { PAGE_WRITABLE } else { 0 })
    PT_ROOT.insert(virt, make_pte(phys, flags))
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

37. ext2 Driver

Device drivers are the most error-prone component of any operating system. They run in privileged kernel mode but interact with hardware that may misbehave, return unexpected data, or fail silently. LateralusOS's driver framework uses Lateralus traits to define safe interfaces: a block device driver implements the BlockDevice trait (read_sector, write_sector, sector_count), a character device implements CharDevice (read_byte, write_byte), and a network interface implements NetDevice (send_frame, recv_frame, get_mac). The trait system ensures that all required methods are implemented and that callers cannot accidentally call driver-specific methods through the generic interface.

The RISC-V instruction set architecture offers a clean, modular design that makes OS implementation particularly straightforward. The ISA is divided into standard extensions: I (integer), M (multiply/divide), A (atomic), F (single-precision float), D (double-precision float), C (compressed 16-bit instructions). The privileged architecture defines three privilege levels: Machine (M-mode, highest), Supervisor (S-mode, for OS kernels), and User (U-mode, for

applications). The RISC-V Supervisor Binary Interface (SBI) provides a standard firmware abstraction layer, allowing OS kernels to interact with platform hardware through well-defined firmware calls rather than direct hardware access.

OS performance optimization requires careful attention to hardware characteristics. Modern CPUs execute instructions out-of-order and speculatively, making the cost of a branch misprediction 15-20 cycles. Memory accesses are dominated by cache behavior: an L1 hit takes ~4 cycles, an L2 hit ~12, an L3 hit ~40, and a DRAM access ~200. The kernel's hot paths -- interrupt dispatch, system call entry, scheduler context switch -- are heavily optimized to minimize cache misses and branch mispredictions. LateralusOS's scheduler data structures are designed to fit in cache: the per-CPU run queue and per-thread control block are sized to avoid cache line splits, and frequently accessed fields are grouped together.

Implementation Details

Writing an operating system in a high-level language was considered impractical for decades because high-level languages required garbage collectors, runtime overhead, and standard libraries that themselves depended on the OS. Lateralus's freestanding mode changes this calculus: the language's core -- arithmetic, pattern matching, closures, traits, and ownership -- is available with zero runtime dependencies. The standard library is optional and modular. A kernel written in Lateralus gets memory safety, trait-based polymorphism, and expressive pattern matching without paying any runtime tax.

The x86-64 boot sequence begins long before the kernel runs. Firmware (UEFI or legacy BIOS) initializes hardware, selects a boot device, loads the bootloader, and jumps to it in a specific CPU mode. For UEFI-based systems, the bootloader runs in 64-bit long mode with a flat memory map provided by the firmware. The kernel must still configure its own page tables, set up the interrupt descriptor table, initialize the APIC interrupt controller, and map its own code and data before it is ready to run Lateralus-level code. LateralusOS uses a minimal assembly stub for this pre-kernel setup, transitioning to Lateralus as early as possible.

```
// CFS scheduler -- pick thread with min vruntime
fn schedule(cpu: &mut CpuState) {
    let next = cpu.run_queue
        |> min_by(fn(t) { t.vruntime })  // O(log n) with red-black tree
    match next {
        Some(thread) => {
            cpu.run_queue.remove(thread.id)
            cpu.current = thread
            context_switch(cpu.current, thread)
        },
        None => { cpu.current = IDLE_THREAD },
    }
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

38. tmpfs: RAM-backed File System

The RISC-V instruction set architecture offers a clean, modular design that makes OS implementation particularly straightforward. The ISA is divided into standard extensions: I (integer), M (multiply/divide), A (atomic), F (single-precision float), D (double-precision float), C (compressed 16-bit instructions). The privileged architecture defines three privilege levels: Machine (M-mode, highest), Supervisor (S-mode, for OS kernels), and User (U-mode, for applications). The RISC-V Supervisor Binary Interface (SBI) provides a standard firmware abstraction layer, allowing OS kernels to interact with platform hardware through well-defined firmware calls rather than direct hardware access.

OS performance optimization requires careful attention to hardware characteristics. Modern CPUs execute instructions out-of-order and speculatively, making the cost of a branch misprediction 15-20

cycles. Memory accesses are dominated by cache behavior: an L1 hit takes ~4 cycles, an L2 hit ~12, an L3 hit ~40, and a DRAM access ~200. The kernel's hot paths -- interrupt dispatch, system call entry, scheduler context switch -- are heavily optimized to minimize cache misses and branch mispredictions. LateralusOS's scheduler data structures are designed to fit in cache: the per-CPU run queue and per-thread control block are sized to avoid cache line splits, and frequently accessed fields are grouped together.

Writing an operating system in a high-level language was considered impractical for decades because high-level languages required garbage collectors, runtime overhead, and standard libraries that themselves depended on the OS. Lateralus's freestanding mode changes this calculus: the language's core -- arithmetic, pattern matching, closures, traits, and ownership -- is available with zero runtime dependencies. The standard library is optional and modular. A kernel written in Lateralus gets memory safety, trait-based polymorphism, and expressive pattern matching without paying any runtime tax.

Implementation Details

The x86-64 boot sequence begins long before the kernel runs. Firmware (UEFI or legacy BIOS) initializes hardware, selects a boot device, loads the bootloader, and jumps to it in a specific CPU mode. For UEFI-based systems, the bootloader runs in 64-bit long mode with a flat memory map provided by the firmware. The kernel must still configure its own page tables, set up the interrupt descriptor table, initialize the APIC interrupt controller, and map its own code and data before it is ready to run Lateralus-level code. LateralusOS uses a minimal assembly stub for this pre-kernel setup, transitioning to Lateralus as early as possible.

The physical memory allocator is the foundation of all kernel memory management. It must track which physical pages are free and which are in use, supporting allocation requests of various sizes. LateralusOS uses a buddy allocator: physical memory is organized in power-of-two blocks, and splitting and merging of blocks is O(log n) in the memory size. The allocator is initialized by parsing the memory map provided by the bootloader, marking firmware-reserved and kernel-occupied regions as allocated. After initialization, all kernel heap allocations and virtual memory mappings draw from the buddy allocator's pool.

```
// VFS: trait for file system drivers
trait FileSystem {
    fn read(&self, inode: &Inode, buf: &mut [u8], offset: u64) -> Result[usize, KernelError]
    fn write(&mut self, inode: &Inode, buf: &[u8], offset: u64) -> Result[usize, KernelError]
    fn lookup(&self, dir: &Inode, name: str) -> Result[Inode, KernelError]
    fn readdir(&self, dir: &Inode) -> Result[list[DirEntry], KernelError]
    fn create(&mut self, dir: &Inode, name: str, kind: InodeKind) -> Result[Inode, KernelError]
    fn unlink(&mut self, dir: &Inode, name: str) -> Result[(), KernelError]
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

39. Block Device Abstraction

OS performance optimization requires careful attention to hardware characteristics. Modern CPUs execute instructions out-of-order and speculatively, making the cost of a branch misprediction 15-20 cycles. Memory accesses are dominated by cache behavior: an L1 hit takes ~4 cycles, an L2 hit ~12, an L3 hit ~40, and a DRAM access ~200. The kernel's hot paths -- interrupt dispatch, system call entry, scheduler context switch -- are heavily optimized to minimize cache misses and branch mispredictions. LateralusOS's scheduler data structures are designed to fit in cache: the

per-CPU run queue and per-thread control block are sized to avoid cache line splits, and frequently accessed fields are grouped together.

Writing an operating system in a high-level language was considered impractical for decades because high-level languages required garbage collectors, runtime overhead, and standard libraries that themselves depended on the OS. Lateralus's freestanding mode changes this calculus: the language's core -- arithmetic, pattern matching, closures, traits, and ownership -- is available with zero runtime dependencies. The standard library is optional and modular. A kernel written in Lateralus gets memory safety, trait-based polymorphism, and expressive pattern matching without paying any runtime tax.

The x86-64 boot sequence begins long before the kernel runs. Firmware (UEFI or legacy BIOS) initializes hardware, selects a boot device, loads the bootloader, and jumps to it in a specific CPU mode. For UEFI-based systems, the bootloader runs in 64-bit long mode with a flat memory map provided by the firmware. The kernel must still configure its own page tables, set up the interrupt descriptor table, initialize the APIC interrupt controller, and map its own code and data before it is ready to run Lateralus-level code. LateralusOS uses a minimal assembly stub for this pre-kernel setup, transitioning to Lateralus as early as possible.

Implementation Details

The physical memory allocator is the foundation of all kernel memory management. It must track which physical pages are free and which are in use, supporting allocation requests of various sizes. LateralusOS uses a buddy allocator: physical memory is organized in power-of-two blocks, and splitting and merging of blocks is O(log n) in the memory size. The allocator is initialized by parsing the memory map provided by the bootloader, marking firmware-reserved and kernel-occupied regions as allocated. After initialization, all kernel heap allocations and virtual memory mappings draw from the buddy allocator's pool.

Virtual memory allows the kernel to present each process with the illusion of a private, contiguous address space independent of the physical layout of RAM. The MMU translates virtual addresses to physical addresses using a hierarchy of page tables. On x86-64, the hierarchy is four levels deep: PML4, PDPT, PD, PT. Each level is a 4KB page containing 512 8-byte entries. A virtual address is split into five fields: the PML4 index, the PDPT index, the PD index, the PT index, and the page offset. Only present pages have valid translations; an access to a non-present page triggers a page fault, which the kernel handles by mapping the page or terminating the process.

```
// RISC-V SBI call wrapper
fn sbi_call(eid: u32, fid: u32, a0: u64, a1: u64, a2: u64) -> SbiReturn {
    // Place args in registers a0-a6, ecall into M-mode firmware
    let (error, value): (i64, i64)
    asm {
        "li a7, {eid}"
        "li a6, {fid}"
        "mv a0, {a0}"
        "mv a1, {a1}"
        "mv a2, {a2}"
```

```
        "ecall"
        "mv {error}, a0"
        "mv {value}, a1"
    }
    return SbiReturn { error, value }
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

40. Character Device Abstraction

Writing an operating system in a high-level language was considered impractical for decades because high-level languages required garbage collectors, runtime overhead, and standard libraries that themselves depended on the OS. Lateralus's freestanding mode changes this calculus: the language's core -- arithmetic, pattern matching, closures, traits, and ownership -- is available with zero runtime dependencies. The standard library is optional and modular. A kernel written in Lateralus gets memory safety, trait-based polymorphism, and expressive pattern matching without paying any runtime tax.

The x86-64 boot sequence begins long before the kernel runs. Firmware (UEFI or legacy BIOS) initializes hardware, selects a boot device, loads the bootloader, and jumps to it in a specific CPU mode. For UEFI-based systems, the bootloader runs in 64-bit long mode with a flat memory map provided by the firmware. The kernel must still configure its own page tables, set up the interrupt descriptor table, initialize the APIC interrupt controller, and map its own code and data before it is ready to run Lateralus-level code. LateralusOS uses a minimal assembly stub for this pre-kernel setup, transitioning to Lateralus as early as possible.

The physical memory allocator is the foundation of all kernel memory management. It must track which physical pages are free and which are in use, supporting allocation requests of various sizes. LateralusOS uses a buddy allocator: physical memory is organized in power-of-two blocks, and splitting and merging of blocks is O(log n) in the memory size. The allocator is initialized by parsing the memory map provided by the bootloader, marking firmware-reserved and kernel-occupied regions as allocated. After initialization, all kernel heap allocations and virtual memory mappings draw from the buddy allocator's pool.

Implementation Details

Virtual memory allows the kernel to present each process with the illusion of a private, contiguous address space independent of the physical layout of RAM. The MMU translates virtual addresses to physical addresses using a hierarchy of page tables. On x86-64, the hierarchy is four levels deep: PML4, PDPT, PD, PT. Each level is a 4KB page containing 512 8-byte entries. A virtual address is split into five fields: the PML4 index, the PDPT index, the PD index, the PT index, and the page offset. Only present pages have valid translations; an access to a non-present page triggers a page fault, which the kernel handles by mapping the page or terminating the process.

The kernel scheduler determines which thread runs on which CPU at each moment. LateralusOS implements a Completely Fair Scheduler (CFS) variant inspired by Linux's CFS. Each thread has a virtual runtime -- the total CPU time it has consumed, weighted by its priority. The scheduler always runs the thread with the lowest virtual runtime, which ensures fairness over time. Priority is implemented as a weight: a high-priority thread accumulates virtual runtime more slowly, so it gets proportionally more CPU time. SMP support requires a per-CPU run queue to avoid lock contention, with periodic load balancing to migrate threads between CPUs.

```
// Kernel entry point (after assembly stub sets up stack)
@no_mangle
pub fn kernel_main(boot_info: &BootInfo) {
    serial_init()
    println("[BOOT] LateralusOS kernel_main entered")
    memory::init(boot_info.memory_map)
    interrupts::init_idt()
    apic::init()
    scheduler::init()
    vfs::init()
    drivers::init_all()
    println("[BOOT] All subsystems initialized")
    scheduler::start()  // does not return
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

41. PCI Bus Enumeration

The x86-64 boot sequence begins long before the kernel runs. Firmware (UEFI or legacy BIOS) initializes hardware, selects a boot device, loads the bootloader, and jumps to it in a specific CPU mode. For UEFI-based systems, the bootloader runs in 64-bit long mode with a flat memory map provided by the firmware. The kernel must still configure its own page tables, set up the interrupt descriptor table, initialize the APIC interrupt controller, and map its own code and data before it is ready to run Lateralus-level code. LateralusOS uses a minimal assembly stub for this pre-kernel setup, transitioning to Lateralus as early as possible.

The physical memory allocator is the foundation of all kernel memory management. It must track which physical pages are free and which are in use, supporting allocation requests of various sizes. LateralusOS uses a buddy allocator: physical memory is organized in power-of-two blocks, and splitting and merging of blocks is O(log n) in the memory size. The allocator is initialized by parsing the memory map provided by the bootloader, marking firmware-reserved and kernel-occupied regions as allocated. After initialization, all kernel heap allocations and virtual memory mappings draw from the buddy allocator's pool.

Virtual memory allows the kernel to present each process with the illusion of a private, contiguous address space independent of the physical layout of RAM. The MMU translates virtual addresses to physical addresses using a hierarchy of page tables. On x86-64, the hierarchy is four levels deep: PML4, PDPT, PD, PT. Each level is a 4KB page containing 512 8-byte entries. A virtual address is split into five fields: the PML4 index, the PDPT index, the PD index, the PT index, and the page offset. Only present pages have valid translations; an access to a non-present page triggers a page fault, which the kernel handles by mapping the page or terminating the process.

Implementation Details

The kernel scheduler determines which thread runs on which CPU at each moment. LateralusOS implements a Completely Fair Scheduler (CFS) variant inspired by Linux's CFS. Each thread has a virtual runtime -- the total CPU time it has consumed, weighted by its priority. The scheduler always runs the thread with the lowest virtual runtime, which ensures fairness over time. Priority is implemented as a weight: a high-priority thread accumulates virtual runtime more slowly, so it gets proportionally more CPU time. SMP support requires a per-CPU run queue to avoid lock contention, with periodic load balancing to migrate threads between CPUs.

Interrupt handling is the mechanism by which hardware events reach the kernel. When a keyboard is pressed, a timer fires, or a disk read completes, the hardware asserts an interrupt request line. The APIC (Advanced Programmable Interrupt Controller) arbitrates between multiple interrupt sources and delivers them to a CPU, which saves its current state and jumps to the appropriate interrupt handler via the IDT (Interrupt Descriptor Table). Interrupt handlers must be fast, non-blocking, and careful about which kernel locks they acquire. LateralusOS's type system enforces these constraints: functions marked as interrupt handlers are restricted to a subset of the kernel API.

```
// Page table entry construction
const PAGE_PRESENT:  u64 = 1 << 0
const PAGE_WRITABLE: u64 = 1 << 1
const PAGE_USER:     u64 = 1 << 2
const PAGE_NOEXEC:   u64 = 1 << 63

fn make_pte(phys: u64, flags: u64) -> u64 {
    // Align to 4KB boundary, set flags
    return (phys & 0xFFFF_FFFF_F000) | flags
}

fn map_page(virt: u64, phys: u64, writable: bool) {
    let flags = PAGE_PRESENT | (if writable { PAGE_WRITABLE } else { 0 })
    PT_ROOT.insert(virt, make_pte(phys, flags))
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

42. AHCI SATA Controller Driver

The physical memory allocator is the foundation of all kernel memory management. It must track which physical pages are free and which are in use, supporting allocation requests of various sizes. LateralusOS uses a buddy allocator: physical memory is organized in power-of-two blocks, and splitting and merging of blocks is O(log n) in the memory size. The allocator is initialized by parsing the memory map provided by the bootloader, marking firmware-reserved and kernel-occupied regions as allocated. After initialization, all kernel heap allocations and virtual memory mappings draw from the buddy allocator's pool.

Virtual memory allows the kernel to present each process with the illusion of a private, contiguous address space independent of the physical layout of RAM. The MMU translates virtual addresses to physical addresses using a hierarchy of page tables. On x86-64, the hierarchy is four levels deep: PML4, PDPT, PD, PT. Each level is a 4KB page containing 512 8-byte entries. A virtual address is split into five fields: the PML4 index, the PDPT index, the PD index, the PT index, and the page offset. Only present pages have valid translations; an access to a non-present page triggers a page fault, which the kernel handles by mapping the page or terminating the process.

The kernel scheduler determines which thread runs on which CPU at each moment. LateralusOS implements a Completely Fair Scheduler (CFS) variant inspired by Linux's CFS. Each thread has a virtual runtime -- the total CPU time it has consumed, weighted by its priority. The scheduler always runs the thread with the lowest virtual runtime, which ensures fairness over time. Priority is implemented as a weight: a high-priority thread accumulates virtual runtime more slowly, so it gets proportionally more CPU time. SMP support requires a per-CPU run queue to avoid lock contention, with periodic load balancing to migrate threads between CPUs.

Implementation Details

Interrupt handling is the mechanism by which hardware events reach the kernel. When a keyboard is pressed, a timer fires, or a disk read completes, the hardware asserts an interrupt request line. The APIC (Advanced Programmable Interrupt Controller) arbitrates between multiple interrupt sources and delivers them to a CPU, which saves its current state and jumps to the appropriate interrupt handler via the IDT (Interrupt Descriptor Table). Interrupt handlers must be fast, non-blocking, and careful about which kernel locks they acquire. LateralusOS's type system enforces these constraints: functions marked as interrupt handlers are restricted to a subset of the kernel API.

The Virtual File System (VFS) layer provides a uniform interface to all file systems. Applications use the same system calls (open, read, write, close, stat) regardless of whether the underlying storage is a local disk, a network share, a FUSE user-space file system, or a synthetic file system like /proc. The VFS layer defines abstract types: Inode (a file or directory), Dentry (a name-to-inode mapping), and File (an open file descriptor). Concrete file system drivers implement these abstractions. LateralusOS supports FAT32 for boot media compatibility and ext2 for general storage, with the VFS layer making both appear identical to user programs.

```
// CFS scheduler -- pick thread with min vruntime
fn schedule(cpu: &mut CpuState) {
    let next = cpu.run_queue
        |> min_by(fn(t) { t.vruntime })  // O(log n) with red-black tree
    match next {
        Some(thread) => {
            cpu.run_queue.remove(thread.id)
            cpu.current = thread
            context_switch(cpu.current, thread)
        },
        None => { cpu.current = IDLE_THREAD },
```

```
    }
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

43. USB Controller Overview

Virtual memory allows the kernel to present each process with the illusion of a private, contiguous address space independent of the physical layout of RAM. The MMU translates virtual addresses to physical addresses using a hierarchy of page tables. On x86-64, the hierarchy is four levels deep: PML4, PDPT, PD, PT. Each level is a 4KB page containing 512 8-byte entries. A virtual address is split into five fields: the PML4 index, the PDPT index, the PD index, the PT index, and the page offset. Only present pages have valid translations; an access to a non-present page triggers a page fault, which the kernel handles by mapping the page or terminating the process.

The kernel scheduler determines which thread runs on which CPU at each moment. LateralusOS implements a Completely Fair Scheduler (CFS) variant inspired by Linux's CFS. Each thread has a virtual runtime -- the total CPU time it has consumed, weighted by its priority. The scheduler always runs the thread with the lowest virtual runtime, which ensures fairness over time. Priority is implemented as a weight: a high-priority thread accumulates virtual runtime more slowly, so it gets proportionally more CPU time. SMP support requires a per-CPU run queue to avoid lock contention, with periodic load balancing to migrate threads between CPUs.

Interrupt handling is the mechanism by which hardware events reach the kernel. When a keyboard is pressed, a timer fires, or a disk read completes, the hardware asserts an interrupt request line. The APIC (Advanced Programmable Interrupt Controller) arbitrates between multiple interrupt sources and delivers them to a CPU, which saves its current state and jumps to the appropriate interrupt handler via the IDT (Interrupt Descriptor Table). Interrupt handlers must be fast, non-blocking, and careful about which kernel locks they acquire. LateralusOS's type system enforces these constraints: functions marked as interrupt handlers are restricted to a subset of the kernel API.

Implementation Details

The Virtual File System (VFS) layer provides a uniform interface to all file systems. Applications use the same system calls (open, read, write, close, stat) regardless of whether the underlying storage is a local disk, a network share, a FUSE user-space file system, or a synthetic file system like /proc. The VFS layer defines abstract types: Inode (a file or directory), Dentry (a name-to-inode mapping), and File (an open file descriptor). Concrete file system drivers implement these abstractions. LateralusOS supports FAT32 for boot media compatibility and ext2 for general storage, with the VFS layer making both appear identical to user programs.

Device drivers are the most error-prone component of any operating system. They run in privileged kernel mode but interact with hardware that may misbehave, return unexpected data, or fail silently.

LateralusOS's driver framework uses Lateralus traits to define safe interfaces: a block device driver implements the BlockDevice trait (read_sector, write_sector, sector_count), a character device implements CharDevice (read_byte, write_byte), and a network interface implements NetDevice (send_frame, recv_frame, get_mac). The trait system ensures that all required methods are implemented and that callers cannot accidentally call driver-specific methods through the generic interface.

```
// VFS: trait for file system drivers
trait FileSystem {
    fn read(&self, inode: &Inode, buf: &mut [u8], offset: u64) -> Result[usize, KernelError]
    fn write(&mut self, inode: &Inode, buf: &[u8], offset: u64) -> Result[usize, KernelError]
    fn lookup(&self, dir: &Inode, name: str) -> Result[Inode, KernelError]
    fn readdir(&self, dir: &Inode) -> Result[list[DirEntry], KernelError]
    fn create(&mut self, dir: &Inode, name: str, kind: InodeKind) -> Result[Inode, KernelError]

    fn unlink(&mut self, dir: &Inode, name: str) -> Result[(), KernelError]
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

44. UART Serial Driver

The kernel scheduler determines which thread runs on which CPU at each moment. LateralusOS implements a Completely Fair Scheduler (CFS) variant inspired by Linux's CFS. Each thread has a virtual runtime -- the total CPU time it has consumed, weighted by its priority. The scheduler always runs the thread with the lowest virtual runtime, which ensures fairness over time. Priority is implemented as a weight: a high-priority thread accumulates virtual runtime more slowly, so it gets proportionally more CPU time. SMP support requires a per-CPU run queue to avoid lock contention, with periodic load balancing to migrate threads between CPUs.

Interrupt handling is the mechanism by which hardware events reach the kernel. When a keyboard is pressed, a timer fires, or a disk read completes, the hardware asserts an interrupt request line. The APIC (Advanced Programmable Interrupt Controller) arbitrates between multiple interrupt sources and delivers them to a CPU, which saves its current state and jumps to the appropriate interrupt handler via the IDT (Interrupt Descriptor Table). Interrupt handlers must be fast, non-blocking, and careful about which kernel locks they acquire. LateralusOS's type system enforces these constraints: functions marked as interrupt handlers are restricted to a subset of the kernel API.

The Virtual File System (VFS) layer provides a uniform interface to all file systems. Applications use the same system calls (open, read, write, close, stat) regardless of whether the underlying storage is a local disk, a network share, a FUSE user-space file system, or a synthetic file system like /proc. The VFS layer defines abstract types: Inode (a file or directory), Dentry (a name-to-inode mapping), and File (an open file descriptor). Concrete file system drivers implement these abstractions. LateralusOS supports

FAT32 for boot media compatibility and ext2 for general storage, with the VFS layer making both appear identical to user programs.

Implementation Details

Device drivers are the most error-prone component of any operating system. They run in privileged kernel mode but interact with hardware that may misbehave, return unexpected data, or fail silently. LateralusOS's driver framework uses Lateralus traits to define safe interfaces: a block device driver implements the BlockDevice trait (read_sector, write_sector, sector_count), a character device implements CharDevice (read_byte, write_byte), and a network interface implements NetDevice (send_frame, recv_frame, get_mac). The trait system ensures that all required methods are implemented and that callers cannot accidentally call driver-specific methods through the generic interface.

The RISC-V instruction set architecture offers a clean, modular design that makes OS implementation particularly straightforward. The ISA is divided into standard extensions: I (integer), M (multiply/divide), A (atomic), F (single-precision float), D (double-precision float), C (compressed 16-bit instructions). The privileged architecture defines three privilege levels: Machine (M-mode, highest), Supervisor (S-mode, for OS kernels), and User (U-mode, for applications). The RISC-V Supervisor Binary Interface (SBI) provides a standard firmware abstraction layer, allowing OS kernels to interact with platform hardware through well-defined firmware calls rather than direct hardware access.

```
// RISC-V SBI call wrapper
fn sbi_call(eid: u32, fid: u32, a0: u64, a1: u64, a2: u64) -> SbiReturn {
    // Place args in registers a0-a6, ecall into M-mode firmware
    let (error, value): (i64, i64)
    asm {
        "li a7, {eid}"
        "li a6, {fid}"

        "mv a0, {a0}"
        "mv a1, {a1}"
        "mv a2, {a2}"
        "ecall"
        "mv {error}, a0"
        "mv {value}, a1"
    }
    return SbiReturn { error, value }
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

45. PS/2 Keyboard and Mouse

Interrupt handling is the mechanism by which hardware events reach the kernel. When a keyboard is pressed, a timer fires, or a disk read completes, the hardware asserts an interrupt request line. The APIC (Advanced Programmable Interrupt Controller) arbitrates between multiple interrupt sources and delivers them to a CPU, which saves its current state and jumps to the appropriate interrupt handler via the IDT (Interrupt Descriptor Table). Interrupt handlers must be fast, non-blocking, and careful about which kernel locks they acquire. LateralusOS's type system enforces these constraints: functions marked as interrupt handlers are restricted to a subset of the kernel API.

The Virtual File System (VFS) layer provides a uniform interface to all file systems. Applications use the same system calls (open, read, write, close, stat) regardless of whether the underlying storage is a local disk, a network share, a FUSE user-space file system, or a synthetic file system like /proc. The VFS layer defines abstract types: Inode (a file or directory), Dentry (a name-to-inode mapping), and File (an open file descriptor). Concrete file system drivers implement these abstractions. LateralusOS supports FAT32 for boot media compatibility and ext2 for general storage, with the VFS layer making both appear identical to user programs.

Device drivers are the most error-prone component of any operating system. They run in privileged kernel mode but interact with hardware that may misbehave, return unexpected data, or fail silently. LateralusOS's driver framework uses Lateralus traits to define safe interfaces: a block device driver implements the BlockDevice trait (read_sector, write_sector, sector_count), a character device implements CharDevice (read_byte, write_byte), and a network interface implements NetDevice (send_frame, recv_frame, get_mac). The trait system ensures that all required methods are implemented and that callers cannot accidentally call driver-specific methods through the generic interface.

Implementation Details

The RISC-V instruction set architecture offers a clean, modular design that makes OS implementation particularly straightforward. The ISA is divided into standard extensions: I (integer), M (multiply/divide), A (atomic), F (single-precision float), D (double-precision float), C (compressed 16-bit instructions). The privileged architecture defines three privilege levels: Machine (M-mode, highest), Supervisor (S-mode, for OS kernels), and User (U-mode, for applications). The RISC-V Supervisor Binary Interface (SBI) provides a standard firmware abstraction layer, allowing OS kernels to interact with platform hardware through well-defined firmware calls rather than direct hardware access.

OS performance optimization requires careful attention to hardware characteristics. Modern CPUs execute instructions out-of-order and speculatively, making the cost of a branch misprediction 15-20 cycles. Memory accesses are dominated by cache behavior: an L1 hit takes ~4 cycles, an L2 hit ~12, an L3 hit ~40, and a DRAM access ~200. The kernel's hot paths -- interrupt dispatch, system call entry, scheduler context switch -- are heavily optimized to minimize cache misses and branch mispredictions. LateralusOS's scheduler data structures are designed to fit in cache: the per-CPU run queue and per-thread control block are sized to avoid cache line splits, and frequently accessed fields are

grouped together.

```
// Kernel entry point (after assembly stub sets up stack)
@no_mangle
pub fn kernel_main(boot_info: &BootInfo) {
    serial_init()
    println("[BOOT] LateralusOS kernel_main entered")
    memory::init(boot_info.memory_map)
    interrupts::init_idt()
    apic::init()
```

```
    scheduler::init()
    vfs::init()
    drivers::init_all()
    println("[BOOT] All subsystems initialized")
    scheduler::start()  // does not return
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

46. VGA Framebuffer Driver

The Virtual File System (VFS) layer provides a uniform interface to all file systems. Applications use the same system calls (open, read, write, close, stat) regardless of whether the underlying storage is a local disk, a network share, a FUSE user-space file system, or a synthetic file system like /proc. The VFS layer defines abstract types: Inode (a file or directory), Dentry (a name-to-inode mapping), and File (an open file descriptor). Concrete file system drivers implement these abstractions. LateralusOS supports FAT32 for boot media compatibility and ext2 for general storage, with the VFS layer making both appear identical to user programs.

Device drivers are the most error-prone component of any operating system. They run in privileged kernel mode but interact with hardware that may misbehave, return unexpected data, or fail silently. LateralusOS's driver framework uses Lateralus traits to define safe interfaces: a block device driver implements the BlockDevice trait (read_sector, write_sector, sector_count), a character device implements CharDevice (read_byte, write_byte), and a network interface implements NetDevice (send_frame, recv_frame, get_mac). The trait system ensures that all required methods are implemented and that callers cannot accidentally call driver-specific methods through the generic interface.

The RISC-V instruction set architecture offers a clean, modular design that makes OS implementation particularly straightforward. The ISA is divided into standard extensions: I (integer), M (multiply/divide), A (atomic), F (single-precision float), D (double-precision float), C (compressed 16-bit instructions). The privileged architecture defines three privilege levels: Machine (M-mode, highest), Supervisor (S-mode, for OS kernels), and User (U-mode, for applications). The RISC-V Supervisor Binary Interface (SBI) provides a standard firmware abstraction layer, allowing OS kernels to interact with platform hardware through well-defined firmware calls rather than direct hardware access.

Implementation Details

OS performance optimization requires careful attention to hardware characteristics. Modern CPUs execute instructions out-of-order and speculatively, making the cost of a branch misprediction 15-20 cycles. Memory accesses are dominated by cache behavior: an L1 hit takes ~4 cycles, an L2 hit ~12, an L3 hit ~40, and a DRAM access ~200. The kernel's hot paths -- interrupt dispatch, system call entry, scheduler context switch -- are heavily optimized to minimize cache misses and branch mispredictions.

LateralusOS's scheduler data structures are designed to fit in cache: the per-CPU run queue and per-thread control block are sized to avoid cache line splits, and frequently accessed fields are grouped together.

Writing an operating system in a high-level language was considered impractical for decades because high-level languages required garbage collectors, runtime overhead, and standard libraries that themselves depended on the OS. Lateralus's freestanding mode changes this calculus: the language's core -- arithmetic, pattern matching, closures, traits, and ownership -- is available with zero runtime dependencies. The standard library is optional and modular. A kernel written in Lateralus gets memory safety, trait-based polymorphism, and expressive pattern matching without paying any runtime tax.

```
// Page table entry construction
const PAGE_PRESENT:  u64 = 1 << 0
const PAGE_WRITABLE: u64 = 1 << 1
const PAGE_USER:     u64 = 1 << 2
const PAGE_NOEXEC:   u64 = 1 << 63

fn make_pte(phys: u64, flags: u64) -> u64 {
    // Align to 4KB boundary, set flags
    return (phys & 0xFFFF_FFFF_F000) | flags
}

fn map_page(virt: u64, phys: u64, writable: bool) {
    let flags = PAGE_PRESENT | (if writable { PAGE_WRITABLE } else { 0 })
    PT_ROOT.insert(virt, make_pte(phys, flags))
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

47. Network Stack: Ethernet

Device drivers are the most error-prone component of any operating system. They run in privileged kernel mode but interact with hardware that may misbehave, return unexpected data, or fail silently. LateralusOS's driver framework uses Lateralus traits to define safe interfaces: a block device driver implements the BlockDevice trait (read_sector, write_sector, sector_count), a character device implements CharDevice (read_byte, write_byte), and a network interface implements NetDevice (send_frame, recv_frame, get_mac). The trait system ensures that all required methods are implemented and that callers cannot accidentally call driver-specific methods through the generic interface.

The RISC-V instruction set architecture offers a clean, modular design that makes OS implementation particularly straightforward. The ISA is divided into standard extensions: I (integer), M (multiply/divide), A (atomic), F (single-precision float), D (double-precision float), C (compressed 16-bit instructions). The privileged architecture defines three privilege levels: Machine (M-mode, highest), Supervisor (S-mode,

for OS kernels), and User (U-mode, for applications). The RISC-V Supervisor Binary Interface (SBI) provides a standard firmware abstraction layer, allowing OS kernels to interact with platform hardware through well-defined firmware calls rather than direct hardware access.

OS performance optimization requires careful attention to hardware characteristics. Modern CPUs execute instructions out-of-order and speculatively, making the cost of a branch misprediction 15-20 cycles. Memory accesses are dominated by cache behavior: an L1 hit takes ~4 cycles, an L2 hit ~12, an L3 hit ~40, and a DRAM access ~200. The kernel's hot paths -- interrupt dispatch, system call entry, scheduler context switch -- are heavily optimized to minimize cache misses and branch mispredictions. LateralusOS's scheduler data structures are designed to fit in cache: the per-CPU run queue and per-thread control block are sized to avoid cache line splits, and frequently accessed fields are grouped together.

Implementation Details

Writing an operating system in a high-level language was considered impractical for decades because high-level

languages required garbage collectors, runtime overhead, and standard libraries that themselves depended on the OS. Lateralus's freestanding mode changes this calculus: the language's core -- arithmetic, pattern matching, closures, traits, and ownership -- is available with zero runtime dependencies. The standard library is optional and modular. A kernel written in Lateralus gets memory safety, trait-based polymorphism, and expressive pattern matching without paying any runtime tax.

The x86-64 boot sequence begins long before the kernel runs. Firmware (UEFI or legacy BIOS) initializes hardware, selects a boot device, loads the bootloader, and jumps to it in a specific CPU mode. For UEFI-based systems, the bootloader runs in 64-bit long mode with a flat memory map provided by the firmware. The kernel must still configure its own page tables, set up the interrupt descriptor table, initialize the APIC interrupt controller, and map its own code and data before it is ready to run Lateralus-level code. LateralusOS uses a minimal assembly stub for this pre-kernel setup, transitioning to Lateralus as early as possible.

```
// CFS scheduler -- pick thread with min vruntime
fn schedule(cpu: &mut CpuState) {
    let next = cpu.run_queue
        |> min_by(fn(t) { t.vruntime })  // O(log n) with red-black tree
    match next {
        Some(thread) => {
            cpu.run_queue.remove(thread.id)
            cpu.current = thread
            context_switch(cpu.current, thread)
        },
        None => { cpu.current = IDLE_THREAD },
    }
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

48. Network Stack: ARP

The RISC-V instruction set architecture offers a clean, modular design that makes OS implementation particularly straightforward. The ISA is divided into standard extensions: I (integer), M (multiply/divide), A (atomic), F (single-precision float), D (double-precision float), C (compressed 16-bit instructions). The privileged architecture defines three privilege levels: Machine (M-mode, highest), Supervisor (S-mode, for OS kernels), and User (U-mode, for applications). The RISC-V Supervisor Binary Interface (SBI) provides a standard firmware abstraction layer, allowing OS kernels to interact with platform hardware through well-defined firmware calls rather than direct hardware access.

OS performance optimization requires careful attention to hardware characteristics. Modern CPUs execute instructions out-of-order and speculatively, making the cost of a branch misprediction 15-20 cycles. Memory accesses are dominated by cache behavior: an L1 hit takes ~4 cycles, an L2 hit ~12, an L3 hit ~40, and a DRAM access ~200. The kernel's hot paths -- interrupt dispatch, system call entry, scheduler context switch -- are heavily optimized to minimize cache misses and branch mispredictions. LateralusOS's scheduler data structures are designed to fit in cache: the per-CPU run queue and per-thread control block are sized to avoid cache line splits, and frequently accessed fields are grouped together.

Writing an operating system in a high-level language was considered impractical for decades because high-level languages required garbage collectors, runtime overhead, and standard libraries that themselves depended on the OS. Lateralus's freestanding mode changes this calculus: the language's core -- arithmetic, pattern matching, closures, traits, and ownership -- is available with zero runtime dependencies. The standard library is optional and modular. A kernel written in Lateralus gets memory safety, trait-based polymorphism, and expressive pattern matching without paying any

runtime tax.

Implementation Details

The x86-64 boot sequence begins long before the kernel runs. Firmware (UEFI or legacy BIOS) initializes hardware, selects a boot device, loads the bootloader, and jumps to it in a specific CPU mode. For UEFI-based systems, the bootloader runs in 64-bit long mode with a flat memory map provided by the firmware. The kernel must still configure its own page tables, set up the interrupt descriptor table, initialize the APIC interrupt controller, and map its own code and data before it is ready to run Lateralus-level code. LateralusOS uses a minimal assembly stub for this pre-kernel setup, transitioning to Lateralus as early as possible.

The physical memory allocator is the foundation of all kernel memory management. It must track which physical pages are free and which are in use, supporting allocation requests of various sizes. LateralusOS uses a buddy allocator: physical memory is organized in power-of-two blocks, and splitting and merging of blocks is O(log n) in the memory size. The allocator is initialized by parsing the memory map provided by the bootloader, marking firmware-reserved and kernel-occupied regions as allocated. After initialization, all kernel heap allocations and virtual memory mappings draw from the buddy allocator's pool.

```
// VFS: trait for file system drivers
trait FileSystem {
    fn read(&self, inode: &Inode, buf: &mut [u8], offset: u64) -> Result[usize, KernelError]
    fn write(&mut self, inode: &Inode, buf: &[u8], offset: u64) -> Result[usize, KernelError]
    fn lookup(&self, dir: &Inode, name: str) -> Result[Inode, KernelError]
```

```
    fn readdir(&self, dir: &Inode) -> Result[list[DirEntry], KernelError]
    fn create(&mut self, dir: &Inode, name: str, kind: InodeKind) -> Result[Inode, KernelError]
    fn unlink(&mut self, dir: &Inode, name: str) -> Result[(), KernelError]
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

49. Network Stack: IPv4

OS performance optimization requires careful attention to hardware characteristics. Modern CPUs execute instructions out-of-order and speculatively, making the cost of a branch misprediction 15-20 cycles. Memory accesses are dominated by cache behavior: an L1 hit takes ~4 cycles, an L2 hit ~12, an L3 hit ~40, and a DRAM access ~200. The kernel's hot paths -- interrupt dispatch, system call entry, scheduler context switch -- are heavily optimized to minimize cache misses and branch mispredictions. LateralusOS's scheduler data structures are designed to fit in cache: the per-CPU run queue and per-thread control block are sized to avoid cache line splits, and frequently accessed fields are grouped together.

Writing an operating system in a high-level language was considered impractical for decades because high-level languages required garbage collectors, runtime overhead, and standard libraries that themselves depended on the OS. Lateralus's freestanding mode changes this calculus: the language's core -- arithmetic, pattern matching, closures, traits, and ownership -- is available with zero runtime dependencies. The standard library is optional and modular. A kernel written in Lateralus gets memory safety, trait-based polymorphism, and expressive pattern matching without paying any runtime tax.

The x86-64 boot sequence begins long before the kernel runs. Firmware (UEFI or legacy BIOS) initializes hardware, selects a boot device, loads the bootloader, and jumps to it in a specific CPU mode. For UEFI-based systems, the bootloader runs in 64-bit long mode with a flat memory map provided by the firmware. The kernel must still configure its own page tables, set up the interrupt descriptor table, initialize the APIC interrupt controller, and map its own code and

data before it is ready to run Lateralus-level code. LateralusOS uses a minimal assembly stub for this pre-kernel setup, transitioning to Lateralus as early as possible.

Implementation Details

The physical memory allocator is the foundation of all kernel memory management. It must track which physical pages are free and which are in use, supporting allocation requests of various sizes. LateralusOS uses a buddy allocator: physical memory is organized in power-of-two blocks, and splitting and merging of blocks is O(log n) in the memory size. The allocator is initialized by parsing the memory map provided by the bootloader, marking firmware-reserved and kernel-occupied regions as allocated. After initialization, all kernel heap allocations and virtual memory mappings draw from the buddy allocator's pool.

Virtual memory allows the kernel to present each process with the illusion of a private, contiguous address space independent of the physical layout of RAM. The MMU translates virtual addresses to physical addresses using a hierarchy of page tables. On x86-64, the hierarchy is four levels deep: PML4, PDPT, PD, PT. Each level is a 4KB page containing 512 8-byte entries. A virtual address is split into five fields: the PML4 index, the PDPT index, the PD index, the PT index, and the page offset. Only present pages have valid translations; an access to a non-present page triggers a page fault, which the kernel handles by mapping the page or terminating the process.

```
// RISC-V SBI call wrapper
fn sbi_call(eid: u32, fid: u32, a0: u64, a1: u64, a2: u64) -> SbiReturn {
    // Place args in registers a0-a6, ecall into M-mode firmware
    let (error, value): (i64, i64)
    asm {
        "li a7, {eid}"
        "li a6, {fid}"
        "mv a0, {a0}"
        "mv a1, {a1}"
        "mv a2, {a2}"
        "ecall"
        "mv {error}, a0"
        "mv {value}, a1"
    }
    return SbiReturn { error, value }
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

50. Network Stack: ICMP

Writing an operating system in a high-level language was considered impractical for decades because high-level languages required garbage collectors, runtime overhead, and standard libraries that themselves depended on the OS. Lateralus's freestanding mode changes this calculus: the language's core -- arithmetic, pattern matching, closures, traits, and ownership -- is available with zero runtime dependencies. The standard library is optional and modular. A kernel written in Lateralus gets memory safety, trait-based polymorphism, and expressive pattern matching without paying any runtime tax.

The x86-64 boot sequence begins long before the kernel runs. Firmware (UEFI or legacy BIOS) initializes hardware, selects a boot device, loads the bootloader, and jumps to it in a specific CPU mode. For UEFI-based systems, the bootloader runs in 64-bit long mode with a flat memory map provided by the firmware. The kernel must still configure its own page tables, set up the interrupt descriptor table, initialize the APIC interrupt controller, and map its own code and data before it is ready to run Lateralus-level code. LateralusOS uses a minimal assembly stub for this pre-kernel setup,

transitioning to Lateralus as early as possible.

The physical memory allocator is the foundation of all kernel memory management. It must track which physical pages are free and which are in use, supporting allocation requests of various sizes. LateralusOS uses a buddy allocator: physical memory is organized in power-of-two blocks, and splitting and merging of blocks is O(log n) in the memory size. The allocator is initialized by parsing the memory map provided by the bootloader, marking firmware-reserved and kernel-occupied regions as allocated. After initialization, all kernel heap allocations and virtual memory mappings draw from the buddy allocator's pool.

Implementation Details

Virtual memory allows the kernel to present each process with the illusion of a private, contiguous address space independent of the physical layout of RAM. The MMU translates virtual addresses to physical addresses using a hierarchy of page tables. On x86-64, the hierarchy is four levels deep: PML4, PDPT, PD, PT. Each level is a 4KB page containing 512 8-byte entries. A virtual address is split into five fields: the PML4 index, the PDPT index, the PD index, the PT index, and the page offset. Only present pages have valid translations; an access to a non-present page triggers a page fault, which the kernel handles by mapping the page or terminating the process.

The kernel scheduler determines which thread runs on which CPU at each moment. LateralusOS implements a Completely Fair Scheduler (CFS) variant inspired by Linux's CFS. Each thread has a virtual runtime -- the total CPU time it has consumed, weighted by its priority. The scheduler always runs the thread with the lowest virtual runtime, which ensures fairness over time. Priority is implemented as a weight: a high-priority thread accumulates virtual runtime more slowly, so it gets proportionally more CPU time. SMP support requires a per-CPU run queue to avoid lock contention, with periodic load balancing to migrate threads between CPUs.

```
// Kernel entry point (after assembly stub sets up stack)
@no_mangle
pub fn kernel_main(boot_info: &BootInfo) {
    serial_init()
    println("[BOOT] LateralusOS kernel_main entered")
    memory::init(boot_info.memory_map)
    interrupts::init_idt()
    apic::init()
    scheduler::init()
    vfs::init()
    drivers::init_all()
    println("[BOOT] All subsystems initialized")
    scheduler::start()  // does not return
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

51. Network Stack: TCP State Machine

The x86-64 boot sequence begins long before the kernel runs. Firmware (UEFI or legacy BIOS) initializes hardware, selects a boot device, loads the bootloader, and jumps to it in a specific CPU mode. For UEFI-based systems, the bootloader runs in 64-bit long mode with a flat memory map provided by the firmware. The kernel must still configure its own page tables, set up the interrupt descriptor table, initialize the APIC interrupt controller, and map its own code and data before it is ready to run Lateralus-level code. LateralusOS uses a minimal assembly stub for this pre-kernel setup, transitioning to Lateralus as early as possible.

The physical memory allocator is the foundation of all kernel memory management. It must track which physical pages

are free and which are in use, supporting allocation requests of various sizes. LateralusOS uses a buddy allocator: physical memory is organized in power-of-two blocks, and splitting and merging of blocks is O(log n) in the memory size. The allocator is initialized by parsing the memory map provided by the bootloader, marking firmware-reserved and kernel-occupied regions as allocated. After initialization, all kernel heap allocations and virtual memory mappings draw from the buddy allocator's pool.

Virtual memory allows the kernel to present each process with the illusion of a private, contiguous address space independent of the physical layout of RAM. The MMU translates virtual addresses to physical addresses using a hierarchy of page tables. On x86-64, the hierarchy is four levels deep: PML4, PDPT, PD, PT. Each level is a 4KB page containing 512 8-byte entries. A virtual address is split into five fields: the PML4 index, the PDPT index, the PD index, the PT index, and the page offset. Only present pages have valid translations; an access to a non-present page triggers a page fault, which the kernel handles by mapping the page or terminating the process.

Implementation Details

The kernel scheduler determines which thread runs on which CPU at each moment. LateralusOS implements a Completely Fair Scheduler (CFS) variant inspired by Linux's CFS. Each thread has a virtual runtime -- the total CPU time it has consumed, weighted by its priority. The scheduler always runs the thread with the lowest virtual runtime, which ensures fairness over time. Priority is implemented as a weight: a high-priority thread accumulates virtual runtime more slowly, so it gets proportionally more CPU time. SMP support requires a per-CPU run queue to avoid lock contention, with periodic load balancing to migrate threads between CPUs.

Interrupt handling is the mechanism by which hardware events reach the kernel. When a keyboard is pressed, a timer fires, or a disk read completes, the hardware asserts an interrupt request line. The APIC (Advanced Programmable Interrupt Controller) arbitrates between multiple interrupt sources and delivers them to a CPU, which saves its current state and jumps to the appropriate interrupt handler via the IDT (Interrupt Descriptor Table). Interrupt handlers must be fast, non-blocking, and careful about which kernel locks they acquire. LateralusOS's type system enforces these constraints: functions marked as interrupt handlers are restricted to a subset of the kernel API.

```
// Page table entry construction
const PAGE_PRESENT:  u64 = 1 << 0
const PAGE_WRITABLE: u64 = 1 << 1
const PAGE_USER:     u64 = 1 << 2
const PAGE_NOEXEC:   u64 = 1 << 63

fn make_pte(phys: u64, flags: u64) -> u64 {
    // Align to 4KB boundary, set flags
    return (phys & 0xFFFF_FFFF_F000) | flags
}
```

```
fn map_page(virt: u64, phys: u64, writable: bool) {
    let flags = PAGE_PRESENT | (if writable { PAGE_WRITABLE } else { 0 })
    PT_ROOT.insert(virt, make_pte(phys, flags))
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

52. Network Stack: UDP

The physical memory allocator is the foundation of all kernel memory management. It must track which physical pages are free and which are in use, supporting allocation requests of various sizes. LateralusOS uses a buddy allocator: physical memory is organized in power-of-two blocks, and splitting and merging of blocks is O(log n) in the memory size.

The allocator is initialized by parsing the memory map provided by the bootloader, marking firmware-reserved and kernel-occupied regions as allocated. After initialization, all kernel heap allocations and virtual memory mappings draw from the buddy allocator's pool.

Virtual memory allows the kernel to present each process with the illusion of a private, contiguous address space independent of the physical layout of RAM. The MMU translates virtual addresses to physical addresses using a hierarchy of page tables. On x86-64, the hierarchy is four levels deep: PML4, PDPT, PD, PT. Each level is a 4KB page containing 512 8-byte entries. A virtual address is split into five fields: the PML4 index, the PDPT index, the PD index, the PT index, and the page offset. Only present pages have valid translations; an access to a non-present page triggers a page fault, which the kernel handles by mapping the page or terminating the process.

The kernel scheduler determines which thread runs on which CPU at each moment. LateralusOS implements a Completely Fair Scheduler (CFS) variant inspired by Linux's CFS. Each thread has a virtual runtime -- the total CPU time it has consumed, weighted by its priority. The scheduler always runs the thread with the lowest virtual runtime, which ensures fairness over time. Priority is implemented as a weight: a high-priority thread accumulates virtual runtime more slowly, so it gets proportionally more CPU time. SMP support requires a per-CPU run queue to avoid lock contention, with periodic load balancing to migrate threads between CPUs.

Implementation Details

Interrupt handling is the mechanism by which hardware events reach the kernel. When a keyboard is pressed, a timer fires, or a disk read completes, the hardware asserts an interrupt request line. The APIC (Advanced Programmable Interrupt Controller) arbitrates between multiple interrupt sources and delivers them to a CPU, which saves its current state and jumps to the appropriate interrupt handler via the IDT (Interrupt Descriptor Table). Interrupt handlers must be fast, non-blocking, and careful about which kernel locks they acquire. LateralusOS's type system enforces these constraints: functions marked as interrupt handlers are restricted to a subset of the kernel API.

The Virtual File System (VFS) layer provides a uniform interface to all file systems. Applications use the same system calls (open, read, write, close, stat) regardless of whether the underlying storage is a local disk, a network share, a FUSE user-space file system, or a synthetic file system like /proc. The VFS layer defines abstract types: Inode (a file or directory), Dentry (a name-to-inode mapping), and File (an open file descriptor). Concrete file system drivers implement these abstractions. LateralusOS supports FAT32 for boot media compatibility and ext2 for general storage, with the VFS layer making both appear identical to user programs.

```
// CFS scheduler -- pick thread with min vruntime
fn schedule(cpu: &mut CpuState) {
    let next = cpu.run_queue
        |> min_by(fn(t) { t.vruntime })  // O(log n) with red-black tree
    match next {
        Some(thread) => {
            cpu.run_queue.remove(thread.id)
            cpu.current = thread
            context_switch(cpu.current, thread)
        },
        None => { cpu.current = IDLE_THREAD },
    }
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

53. Network Stack: DHCP Client

Virtual memory allows the kernel to present each process with the illusion of a private, contiguous address space independent of the physical layout of RAM. The MMU translates virtual addresses to physical addresses using a hierarchy of page tables. On x86-64, the hierarchy is four levels deep: PML4, PDPT, PD, PT. Each level is a 4KB page containing 512 8-byte entries. A virtual address is split into five fields: the PML4 index, the PDPT index, the PD index, the PT index, and the page offset. Only present pages have valid translations; an access to a non-present page triggers a page fault, which the kernel handles by mapping the page or terminating the process.

The kernel scheduler determines which thread runs on which CPU at each moment. LateralusOS implements a Completely Fair Scheduler (CFS) variant inspired by Linux's CFS. Each thread has a virtual runtime -- the total CPU time it has consumed, weighted by its priority. The scheduler always runs the thread with the lowest virtual runtime, which ensures fairness over time. Priority is implemented as a weight: a high-priority thread accumulates virtual runtime more slowly, so it gets proportionally more CPU time. SMP support requires a per-CPU run queue to avoid lock contention, with periodic load balancing to migrate threads between CPUs.

Interrupt handling is the mechanism by which hardware events reach the kernel. When a keyboard is pressed, a timer fires, or a disk read completes, the hardware asserts an interrupt request line. The APIC (Advanced Programmable Interrupt Controller) arbitrates between multiple interrupt sources and delivers them to a CPU, which saves its current state and jumps to the appropriate interrupt handler via the IDT (Interrupt Descriptor Table). Interrupt handlers must be fast, non-blocking, and careful about which kernel locks they acquire. LateralusOS's type system enforces these constraints: functions marked as interrupt handlers are restricted to a subset of the kernel API.

Implementation Details

The Virtual File System (VFS) layer provides a uniform interface to all file systems. Applications use the same system calls (open, read, write, close, stat) regardless of whether the underlying storage is a local disk, a network share, a FUSE user-space file system, or a synthetic file system like /proc. The VFS layer defines abstract types: Inode (a file or directory), Dentry (a name-to-inode mapping), and File (an open file descriptor). Concrete file system drivers implement these abstractions. LateralusOS supports FAT32 for boot media compatibility and ext2 for general storage, with the VFS layer making both appear identical to user programs.

Device drivers are the most error-prone component of any operating system. They run in privileged kernel mode but interact with hardware that may misbehave, return unexpected data, or fail silently. LateralusOS's driver framework uses Lateralus traits to define safe interfaces: a block device driver implements the BlockDevice trait (read_sector, write_sector, sector_count), a character device implements CharDevice (read_byte, write_byte), and a network interface implements NetDevice (send_frame, recv_frame, get_mac). The trait system ensures that all required methods are implemented and that callers cannot accidentally call driver-specific methods through the generic interface.

```
// VFS: trait for file system drivers
trait FileSystem {
    fn read(&self, inode: &Inode, buf: &mut [u8], offset: u64) -> Result[usize, KernelError]
    fn write(&mut self, inode: &Inode, buf: &[u8], offset: u64) -> Result[usize, KernelError]
    fn lookup(&self, dir: &Inode, name: str) -> Result[Inode, KernelError]
    fn readdir(&self, dir: &Inode) -> Result[list[DirEntry], KernelError]
    fn create(&mut self, dir: &Inode, name: str, kind: InodeKind) -> Result[Inode, KernelError]
    fn unlink(&mut self, dir: &Inode, name: str) -> Result[(), KernelError]
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

54. RISC-V: Privilege Levels

The kernel scheduler determines which thread runs on which CPU at each moment. LateralusOS implements a Completely Fair Scheduler (CFS) variant inspired by Linux's CFS. Each thread has a virtual runtime -- the total CPU time it has consumed, weighted by its priority. The scheduler always runs the thread with the lowest virtual runtime, which ensures fairness over time. Priority is implemented as a weight: a high-priority thread accumulates virtual runtime more slowly, so it gets proportionally more CPU time. SMP support requires a per-CPU run queue to avoid lock contention, with periodic load balancing to migrate threads between CPUs.

Interrupt handling is the mechanism by which hardware events reach the kernel. When a keyboard is pressed, a timer fires, or a disk read completes, the hardware asserts an interrupt request line. The APIC (Advanced Programmable Interrupt Controller) arbitrates between multiple interrupt sources and delivers them to a CPU, which saves its current state and jumps to the appropriate interrupt handler via the IDT (Interrupt Descriptor Table). Interrupt handlers must be fast, non-blocking, and careful about which kernel locks they acquire. LateralusOS's type system enforces these constraints: functions marked as interrupt handlers are restricted to a subset of the kernel API.

The Virtual File System (VFS) layer provides a uniform interface to all file systems. Applications use the same system calls (open, read, write, close, stat) regardless of whether the underlying storage is a local disk, a network share, a FUSE user-space file system, or a synthetic file system like /proc. The VFS layer defines abstract types: Inode (a file or directory), Dentry (a name-to-inode mapping), and File (an open file descriptor). Concrete file system drivers implement these abstractions. LateralusOS supports FAT32 for boot media compatibility and ext2 for general storage, with the VFS layer making both appear identical to user programs.

Implementation Details

Device drivers are the most error-prone component of any operating system. They run in privileged kernel mode but interact with hardware that may misbehave, return unexpected data, or fail silently. LateralusOS's driver framework uses Lateralus traits to define safe interfaces: a block device driver implements the BlockDevice trait (read_sector, write_sector, sector_count), a character device implements CharDevice (read_byte, write_byte), and a network interface implements NetDevice (send_frame, recv_frame, get_mac). The trait system ensures that all required methods are implemented and that callers cannot accidentally call driver-specific methods through the generic interface.

The RISC-V instruction set architecture offers a clean, modular design that makes OS implementation particularly straightforward. The ISA is divided into standard extensions: I (integer), M (multiply/divide), A (atomic), F (single-precision float), D (double-precision float), C (compressed 16-bit instructions). The privileged architecture defines three privilege levels: Machine (M-mode, highest), Supervisor (S-mode, for OS kernels), and User (U-mode, for applications). The RISC-V Supervisor Binary Interface (SBI) provides a standard firmware abstraction layer, allowing OS kernels to interact with platform hardware through well-defined firmware calls rather than direct hardware access.

```
// RISC-V SBI call wrapper
fn sbi_call(eid: u32, fid: u32, a0: u64, a1: u64, a2: u64) -> SbiReturn {
    // Place args in registers a0-a6, ecall into M-mode firmware
    let (error, value): (i64, i64)
    asm {
        "li a7, {eid}"
        "li a6, {fid}"
        "mv a0, {a0}"
        "mv a1, {a1}"
        "mv a2, {a2}"
```

```
        "ecall"
        "mv {error}, a0"
        "mv {value}, a1"
    }
    return SbiReturn { error, value }
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

55. RISC-V: SBI Interface

Interrupt handling is the mechanism by which hardware events reach the kernel. When a keyboard is pressed, a timer fires, or a disk read completes, the hardware asserts an interrupt request line. The APIC (Advanced Programmable Interrupt Controller) arbitrates between multiple interrupt sources and delivers them to a CPU, which saves its current state and jumps to the appropriate interrupt handler via the IDT (Interrupt Descriptor Table). Interrupt handlers must be fast, non-blocking, and careful about which kernel locks they acquire. LateralusOS's type system enforces these constraints: functions marked as interrupt handlers are restricted to a subset of the kernel API.

The Virtual File System (VFS) layer provides a uniform interface to all file systems. Applications use the same system calls (open, read, write, close, stat) regardless of whether the underlying storage is a local disk, a network share, a FUSE user-space file system, or a synthetic file system like /proc. The VFS layer defines abstract types: Inode (a file or directory), Dentry (a name-to-inode mapping), and File (an open file descriptor). Concrete file system drivers implement these abstractions. LateralusOS supports FAT32 for boot media compatibility and ext2 for general storage, with the VFS layer making both appear identical to user programs.

Device drivers are the most error-prone component of any operating system. They run in privileged kernel mode but interact with hardware that may misbehave, return unexpected data, or fail silently. LateralusOS's driver framework uses Lateralus traits to define safe interfaces: a block device driver implements the BlockDevice trait (read_sector, write_sector, sector_count), a character device implements CharDevice (read_byte, write_byte), and a network interface implements NetDevice (send_frame, recv_frame, get_mac). The trait system ensures that all required methods are implemented and that callers cannot accidentally call driver-specific methods through the generic interface.

Implementation Details

The RISC-V instruction set architecture offers a clean, modular design that makes OS implementation particularly straightforward. The ISA is divided into standard extensions: I (integer), M (multiply/divide), A (atomic), F (single-precision float), D (double-precision float), C (compressed 16-bit instructions). The privileged architecture defines three privilege levels: Machine (M-mode, highest), Supervisor (S-mode, for OS kernels), and User (U-mode, for applications). The RISC-V Supervisor Binary Interface (SBI)

provides a standard firmware abstraction layer, allowing OS kernels to interact with platform hardware through well-defined firmware calls rather than direct hardware access.

OS performance optimization requires careful attention to hardware characteristics. Modern CPUs execute instructions out-of-order and speculatively, making the cost of a branch misprediction 15-20 cycles. Memory accesses are dominated by cache behavior: an L1 hit takes ~4 cycles, an L2 hit ~12, an L3 hit ~40, and a DRAM access ~200. The kernel's hot paths -- interrupt dispatch, system call entry, scheduler context switch -- are heavily optimized to minimize cache misses and branch mispredictions. LateralusOS's scheduler data structures are designed to fit in cache: the per-CPU run queue and per-thread control block are sized to avoid cache line splits, and frequently accessed fields are grouped together.

```
// Kernel entry point (after assembly stub sets up stack)
@no_mangle
pub fn kernel_main(boot_info: &BootInfo) {
    serial_init()
    println("[BOOT] LateralusOS kernel_main entered")
    memory::init(boot_info.memory_map)
    interrupts::init_idt()
    apic::init()
    scheduler::init()
    vfs::init()
    drivers::init_all()
    println("[BOOT] All subsystems initialized")
    scheduler::start()  // does not return
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

56. RISC-V: Virtual Memory (Sv39/Sv48)

The Virtual File System (VFS) layer provides a uniform interface to all file systems. Applications use the same system calls (open, read, write, close, stat) regardless of whether the underlying storage is a local disk, a network share, a FUSE user-space file system, or a synthetic file system like /proc. The VFS layer defines abstract types: Inode (a file or directory), Dentry (a name-to-inode mapping), and File (an open file descriptor). Concrete file system drivers implement these abstractions. LateralusOS supports FAT32 for boot media compatibility and ext2 for general storage, with the VFS layer making both appear identical to user programs.

Device drivers are the most error-prone component of any operating system. They run in privileged kernel mode but interact with hardware that may misbehave, return unexpected data, or fail silently. LateralusOS's driver framework uses Lateralus traits to define safe interfaces: a block device driver implements the BlockDevice trait (read_sector, write_sector, sector_count), a character device

implements CharDevice (read_byte, write_byte), and a network interface implements NetDevice (send_frame, recv_frame, get_mac). The trait system ensures that all required methods are implemented and that callers cannot accidentally call driver-specific methods through the generic interface.

The RISC-V instruction set architecture offers a clean, modular design that makes OS implementation particularly straightforward. The ISA is divided into standard extensions: I (integer), M (multiply/divide), A (atomic), F (single-precision float), D (double-precision float), C (compressed 16-bit instructions). The privileged architecture defines three privilege levels: Machine (M-mode, highest), Supervisor (S-mode, for OS kernels), and User (U-mode, for applications). The RISC-V Supervisor Binary Interface (SBI) provides a standard firmware abstraction layer, allowing OS kernels to interact with platform hardware through well-defined firmware calls rather than direct hardware access.

Implementation Details

OS performance optimization requires careful attention to hardware characteristics. Modern CPUs execute instructions out-of-order and speculatively, making the cost of a branch misprediction 15-20 cycles. Memory accesses are dominated by cache behavior: an L1 hit takes ~4 cycles, an L2 hit ~12, an L3 hit ~40, and a DRAM access ~200. The kernel's hot paths -- interrupt dispatch, system call entry, scheduler context switch -- are heavily optimized to minimize cache misses and branch mispredictions. LateralusOS's scheduler data structures are designed to fit in cache: the per-CPU run queue and per-thread control block are sized to avoid cache line splits, and frequently accessed fields are grouped together.

Writing an operating system in a high-level language was considered impractical for decades because high-level languages required garbage collectors, runtime overhead, and standard libraries that themselves depended on the OS. Lateralus's freestanding mode changes this calculus: the language's core -- arithmetic, pattern matching, closures, traits, and ownership -- is available with zero runtime dependencies. The standard library is optional and modular. A kernel written in Lateralus gets memory safety, trait-based polymorphism, and expressive pattern matching without paying any runtime tax.

```
// Page table entry construction
const PAGE_PRESENT:  u64 = 1 << 0
const PAGE_WRITABLE: u64 = 1 << 1
const PAGE_USER:     u64 = 1 << 2
const PAGE_NOEXEC:   u64 = 1 << 63

fn make_pte(phys: u64, flags: u64) -> u64 {
    // Align to 4KB boundary, set flags
    return (phys & 0xFFFF_FFFF_F000) | flags

}

fn map_page(virt: u64, phys: u64, writable: bool) {
    let flags = PAGE_PRESENT | (if writable { PAGE_WRITABLE } else { 0 })
    PT_ROOT.insert(virt, make_pte(phys, flags))
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

57. RISC-V: Interrupt Handling (PLIC/CLINT)

Device drivers are the most error-prone component of any operating system. They run in privileged kernel mode but interact with hardware that may misbehave, return unexpected data, or fail silently. LateralusOS's driver framework uses Lateralus traits to define safe interfaces: a block device driver implements the BlockDevice trait (read_sector, write_sector, sector_count), a character device implements CharDevice (read_byte, write_byte), and a network interface implements NetDevice (send_frame, recv_frame, get_mac). The trait system ensures that all required methods are implemented and that callers cannot accidentally call driver-specific methods through the generic interface.

The RISC-V instruction set architecture offers a clean, modular design that makes OS implementation particularly straightforward. The ISA is divided into standard extensions: I (integer), M (multiply/divide), A (atomic), F (single-precision float), D (double-precision float), C (compressed 16-bit instructions). The privileged architecture defines three privilege levels: Machine (M-mode, highest), Supervisor (S-mode, for OS kernels), and User (U-mode, for applications). The RISC-V Supervisor Binary Interface (SBI) provides a standard firmware abstraction layer, allowing OS kernels to interact with platform hardware through well-defined firmware calls rather than direct hardware access.

OS performance optimization requires careful attention to hardware characteristics. Modern CPUs execute instructions out-of-order and speculatively, making the cost of a branch misprediction 15-20 cycles. Memory accesses are dominated by cache behavior: an L1 hit takes ~4 cycles, an L2 hit ~12, an L3 hit ~40, and a DRAM access ~200. The kernel's hot paths -- interrupt dispatch, system call entry, scheduler context switch -- are heavily optimized to minimize cache misses and branch mispredictions. LateralusOS's scheduler data structures are designed to fit in cache: the per-CPU run queue and per-thread control block are sized to avoid cache line splits, and frequently accessed fields are grouped together.

Implementation Details

Writing an operating system in a high-level language was considered impractical for decades because high-level languages required garbage collectors, runtime overhead, and standard libraries that themselves depended on the OS. Lateralus's freestanding mode changes this calculus: the language's core -- arithmetic, pattern matching, closures, traits, and ownership -- is available with zero runtime dependencies. The standard library is optional and modular. A kernel written in Lateralus gets memory safety, trait-based polymorphism, and expressive pattern matching without paying any runtime tax.

The x86-64 boot sequence begins long before the kernel runs. Firmware (UEFI or legacy BIOS) initializes hardware, selects a boot device, loads the bootloader, and jumps to it in a specific CPU mode. For UEFI-based systems, the bootloader runs in 64-bit long mode with a flat memory map provided by the firmware. The kernel must still configure its own page tables, set up the interrupt descriptor table, initialize the APIC interrupt controller, and map its own code and data before it is ready to run Lateralus-level code. LateralusOS uses a minimal assembly stub for this pre-kernel setup, transitioning to Lateralus as early as possible.

```
// CFS scheduler -- pick thread with min vruntime
fn schedule(cpu: &mut CpuState) {

    let next = cpu.run_queue
        |> min_by(fn(t) { t.vruntime })  // O(log n) with red-black tree
    match next {
```

```
        Some(thread) => {
            cpu.run_queue.remove(thread.id)
            cpu.current = thread
            context_switch(cpu.current, thread)
        },
        None => { cpu.current = IDLE_THREAD },
    }
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

58. RISC-V: UART NS16550 Driver

The RISC-V instruction set architecture offers a clean, modular design that makes OS implementation particularly straightforward. The ISA is divided into standard extensions: I (integer), M (multiply/divide), A (atomic), F (single-precision float), D (double-precision float), C (compressed 16-bit instructions). The privileged architecture defines three privilege levels: Machine (M-mode, highest), Supervisor (S-mode, for OS kernels), and User (U-mode, for applications). The RISC-V Supervisor Binary Interface (SBI) provides a standard firmware abstraction layer, allowing OS kernels to interact with platform hardware through well-defined firmware calls rather than direct hardware access.

OS performance optimization requires careful attention to hardware characteristics. Modern CPUs execute instructions out-of-order and speculatively, making the cost of a branch misprediction 15-20 cycles. Memory accesses are dominated by cache behavior: an L1 hit takes ~4 cycles, an L2 hit ~12, an L3 hit ~40, and a DRAM access ~200. The kernel's hot paths -- interrupt dispatch, system call entry, scheduler context switch -- are heavily optimized to minimize cache misses and branch mispredictions. LateralusOS's scheduler data structures are designed to fit in cache: the per-CPU run queue and per-thread control block are sized to avoid cache line splits, and frequently accessed fields are grouped together.

Writing an operating system in a high-level language was considered impractical for decades because high-level languages required garbage collectors, runtime overhead, and standard libraries that themselves depended on the OS. Lateralus's freestanding mode changes this calculus: the language's core -- arithmetic, pattern matching, closures, traits, and ownership -- is available with zero runtime dependencies. The standard library is optional and modular. A kernel written in Lateralus gets memory safety, trait-based polymorphism, and expressive pattern matching without paying any runtime tax.

Implementation Details

The x86-64 boot sequence begins long before the kernel runs. Firmware (UEFI or legacy BIOS) initializes hardware, selects a boot device, loads the bootloader, and jumps to it in a specific CPU mode. For UEFI-based systems, the bootloader runs in 64-bit long mode with a flat memory map provided by the firmware. The kernel must still configure its own page tables, set up the interrupt descriptor table,

initialize the APIC interrupt controller, and map its own code and data before it is ready to run Lateralus-level code. LateralusOS uses a minimal assembly stub for this pre-kernel setup, transitioning to Lateralus as early as possible.

The physical memory allocator is the foundation of all kernel memory management. It must track which physical pages are free and which are in use, supporting allocation requests of various sizes. LateralusOS uses a buddy allocator: physical memory is organized in power-of-two blocks, and splitting and merging of blocks is O(log n) in the memory size. The allocator is initialized by parsing the memory map provided by the bootloader, marking firmware-reserved and

kernel-occupied regions as allocated. After initialization, all kernel heap allocations and virtual memory mappings draw from the buddy allocator's pool.

```
// VFS: trait for file system drivers
trait FileSystem {
    fn read(&self, inode: &Inode, buf: &mut [u8], offset: u64) -> Result[usize, KernelError]
    fn write(&mut self, inode: &Inode, buf: &[u8], offset: u64) -> Result[usize, KernelError]
    fn lookup(&self, dir: &Inode, name: str) -> Result[Inode, KernelError]
    fn readdir(&self, dir: &Inode) -> Result[list[DirEntry], KernelError]
    fn create(&mut self, dir: &Inode, name: str, kind: InodeKind) -> Result[Inode, KernelError]
    fn unlink(&mut self, dir: &Inode, name: str) -> Result[(), KernelError]
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

59. Security: KASLR

OS performance optimization requires careful attention to hardware characteristics. Modern CPUs execute instructions out-of-order and speculatively, making the cost of a branch misprediction 15-20 cycles. Memory accesses are dominated by cache behavior: an L1 hit takes ~4 cycles, an L2 hit ~12, an L3 hit ~40, and a DRAM access ~200. The kernel's hot paths -- interrupt dispatch, system call entry, scheduler context switch -- are heavily optimized to minimize cache misses and branch mispredictions. LateralusOS's scheduler data structures are designed to fit in cache: the per-CPU run queue and per-thread control block are sized to avoid cache line splits, and frequently accessed fields are grouped together.

Writing an operating system in a high-level language was considered impractical for decades because high-level languages required garbage collectors, runtime overhead, and standard libraries that themselves depended on the OS. Lateralus's freestanding mode changes this calculus: the language's core -- arithmetic, pattern matching, closures, traits, and ownership -- is available with zero runtime dependencies. The standard library is optional and modular. A kernel written in Lateralus gets memory safety, trait-based polymorphism, and expressive pattern matching without paying any runtime tax.

The x86-64 boot sequence begins long before the kernel runs. Firmware (UEFI or legacy BIOS) initializes hardware, selects a boot device, loads the bootloader, and jumps to it in a specific CPU mode. For UEFI-based systems, the bootloader runs in 64-bit long mode with a flat memory map provided by the firmware. The kernel must still configure its own page tables, set up the interrupt descriptor table, initialize the APIC interrupt controller, and map its own code and data before it is ready to run Lateralus-level code. LateralusOS uses a minimal assembly stub for this pre-kernel setup, transitioning to Lateralus as early as possible.

Implementation Details

The physical memory allocator is the foundation of all kernel memory management. It must track which physical pages are free and which are in use, supporting allocation requests of various sizes. LateralusOS uses a buddy allocator: physical memory is organized in power-of-two blocks, and splitting and merging of blocks is O(log n) in the memory size. The allocator is initialized by parsing the memory map provided by the bootloader, marking firmware-reserved and kernel-occupied regions as allocated. After initialization, all kernel heap allocations and virtual memory mappings draw from the buddy allocator's pool.

Virtual memory allows the kernel to present each process with the illusion of a private, contiguous address space independent of the physical layout of RAM. The MMU translates virtual addresses to physical addresses using a hierarchy of page tables. On x86-64, the hierarchy is four levels deep: PML4, PDPT, PD, PT. Each level is a 4KB page

containing 512 8-byte entries. A virtual address is split into five fields: the PML4 index, the PDPT index, the PD index, the PT index, and the page offset. Only present pages have valid translations; an access to a non-present page triggers a page fault, which the kernel handles by mapping the page or terminating the process.

```
// RISC-V SBI call wrapper
fn sbi_call(eid: u32, fid: u32, a0: u64, a1: u64, a2: u64) -> SbiReturn {
    // Place args in registers a0-a6, ecall into M-mode firmware
    let (error, value): (i64, i64)
    asm {
        "li a7, {eid}"
        "li a6, {fid}"
        "mv a0, {a0}"
        "mv a1, {a1}"
        "mv a2, {a2}"
        "ecall"
        "mv {error}, a0"
        "mv {value}, a1"
    }
    return SbiReturn { error, value }
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

60. Security: SMEP/SMAP

Writing an operating system in a high-level language was considered impractical for decades because high-level languages required garbage collectors, runtime overhead, and standard libraries that themselves depended on the OS. Lateralus's freestanding mode changes this calculus: the language's core -- arithmetic, pattern matching, closures, traits, and ownership -- is available with zero runtime dependencies. The standard library is optional and modular. A kernel written in Lateralus gets memory safety, trait-based polymorphism, and expressive pattern matching without paying any runtime tax.

The x86-64 boot sequence begins long before the kernel runs. Firmware (UEFI or legacy BIOS) initializes hardware, selects a boot device, loads the bootloader, and jumps to it in a specific CPU mode. For UEFI-based systems, the bootloader runs in 64-bit long mode with a flat memory map provided by the firmware. The kernel must still configure its own page tables, set up the interrupt descriptor table, initialize the APIC interrupt controller, and map its own code and data before it is ready to run Lateralus-level code. LateralusOS uses a minimal assembly stub for this pre-kernel setup, transitioning to Lateralus as early as possible.

The physical memory allocator is the foundation of all kernel memory management. It must track which physical pages are free and which are in use, supporting allocation requests of various sizes. LateralusOS uses a buddy allocator: physical memory is organized in power-of-two blocks, and splitting and merging of blocks is O(log n) in the memory size. The allocator is initialized by parsing the memory map provided by the bootloader, marking firmware-reserved and kernel-occupied regions as allocated. After initialization, all kernel heap allocations and virtual memory mappings draw from the buddy allocator's pool.

Implementation Details

Virtual memory allows the kernel to present each process with the illusion of a private, contiguous address space independent of the physical layout of RAM. The MMU translates virtual addresses to physical addresses using a hierarchy of page tables. On x86-64, the hierarchy is four levels deep: PML4, PDPT, PD, PT. Each level is a 4KB page containing 512 8-byte entries. A virtual address is split into five fields: the PML4 index, the PDPT index, the PD index,

the PT index, and the page offset. Only present pages have valid translations; an access to a non-present page triggers a page fault, which the kernel handles by mapping the page or terminating the process.

The kernel scheduler determines which thread runs on which CPU at each moment. LateralusOS implements a Completely Fair Scheduler (CFS) variant inspired by Linux's CFS. Each thread has a virtual runtime -- the total CPU time it has consumed, weighted by its priority. The scheduler always runs the thread with the lowest virtual runtime, which ensures fairness over time. Priority is implemented as a weight: a high-priority thread accumulates virtual runtime more slowly, so it gets proportionally more CPU time. SMP support requires a per-CPU run queue to avoid lock contention, with periodic load balancing to migrate threads between CPUs.

```
// Kernel entry point (after assembly stub sets up stack)
@no_mangle
pub fn kernel_main(boot_info: &BootInfo) {
    serial_init()
    println("[BOOT] LateralusOS kernel_main entered")
    memory::init(boot_info.memory_map)
    interrupts::init_idt()
    apic::init()
```

```
    scheduler::init()
    vfs::init()
    drivers::init_all()
    println("[BOOT] All subsystems initialized")
    scheduler::start()  // does not return
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

61. Security: Seccomp

The x86-64 boot sequence begins long before the kernel runs. Firmware (UEFI or legacy BIOS) initializes hardware, selects a boot device, loads the bootloader, and jumps to it in a specific CPU mode. For UEFI-based systems, the bootloader runs in 64-bit long mode with a flat memory map provided by the firmware. The kernel must still configure its own page tables, set up the interrupt descriptor table, initialize the APIC interrupt controller, and map its own code and data before it is ready to run Lateralus-level code. LateralusOS uses a minimal assembly stub for this pre-kernel setup, transitioning to Lateralus as early as possible.

The physical memory allocator is the foundation of all kernel memory management. It must track which physical pages are free and which are in use, supporting allocation requests of various sizes. LateralusOS uses a buddy allocator: physical memory is organized in power-of-two blocks, and splitting and merging of blocks is O(log n) in the memory size. The allocator is initialized by parsing the memory map provided by the bootloader, marking firmware-reserved and kernel-occupied regions as allocated. After initialization, all kernel heap allocations and virtual memory mappings draw from the buddy allocator's pool.

Virtual memory allows the kernel to present each process with the illusion of a private, contiguous address space independent of the physical layout of RAM. The MMU translates virtual addresses to physical addresses using a hierarchy of page tables. On x86-64, the hierarchy is four levels deep: PML4, PDPT, PD, PT. Each level is a 4KB page containing 512 8-byte entries. A virtual address is split into five fields: the PML4 index, the PDPT index, the PD index, the PT index, and the page offset. Only present pages have valid translations; an access to a non-present page triggers a page fault, which the kernel handles by mapping the page or terminating the process.

Implementation Details

The kernel scheduler determines which thread runs on which CPU at each moment. LateralusOS implements a Completely Fair Scheduler (CFS) variant inspired by Linux's CFS. Each thread has a virtual runtime -- the total CPU time it has consumed, weighted by its priority. The scheduler always runs the thread with the lowest virtual runtime, which ensures fairness over time. Priority is implemented as a weight: a high-priority thread accumulates virtual runtime more slowly, so it gets proportionally more CPU time. SMP support requires a per-CPU run queue to avoid lock contention, with periodic load

balancing to migrate threads between CPUs.

Interrupt handling is the mechanism by which hardware events reach the kernel. When a keyboard is pressed, a timer fires, or a disk read completes, the hardware asserts an interrupt request line. The APIC (Advanced Programmable Interrupt Controller) arbitrates between multiple interrupt sources and delivers them to a CPU, which saves its current state and jumps to the appropriate interrupt handler via the IDT (Interrupt Descriptor Table). Interrupt handlers must be fast, non-blocking, and careful about which kernel locks they acquire. LateralusOS's type system enforces these constraints: functions marked as interrupt handlers are restricted to a subset of the kernel API.

```
// Page table entry construction
const PAGE_PRESENT:  u64 = 1 << 0
const PAGE_WRITABLE: u64 = 1 << 1
const PAGE_USER:     u64 = 1 << 2
const PAGE_NOEXEC:   u64 = 1 << 63

fn make_pte(phys: u64, flags: u64) -> u64 {
    // Align to 4KB boundary, set flags
    return (phys & 0xFFFF_FFFF_F000) | flags
}

fn map_page(virt: u64, phys: u64, writable: bool) {
    let flags = PAGE_PRESENT | (if writable { PAGE_WRITABLE } else { 0 })
    PT_ROOT.insert(virt, make_pte(phys, flags))
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

62. OS Performance Benchmarking

The physical memory allocator is the foundation of all kernel memory management. It must track which physical pages are free and which are in use, supporting allocation requests of various sizes. LateralusOS uses a buddy allocator: physical memory is organized in power-of-two blocks, and splitting and merging of blocks is O(log n) in the memory size. The allocator is initialized by parsing the memory map provided by the bootloader, marking firmware-reserved and kernel-occupied regions as allocated. After initialization, all kernel heap allocations and virtual memory mappings draw from the buddy allocator's pool.

Virtual memory allows the kernel to present each process with the illusion of a private, contiguous address space independent of the physical layout of RAM. The MMU translates virtual addresses to physical addresses using a hierarchy of page tables. On x86-64, the hierarchy is four levels deep: PML4, PDPT, PD, PT. Each level is a 4KB page containing 512 8-byte entries. A virtual address is split into five fields: the PML4 index, the PDPT index, the PD index, the PT index, and the page offset. Only present pages have valid translations; an access to a non-present page triggers a page fault, which the

kernel handles by mapping the page or terminating the process.

The kernel scheduler determines which thread runs on which CPU at each moment. LateralusOS implements a Completely Fair Scheduler (CFS) variant inspired by Linux's CFS. Each thread has a virtual runtime -- the total CPU time it has consumed, weighted by its priority. The scheduler always runs the thread with the lowest virtual runtime,

which ensures fairness over time. Priority is implemented as a weight: a high-priority thread accumulates virtual runtime more slowly, so it gets proportionally more CPU time. SMP support requires a per-CPU run queue to avoid lock contention, with periodic load balancing to migrate threads between CPUs.

Implementation Details

Interrupt handling is the mechanism by which hardware events reach the kernel. When a keyboard is pressed, a timer fires, or a disk read completes, the hardware asserts an interrupt request line. The APIC (Advanced Programmable Interrupt Controller) arbitrates between multiple interrupt sources and delivers them to a CPU, which saves its current state and jumps to the appropriate interrupt handler via the IDT (Interrupt Descriptor Table). Interrupt handlers must be fast, non-blocking, and careful about which kernel locks they acquire. LateralusOS's type system enforces these constraints: functions marked as interrupt handlers are restricted to a subset of the kernel API.

The Virtual File System (VFS) layer provides a uniform interface to all file systems. Applications use the same system calls (open, read, write, close, stat) regardless of whether the underlying storage is a local disk, a network share, a FUSE user-space file system, or a synthetic file system like /proc. The VFS layer defines abstract types: Inode (a file or directory), Dentry (a name-to-inode mapping), and File (an open file descriptor). Concrete file system drivers implement these abstractions. LateralusOS supports FAT32 for boot media compatibility and ext2 for general storage, with the VFS layer making both appear identical to user programs.

```
// CFS scheduler -- pick thread with min vruntime
fn schedule(cpu: &mut CpuState) {
    let next = cpu.run_queue
        |> min_by(fn(t) { t.vruntime })  // O(log n) with red-black tree
    match next {
        Some(thread) => {
            cpu.run_queue.remove(thread.id)
            cpu.current = thread
            context_switch(cpu.current, thread)
        },
        None => { cpu.current = IDLE_THREAD },
    }
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

63. OS Testing Strategy

Virtual memory allows the kernel to present each process with the illusion of a private, contiguous address space independent of the physical layout of RAM. The MMU translates virtual addresses to physical addresses using a hierarchy of page tables. On x86-64, the hierarchy is four levels deep: PML4, PDPT, PD, PT. Each level is a 4KB page containing 512 8-byte entries. A virtual address is split into five fields: the PML4 index, the PDPT index, the PD index, the PT index, and the page offset. Only present pages have valid translations; an access to a non-present page triggers a page fault, which the kernel handles by mapping the page or terminating the process.

The kernel scheduler determines which thread runs on which CPU at each moment. LateralusOS implements a Completely Fair Scheduler (CFS) variant inspired by Linux's CFS. Each thread has a virtual runtime -- the total CPU time it has consumed, weighted by its priority. The scheduler always runs the thread with the lowest virtual runtime, which ensures fairness over time. Priority is implemented as a weight: a high-priority thread accumulates virtual runtime more slowly, so it gets proportionally more CPU time. SMP support requires a per-CPU run queue to avoid lock contention, with periodic load balancing to migrate threads between CPUs.

Interrupt handling is the mechanism by which hardware events reach the kernel. When a keyboard is pressed, a timer fires, or a disk read completes, the hardware asserts an interrupt request line. The APIC (Advanced Programmable Interrupt Controller) arbitrates between multiple interrupt sources and delivers them to a CPU, which saves its current state and jumps to the appropriate interrupt handler via the IDT (Interrupt Descriptor Table). Interrupt handlers must be fast, non-blocking, and careful about which kernel locks they acquire. LateralusOS's type system enforces these constraints: functions marked as interrupt handlers are restricted to a subset of the kernel API.

Implementation Details

The Virtual File System (VFS) layer provides a uniform interface to all file systems. Applications use the same system calls (open, read, write, close, stat) regardless of whether the underlying storage is a local disk, a network share, a FUSE user-space file system, or a synthetic file system like /proc. The VFS layer defines abstract types: Inode (a file or directory), Dentry (a name-to-inode mapping), and File (an open file descriptor). Concrete file system drivers implement these abstractions. LateralusOS supports FAT32 for boot media compatibility and ext2 for general storage, with the VFS layer making both appear identical to user programs.

Device drivers are the most error-prone component of any operating system. They run in privileged kernel mode but interact with hardware that may misbehave, return unexpected data, or fail silently. LateralusOS's driver framework uses Lateralus traits to define safe interfaces: a block device driver implements the BlockDevice trait (read_sector, write_sector, sector_count), a character device implements CharDevice (read_byte, write_byte), and a network interface implements NetDevice (send_frame, recv_frame, get_mac). The trait system ensures that all required methods are implemented and that callers cannot accidentally call driver-specific methods through the generic interface.

```
// VFS: trait for file system drivers
trait FileSystem {
    fn read(&self, inode: &Inode, buf: &mut [u8], offset: u64) -> Result[usize, KernelError]
    fn write(&mut self, inode: &Inode, buf: &[u8], offset: u64) -> Result[usize, KernelError]
    fn lookup(&self, dir: &Inode, name: str) -> Result[Inode, KernelError]
    fn readdir(&self, dir: &Inode) -> Result[list[DirEntry], KernelError]
    fn create(&mut self, dir: &Inode, name: str, kind: InodeKind) -> Result[Inode, KernelError]
    fn unlink(&mut self, dir: &Inode, name: str) -> Result[(), KernelError]
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

64. OS Build System

The kernel scheduler determines which thread runs on which CPU at each moment. LateralusOS implements a Completely Fair Scheduler (CFS) variant inspired by Linux's CFS. Each thread has a virtual runtime -- the total CPU time it has consumed, weighted by its priority. The scheduler always runs the thread with the lowest virtual runtime, which ensures fairness over time. Priority is implemented as a weight: a high-priority thread accumulates virtual runtime more slowly, so it gets proportionally more CPU time. SMP support requires a per-CPU run queue to avoid lock contention, with periodic load balancing to migrate threads between CPUs.

Interrupt handling is the mechanism by which hardware events reach the kernel. When a keyboard is pressed, a timer fires, or a disk read completes, the hardware asserts an interrupt request line. The APIC (Advanced Programmable Interrupt Controller) arbitrates between multiple interrupt sources and delivers them to a CPU, which saves its current state and jumps to the appropriate interrupt handler via the IDT (Interrupt Descriptor Table). Interrupt handlers must be fast, non-blocking, and careful about which kernel locks they acquire. LateralusOS's type system enforces these constraints: functions marked as interrupt handlers are restricted to a subset of the kernel API.

The Virtual File System (VFS) layer provides a uniform interface to all file systems. Applications use the same system calls (open, read, write, close, stat) regardless of whether the underlying storage is a local disk, a network share, a FUSE user-space file system, or a synthetic file system like /proc. The VFS layer defines abstract types: Inode (a file or directory), Dentry (a name-to-inode mapping), and File (an open file descriptor). Concrete file system drivers implement these abstractions. LateralusOS supports FAT32 for boot media compatibility and ext2 for general storage, with the VFS layer making both appear identical to user programs.

Implementation Details

Device drivers are the most error-prone component of any operating system. They run in privileged kernel mode but interact with hardware that may misbehave, return unexpected data, or fail silently. LateralusOS's driver framework uses Lateralus traits to define safe interfaces: a block device driver implements the BlockDevice trait (read_sector, write_sector, sector_count), a character device implements CharDevice (read_byte, write_byte), and a network interface implements NetDevice (send_frame, recv_frame, get_mac). The trait system ensures that all required methods are implemented and that callers cannot accidentally call driver-specific methods through the generic interface.

The RISC-V instruction set architecture offers a clean, modular design that makes OS implementation particularly straightforward. The ISA is divided into standard extensions: I (integer), M (multiply/divide), A (atomic), F (single-precision float), D (double-precision float), C (compressed 16-bit instructions). The

privileged architecture defines three privilege levels: Machine (M-mode, highest), Supervisor (S-mode, for OS kernels), and User (U-mode, for applications). The RISC-V Supervisor Binary Interface (SBI) provides a standard firmware abstraction layer, allowing OS kernels to interact with platform hardware through well-defined firmware calls rather than direct hardware access.

```
// RISC-V SBI call wrapper
fn sbi_call(eid: u32, fid: u32, a0: u64, a1: u64, a2: u64) -> SbiReturn {
    // Place args in registers a0-a6, ecall into M-mode firmware
    let (error, value): (i64, i64)
    asm {
        "li a7, {eid}"
        "li a6, {fid}"
        "mv a0, {a0}"
        "mv a1, {a1}"
        "mv a2, {a2}"
        "ecall"
        "mv {error}, a0"
        "mv {value}, a1"
    }
    return SbiReturn { error, value }
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

65. Cross-Compilation Toolchain

Interrupt handling is the mechanism by which hardware events reach the kernel. When a keyboard is pressed, a timer fires, or a disk read completes, the hardware asserts an interrupt request line. The APIC (Advanced Programmable Interrupt Controller) arbitrates between multiple interrupt sources and delivers them to a CPU, which saves its current state and jumps to the appropriate interrupt handler via the IDT (Interrupt Descriptor Table). Interrupt handlers must be fast, non-blocking, and careful about which kernel locks they acquire. LateralusOS's type system enforces these constraints: functions marked as interrupt handlers are restricted to a subset of the kernel API.

The Virtual File System (VFS) layer provides a uniform interface to all file systems. Applications use the same system

calls (open, read, write, close, stat) regardless of whether the underlying storage is a local disk, a network share, a FUSE user-space file system, or a synthetic file system like /proc. The VFS layer defines abstract types: Inode (a file or directory), Dentry (a name-to-inode mapping), and File (an open file descriptor). Concrete file system drivers implement these abstractions. LateralusOS supports FAT32 for boot media compatibility and ext2 for general storage, with the VFS layer making both appear identical to user programs.

Device drivers are the most error-prone component of any operating system. They run in privileged kernel mode but interact with hardware that may misbehave, return unexpected data, or fail silently. LateralusOS's driver framework uses Lateralus traits to define safe interfaces: a block device driver implements the BlockDevice trait (read_sector, write_sector, sector_count), a character device implements CharDevice (read_byte, write_byte), and a network interface implements NetDevice (send_frame, recv_frame, get_mac). The trait system ensures that all required methods are implemented and that callers cannot accidentally call driver-specific methods through the generic interface.

Implementation Details

The RISC-V instruction set architecture offers a clean, modular design that makes OS implementation particularly straightforward. The ISA is divided into standard extensions: I (integer), M (multiply/divide), A (atomic), F (single-precision float), D (double-precision float), C (compressed 16-bit instructions). The privileged architecture defines three privilege levels: Machine (M-mode, highest), Supervisor (S-mode, for OS kernels), and User (U-mode, for applications). The RISC-V Supervisor Binary Interface (SBI) provides a standard firmware abstraction layer, allowing OS kernels to interact with platform hardware through well-defined firmware calls rather than direct hardware access.

OS performance optimization requires careful attention to hardware characteristics. Modern CPUs execute instructions out-of-order and speculatively, making the cost of a branch misprediction 15-20 cycles. Memory accesses are dominated by cache behavior: an L1 hit takes ~4 cycles, an L2 hit ~12, an L3 hit ~40, and a DRAM access ~200. The kernel's hot paths -- interrupt dispatch, system call entry, scheduler context switch -- are heavily optimized to minimize cache misses and branch mispredictions. LateralusOS's scheduler data structures are designed to fit in cache: the per-CPU run queue and per-thread control block are sized to avoid cache line splits, and frequently accessed fields are grouped together.

```
// Kernel entry point (after assembly stub sets up stack)
@no_mangle
pub fn kernel_main(boot_info: &BootInfo) {
    serial_init()
    println("[BOOT] LateralusOS kernel_main entered")
    memory::init(boot_info.memory_map)
    interrupts::init_idt()
    apic::init()
    scheduler::init()
    vfs::init()
    drivers::init_all()
    println("[BOOT] All subsystems initialized")
    scheduler::start()  // does not return
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

66. Conclusion

The Virtual File System (VFS) layer provides a uniform interface to all file systems. Applications use the same system calls (open, read, write, close, stat) regardless of whether the underlying storage is a local disk, a network share, a

FUSE user-space file system, or a synthetic file system like /proc. The VFS layer defines abstract types: Inode (a file or directory), Dentry (a name-to-inode mapping), and File (an open file descriptor). Concrete file system drivers implement these abstractions. LateralusOS supports FAT32 for boot media compatibility and ext2 for general storage, with the VFS layer making both appear identical to user programs.

Device drivers are the most error-prone component of any operating system. They run in privileged kernel mode but interact with hardware that may misbehave, return unexpected data, or fail silently. LateralusOS's driver framework uses Lateralus traits to define safe interfaces: a block device driver implements the BlockDevice trait (read_sector, write_sector, sector_count), a character device implements CharDevice (read_byte, write_byte), and a network interface implements NetDevice (send_frame, recv_frame, get_mac). The trait system ensures that all required methods are implemented and that callers cannot accidentally call driver-specific methods through the generic interface.

The RISC-V instruction set architecture offers a clean, modular design that makes OS implementation particularly straightforward. The ISA is divided into standard extensions: I (integer), M (multiply/divide), A (atomic), F (single-precision float), D (double-precision float), C (compressed 16-bit instructions). The privileged architecture defines three privilege levels: Machine (M-mode, highest), Supervisor (S-mode, for OS kernels), and User (U-mode, for applications). The RISC-V Supervisor Binary Interface (SBI) provides a standard firmware abstraction layer, allowing OS kernels to interact with platform hardware through well-defined firmware calls rather than direct hardware access.

Implementation Details

OS performance optimization requires careful attention to hardware characteristics. Modern CPUs execute instructions out-of-order and speculatively, making the cost of a branch misprediction 15-20 cycles. Memory accesses are dominated by cache behavior: an L1 hit takes ~4 cycles, an L2 hit ~12, an L3 hit ~40, and a DRAM access ~200. The kernel's hot paths -- interrupt dispatch, system call entry, scheduler context switch -- are heavily optimized to minimize cache misses and branch mispredictions. LateralusOS's scheduler data structures are designed to fit in cache: the per-CPU run queue and per-thread control block are sized to avoid cache line splits, and frequently accessed fields are grouped together.

Writing an operating system in a high-level language was considered impractical for decades because high-level languages required garbage collectors, runtime overhead, and standard libraries that themselves depended on the OS. Lateralus's freestanding mode changes this calculus: the language's core -- arithmetic, pattern matching, closures, traits, and ownership -- is available with zero runtime dependencies. The standard library is optional and modular. A kernel written in Lateralus gets memory safety, trait-based polymorphism, and expressive pattern matching without paying any runtime tax.

```
// Page table entry construction
const PAGE_PRESENT:  u64 = 1 << 0
const PAGE_WRITABLE: u64 = 1 << 1
const PAGE_USER:     u64 = 1 << 2
const PAGE_NOEXEC:   u64 = 1 << 63

fn make_pte(phys: u64, flags: u64) -> u64 {
    // Align to 4KB boundary, set flags
```

```
    return (phys & 0xFFFF_FFFF_F000) | flags
}

fn map_page(virt: u64, phys: u64, writable: bool) {
    let flags = PAGE_PRESENT | (if writable { PAGE_WRITABLE } else { 0 })
    PT_ROOT.insert(virt, make_pte(phys, flags))
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

67. Appendix: System Call Reference

Device drivers are the most error-prone component of any operating system. They run in privileged kernel mode but interact with hardware that may misbehave, return unexpected data, or fail silently. LateralusOS's driver framework uses Lateralus traits to define safe interfaces: a block device driver implements the BlockDevice trait (read_sector, write_sector, sector_count), a character device implements CharDevice (read_byte, write_byte), and a network interface implements NetDevice (send_frame, recv_frame, get_mac). The trait system ensures that all required methods are implemented and that callers cannot accidentally call driver-specific methods through the generic interface.

The RISC-V instruction set architecture offers a clean, modular design that makes OS implementation particularly straightforward. The ISA is divided into standard extensions: I (integer), M (multiply/divide), A (atomic), F (single-precision float), D (double-precision float), C (compressed 16-bit instructions). The privileged architecture defines three privilege levels: Machine (M-mode, highest), Supervisor (S-mode, for OS kernels), and User (U-mode, for applications). The RISC-V Supervisor Binary Interface (SBI) provides a standard firmware abstraction layer, allowing OS kernels to interact with platform hardware through well-defined firmware calls rather than direct hardware access.

OS performance optimization requires careful attention to hardware characteristics. Modern CPUs execute instructions out-of-order and speculatively, making the cost of a branch misprediction 15-20 cycles. Memory accesses are dominated by cache behavior: an L1 hit takes ~4 cycles, an L2 hit ~12, an L3 hit ~40, and a DRAM access ~200. The kernel's hot paths -- interrupt dispatch, system call entry, scheduler context switch -- are heavily optimized to minimize cache misses and branch mispredictions. LateralusOS's scheduler data structures are designed to fit in cache: the per-CPU run queue and per-thread control block are sized to avoid cache line splits, and frequently accessed fields are grouped together.

Implementation Details

Writing an operating system in a high-level language was considered impractical for decades because high-level languages required garbage collectors, runtime overhead, and standard libraries that themselves depended on the OS. Lateralus's freestanding mode changes this calculus: the language's

core -- arithmetic, pattern matching, closures, traits, and ownership -- is available with zero runtime dependencies. The standard library is optional and modular. A kernel written in Lateralus gets memory safety, trait-based polymorphism, and expressive pattern matching without paying any runtime tax.

The x86-64 boot sequence begins long before the kernel runs. Firmware (UEFI or legacy BIOS) initializes hardware, selects a boot device, loads the bootloader, and jumps to it in a specific CPU mode. For UEFI-based systems, the bootloader runs in 64-bit long mode with a flat memory map provided by the firmware. The kernel must still configure its own page tables, set up the interrupt descriptor table, initialize the APIC interrupt controller, and map its own code and data before it is ready to run Lateralus-level code. LateralusOS uses a minimal assembly stub for this pre-kernel setup, transitioning to Lateralus as early as possible.

```
// CFS scheduler -- pick thread with min vruntime
fn schedule(cpu: &mut CpuState) {
    let next = cpu.run_queue
        |> min_by(fn(t) { t.vruntime })  // O(log n) with red-black tree
    match next {
        Some(thread) => {
            cpu.run_queue.remove(thread.id)
            cpu.current = thread
            context_switch(cpu.current, thread)
        },
        None => { cpu.current = IDLE_THREAD },
    }
}
```

Subsystem Lines of Lateralus Lines of ASM Dependencies

Bootloader 620 320 none

Memory mgr 1,840 0 bootloader

Scheduler 1,230 0 memory mgr

Interrupt ctrl 540 20 none

VFS layer 2,100 0 memory mgr

FAT32 driver 980 0 VFS layer

Network stack 3,400 0 memory mgr

Lateralus is an open-source, zero-dependency programming language. Project home: https://lateralus.dev. Source: github.com/bad-antics/lateralus-lang. Released under CC BY 4.0.